UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Escola d'Enginyeria de Barcelona Est

TREBALL FI DE GRAU

Grau en Enginyeria Elèctrica

CONDUCCIÓ D'UN VEHICLE AUTÒNOM MITJANÇANT TÈCNIQUES D'INTEL·LIGÈNCIA ARTIFICIAL



Memòria i Annexes

Autor: Albert Sagré Martínez
Director: Joaquim Blesa Izquierdo
Co-Director: Pedro Ponsa Asensio

Convocatòria: Gener 2023

Resum

L'objectiu d'aquest treball final de grau (TFG) és la implementació d'intel·ligència artificial en un vehicle per aconseguir un nivell de conducció autònoma elevat, amb capacitat per circular sense interacci externa per un circuit, aplicant control dinàmic de velocitat i esquivant obstacles en la ruta.

Així, es farà ús del kit "JetRacer Pro AI Kit" de la marca Waveshare, que consisteix en un vehicle radiocontrol equipat amb càmera i una placa Jetson Nano de Nvidia, capaç de realitzar càlculs complexes que permeten la implementació d'intel·ligència artificial.

La programació del vehicle es realitzarà utilitzant el llenguatge Python a través de la plataforma Jupyterlab, que permet una connexió remota al vehicle. L'aprenentatge del vehicle es durà a terme mitjançant xarxes neuronals, destacant entre elles Resnet18 per la seva eficiència i temps d'entrenament.

Finalment, gràcies al codi creat i a l'ús de Resnet18, s'aconsegueix que el vehicle compleixi les expectatives, sent capaç de circular sense intervenció externa, regulant la seva velocitat de forma autònoma i esquivant obstacles a la ruta, tot mantenint una supervisió remota de totes les variables.

Cal destacar que aquest projecte obre la porta a altres possibles projectes per aprofundir en els aspectes tractats, ja sigui optimitzant la resposta del vehicle i del model o millorant el coneixement en xarxes neuronals i intel·ligència artificial.

Paraules clau: *conducció autònoma, Jetracer, Jetson nano, Nvidia, càmera, Python, xarxes neuronals, intel·ligència artificial, Resnet18.*

Resumen

El objetivo de este trabajo final de grado (TFG) es la implementación de inteligencia artificial en un vehículo para conseguir un nivel de conducción autónoma elevado, con capacidad para circular sin interacción externa por un circuito, aplicando control dinámico de velocidad y esquivando obstáculos en la ruta.

Así, se hará uso del kit "JetRacer Pro AI Kit" de la marca Waveshare, que consiste en un vehículo radiocontrol equipado con cámara y una placa Jetson Nano de Nvidia, capaz de realizar cálculos complejos que permiten la implementación de inteligencia artificial.

La programación del vehículo se realizará utilizando el lenguaje Python a través de la plataforma Jupyterlab, que permite una conexión remota al vehículo. El aprendizaje del vehículo se llevará a cabo mediante redes neuronales, destacando entre ellas Resnet18 por su eficiencia y tiempo de entrenamiento.

Por último, gracias al código creado y al uso de Resnet18, se consigue que el vehículo cumpla las expectativas, siendo capaz de circular sin intervención externa, regulando su velocidad de forma autónoma y esquivando obstáculos en la ruta, manteniendo una supervisión remota de todas las variables.

Cabe destacar que este proyecto abre la puerta a otros posibles proyectos para profundizar en los aspectos tratados, ya sea optimizando la respuesta del vehículo y el modelo o mejorando el conocimiento en redes neuronales e inteligencia artificial.

Palabras clave: *conducción autónoma, Jetracer, Jetson nano, Nvidia, cámara, Python, redes neuronales, inteligencia artificial, Resnet18.*

Abstract

The objective of this final degree project (TFG) is the implementation of artificial intelligence in a vehicle to achieve a high level of autonomous driving, with the ability to drive without external interaction on a circuit, applying dynamic speed control and avoiding obstacles on the route.

Thus, "JetRacer Pro AI Kit" from Waveshare will be used, which consists of a radio-controlled vehicle equipped with a camera and a Jetson Nano board from Nvidia, capable of performing complex calculations that allow the implementation of artificial intelligence.

Python will be the programming language used in this project through Jupyterlab platform, which allows a remote connection to the vehicle. Neural networks will be in charge of the learning process, highlighting among them Resnet18 for its efficiency and training time.

Finally, thanks to the created code and the use of Resnet18, it is possible for the vehicle to meet expectations, being able to run through the course without external intervention, changing its speed autonomously and avoiding obstacles on the route, maintaining remote supervision of all variables.

It should be noted that this project opens a wide variety of possibilities to other projects to deepen the aspects covered, either optimizing the response of the vehicle and the model or improving knowledge in neural networks and artificial intelligence.

Keywords: *autonomous driving, Jetracer, Jetson nano, Nvidia, camera, Python, neural networks, artificial intelligence, Resnet18.*

Agraïments

En primer lloc vull agrair el meus tutors Joaquim Blesa i Pedro Ponsa per brindar-me la possibilitat de realitzar aquest treball que m'ha permès conèixer amb major profunditat el que espero que sigui el meu futur professional. Gràcies pels recursos, els consells i la orientació per a poder dur a terme aquest projecte de la millor forma possible.

En segon lloc agrair l'ajuda rebuda per part dels mestres de laboratori, tant en el muntatge com en la connexió del vehicle a la xarxa de la universitat.

També vull agrair especialment a la meva família el suport que he rebut tota la vida i, en concret, en aquesta última etapa de la carrera. Gràcies per la confiança, la motivació, el recolzament i la voluntat de perseverar per poder-me ajudar a arribar on soc ara, tant a nivell personal, com a nivell professional.

En últim lloc, agrair a totes aquelles persones que han col·laborat en aquest projecte sigui de forma directe o indirecte.

Glossari

API(*Application Programming Interface*): interfície que permet la visualització de paràmetres i la interacció entre programes.

Capa convolucional: tipus de capa de xarxes neuronals utilitzada per l'anàlisi d'imatges o dades en dos dimensions. Consisteix en l'aplicació d'un filtre a una imatge o un de mapa de característiques que serveixen per detectar patrons o característiques específiques.

CNN*(Convolutional Neural Network)*: xarxa neuronal especialment dissenyada per processar dades en dos dimensions. Solen ser utilitzades en tasques d'anàlisi d'imatges, ja que poden capturar relacions espacials i textures que són difícils de detectar mitjançant altres tècniques.

Deadlines: terminis o dates límit establerts per a completar una tasca o projecte.

Deep Learning: tècnica d'aprenentatge automàtic que utilitza xarxes neuronals profundes amb la finalitat de realitzar prediccions o prendre decisions.

DNN*(Deep Neural Network):* xarxa neuronal profunda amb capes ocultes entre l'entrada i la sortida. Es poden entrenar amb grans conjunts de dades etiquetades i són tenen gran éxit en tasques com la classificació d'imatges, la generació de text o la predicció de resultats.

Driverless: terme que fa referència a vehicles que poden circular de forma autònoma, sense la necessitat d'un conductor humà.

EPOCH: es una iteració completa a través del conjunt de dades d'entrenament.

***Framework*:** conjunt llibreries de codi que es poden utilitzar per a desenvolupar aplicacions o programari. Aquestes funcions i classes tenen un conjunt d'eines i funcionalitats comuns que es poden utilitzar en molts projectes diferents, reduint el temps de desenvolupament i estandarditzant les aplicacions.

GPU*(Graphic Processing Unit):* circuits integrats dissenyats per accelerar les operacions matemàtiques que es fan servir en aplicacions gràfiques. Posseïen una potència de càlcul major a les que tenen els processadors i major capacitat per processar grans quantitats de dades ja que permeten el treball en paral·lel.

IA: intel·ligència artificial. Capacitat de les màquines per realitzar tasques que s'atribueixen a la intel·ligència humana com el raonament o la resolució de problemes.

LiDAR(*Light Detection and Ranging*): és un sistema de mesura per mitjà de la radiació làser. Consta d'un emissor que envia una sèrie de ràfegues i un receptor que detecta la radiació reflectida. Coneixent la diferència de temps entre l'emissió del pols làser i la seva detecció, es pot calcular la distància fins a l'objecte.

Machine Learning: aprenentatge automàtic. Es basa en la construcció de models matemàtics amb un conjunt de dades anomenat ''dades d'entrenament'' i un conjunt de "dades de validació". Els models poden ser de tipus supervisat(dades d'entrenament amb parella d'entrada-sortida) o no supervisat(les dades d'entrenament no inclouen cap tipus d'etiqueta).

Overfitting: paràmetre que serveix per definir la problemàtica existent en un model de *machine learning* per un excés d'entrenament, provocant unes prediccions pobres en modificar-se lleument les dades d'entrada. Això no permet que la xarxa neuronal generalitzi.

SLAM(*Simultaneous localization and mapping):* és una tècnica utilitzada per robots i vehicles autònoms per construir un mapa d'un entorn desconegut. Aquest mapa permet realitzara representacions 3D i traçar una predicció de la trajectòria a seguir a dins d'aquest entorn.

SoC *(State of Charge):* nivell de càrrega d'una bateria en relació a la seva capacitat. Les unitats de SoC són percentatges (0% = buit; 100% = ple).

SoH *(State of Health):* paràmetre que serveix per valorar l'estat d'una bateria comparant-la amb les condicions ideals. Les unitats de SoH són percentatges (100% = l'estat de la bateria coincideix amb les seves especificacions). En molts àmbits, i especialment en automoció, quan aquest valor es troba per sota del 80% es considera que la bateria s'ha de substituir.

Tensor: Un tensor és un objecte matemàtic utilitzat en la computació científica per a representar dades multidimensionals. En Pytorch, els tensors són objectes que es fan servir per a representar dades que es volen processar amb la GPU. Aquest concepte és molt important en el context del *machine learning* ja que moltes operacions es basen en operacions amb tensors.

Xarxa neuronal: model matemàtic inspirat en el funcionament del cervell humà, que emula un conjunt de «neurones» interconnectades a fi de processar dades d'entrada i prendre decisions basades en aquesta informació.

Índex

RESUM _____

RESUMEN _____ **I**

ABSTRACT _____ **II**

AGRAÏMENTS _____ **IV**

GLOSSARI _____ **VI**

1. PREFACI _____ **14**

1.1. Origen del treball 14

1.2. Motivació 15

1.3. Requeriments previs 15

2. INTRODUCCIÓ _____ **17**

2.1. Objectius del treball 17

2.2. Abast del treball 17

2.3. Metodologia 18

3. ESTAT DE L'ART _____ **20**

3.1. Vehicle autònom i conducció autònoma 21

3.1.1. Nivells de conducció autònoma 22

3.1.2. Dispositius de conducció autònoma 24

3.1.3. Avantatges/desavantatges de la conducció autònoma 26

3.2. Xarxes Neuronals 27

3.2.1. Xarxes neuronals aplicades a la conducció autònoma 28

4. HARDWARE _____ **29**

4.1. Projectes RC autònoms (context) 35

4.1.1. JetBot 35

4.1.2. Triton AI (*Autonomous Racing Team*) 36

4.1.3. Donkey Car 38

4.1.4. Vehicle desenvolupat a l'assignatura ISCA (Implementació de Sistemes de Control Automàtic) 39

5. SOFTWARE _____ **41**

5.1. Programari base i SO 41

5.2. Python: llibreries per a conducció autònoma 41

5.2.1. Open CV 42
5.2.2. PyTorch 42
5.2.3. Keras 42
5.2.4. TensorFlow 43
5.2.5. TensorRT 43
5.2.6. Numpy 44
5.3. Xarxes neuronals 45
5.4. Ús de llibreries i elecció de la xarxa neuronal 47

6. IMPLEMENTACIÓ 48
6.1. Jetracer Pro AI kit by Waveshare 49
6.2. Posada en funcionament 50
6.2.1. Carregar Imatge a la targeta SD 50
6.2.1. Connexió remota al vehicle mitjançant JupyterLab 53
6.2.2. Connexió i funcionament mitjançant comandament bluetooth 55
6.2.3. Procés d'obtenció de dades i entrenament de la xarxa neuronal 57
6.2.4. Model capaç de realitzar conducció autònoma 60

7. FUNCIONAMENT I RESULTATS 62
7.1. 1r Model/iteració 62
7.2. 2n Model/iteració 62
7.2.1. Implementació d'interfície de control amb regulació manual de velocitat 64
7.3. 3r Model/iteració 64
7.3.1. Implementació de codi per al control dinàmic de velocitat 66
7.4. 4t Model/iteració 66
7.4.1. Interfície de control amb visualització en temps real 66
7.4.2. Implementació parada d'emergència 67
7.4.3. Implementació control dinàmic de velocitat amb modes de velocitat 67
7.4.4. Evitar obstacles 69
7.4.5. Resum progressió iteracions 70

8. PROPOSTES DE MILLORA 71
8.1. Components 71
8.1.1. Bateria amb major capacitat 71
8.1.2. Filtre de llum anti-enlluernament 71
8.1.3. Jetson AGX Xavier 72
8.2. Funcionalitats 73
8.1.4. Millora de la interfície gràfica 73

8.2.1. Reconeixement de senyals i parada automàtica....73
8.2.2. Estat de la bateria: SoC(*State Of Charge*) i SoH (*State Of Health)* 74

9. ANÀLISI DE L'IMPACTE AMBIENTAL 75

CONCLUSIONS 76

PRESSUPOST I/O ANÀLISI ECONÒMIC 78

BIBLIOGRAFIA 79

ANNEXES 86

A1. Circuits de proves ... 86
A2. Fitxa tècnica i muntatge del kit ... 88
A3. Arxiu original *basic_motion* ... 89
A4. Arxiu original *teleoperation* ... 90
A5. Arxiu original *interactive_regressión* ... 91
A6. Arxiu original *road_following* ... 97
A7. Arxiu road_following_V2 ... 99
A8. Arxiu road_following_V3 ... 100
A9. Arxiu road_following_V4 ... 101

Índex de figures

Figura 1.1 Certificacions obtingudes dels cursos realitzats (Fonts: [3], [4]) _____ 16

Figura 2.1 Diagrama de Gantt de la planificació del projecte (Font: pròpia) _____ 19

Figura 3.1 Nivells de conducció autònoma segons SAE International (Font: [15])_____ 23

Figura 3.2 Funcionament dels sensors ultrasònics d'un vehicle (Fonts: [16], [17]) _____ 24

Figura 3.3 Imatges captades al cockpit d'un vehicle equipat amb infrarojos. (Fonts: [18], [19])__ 24

Figura 3.4 Vehicles equipats amb sistema de càmeres per a la conducció autònoma (Fonts: [20], [21]) _____ 25

Figura 3.5 Il·lustracions del funcionament d'un radar (Fonts: [22], [23])_____ 25

Figura 3.6 Mapa 3D elaborat pel sensor LiDAR de l'empresa Velodyne (Fonts: [24], [26]) _____ 26

Figura 4.1 Nvidia Jetson Nano Developer Kit (Font: []) _____ 29

Figura 4.2 Nvidia Jetson AGX Xavier Module 32GB (Fonts: [39], [40])_____ 30

Figura 4.3 Google Coral Dev Board (Fonts: [41],[42]) _____ 31

Figura 4.4 Raspberry Pi 4 Model B (Font: [43]) _____ 32

Figura 4.5 Nvidia Jetson TX2I (Font: [44]) _____ 33

Figura 4.6 JetBot AI Kit venut per l'empresa Waveshare. (Font: [45])_____ 35

Figura 4.7 SLAM Racer by Triton AI. (Font: [46]) _____ 36

Figura 4.8 JetRacer by Triton AI. (Font: [46]) _____ 37

Figura 4.9 ROS Racer by Triton AI. (Font: [46]) _____ 37

Figura 4.10 Donkey Car (Font:[47]) _____ 38

Figura 4.11 Vehicle utilitzat a l'assignatura d'ISCA. D12 Kei Truck (Font: [48]) _____ 39

Figura 6.1 JetRacer Pro AI Kit by Waveshare (Fonts: [56],[57]) _____ 49

Figura 6.2 Programari Balena Etcher (Font: pròpia) _____ 50

Figura 6.3 Escriptori d'inici del sistema de la Jetson Nano (Font: pròpia)_____ 51

Figura 6.4 Procés de connexió de la Jetson Nano a la xarxa wifi (Font: pròpia) _____ 52

Figura 6.5 Pantalla oled del vehicle (Font: pròpia) _____ 53

Figura 6.6 Imatge d'accés a JupyterLab amb el menu disponible (Font: [pròpia]) _____ 53

Figura 6.7 Exemple de la importació de llibreries i les explicacions esmentades (Font: [58])____ 54

Figura 6.8 Interfície per al reconeixement del comandament bluetooth (Font: [64]) _____ 55

Figura 6.9 Interfície de connexió del comandament bluetooth (Font: [58])_____ 56

Figura 6.10 Vinculació dels botons del comandament amb el vehicle (Font: [58])_____ 56

Figura 6.11 Codi encarregat de crear i transformar les dades provinents de la càmera (Font: [58])57

Figura 6.12 Interfície de recol·lecció de dades (Font: [58])_____ 58

Figura 6.13 Interfície d'entrenament del model (Font: [58]) _____ 59

Figura 6.14 Interfície gràfica completa del quadern interactive_regression (Font: [58]) _____ 59

Figura 6.15 Aspecte de la interfície gràfica durant l'entrenament (Font: [58])_____ 60

Figura 6.16 Codi del road_following (Font: [58]) _____ 61

Figura 7.1 Algorisme d'execució de road_following amb bucle while (Font: [58]) _____ 62

Figura 7.2 Algorisme d'execució de road_following amb funció (Font: pròpia) _____ 63

Figura 7.3 Codi i visualització de la interfície de control (Font: pròpia) _____ 64

Figura 7.4 Resposta sobre la consulta de control dinàmic de velocitat (Font: pròpia) _____ 64

Figura 7.5 Codi necessari per a la correcció de la distorsió d'imatge existent (Font: [61]) _____ 65

Figura 7.6 Aspecte de les imatges abans i després de la correcció aplicada (Font: pròpia) _____ 65

Figura 7.7 Codi inicial control dinàmic velocitat (Font: pròpia)_____ 66

Figura 7.8 Codi de la interfície gràfica final (Font: pròpia) _____ 66

Figura 7.9 Interfície gràfica completa (Font: pròpia) _____ 67

Figura 7.10 Implementació i visualització de la parada d'emergència (Font: pròpia) _____ 67

Figura 7.11 Codi encarregat de gestionar el control dinàmic de velocitat (Font: pròpia) _____ 68

Figura 7.12 Imatges de la predicció evitant obstacles (Font: [64])_____ 69

Figura 8.1 Nvidia Jetson AGX Xavier (Font: [49]) _____ 72

Figura 8.2 Interficie gràfica amb representació de paràmetres més visual (Font: [62]) _____ 73

Figura A.1 Imatge del circuit de proves inicial (Font: pròpia)_____ 86

Figura A.2 Imatges del circuit de proves final, laboratori 5.4 de la EEBE (Font: pròpia) _____ 86

Figura A.3 Dimensions del circuit de proves final (Font: pròpia) _____ 87

Figura A.4 Components i instruccions de muntatge de JetRacer Pro AI Kit by Waveshare (Font: [59]) _____ 88

1. Prefaci

1.1. Origen del treball

Aquest treball s'inspira en l'interès personal que tinc en el món de la automoció i la tecnologia, i es va formular com a idea per a ser un TFG durant el meu Erasmus, a partir de les matèries que he estudiat. Gràcies al suport dels professors Joaquim Blesa i Pere Ponsa, he pogut dur a terme aquesta iniciativa.

He treballat activament tant en el sector de la tecnologia com en el de l'automoció, i considero que els sistemes de conducció autònoma poden aportar innovació, seguretat i comoditat a la societat, i transformar radicalment la nostra forma de viure. A més, després de conduir i circular molts quilòmetres, he pogut constatar la imprevisibilitat de les persones al volant, el que sovint provoca situacions perilloses i una circulació més densa en les carreteres amb cues innecessàries.

Vaig conèixer l'existència d'aquests sistemes durant la meva estada a l'equip de Formula Student de la universitat, e-Tech Racing, on vaig descobrir la modalitat de vehicles *driverless*, que em va suscitar un gran interès. Des d'aleshores, he procurat mantenir-me informat dels avenços en aquest tema, especialment sobre les innovacions realitzades per Tesla, considerada una de les primeres empreses en innovar en conducció autònoma. El sistema de Tesla, avui dia, utilitza únicament un sistema de càmeres i visió per ordinador amb tècniques d'intel·ligència artificial per desenvolupar els seus sistemes de conducció autònoma.

A través de la programació i els sistemes de conducció autònoma, es pot reduir la perillositat en carretera a nivells mai vistos, així com la durada dels trajectes, gràcies a la previsió i la comunicació entre vehicles. A més, la implementació d'aquests sistemes podria disminuir significativament la contaminació, millorant la qualitat de vida de tota la població.

1.2. Motivació

La motivació principal per a la realització d'aquest projecte és el meu interès des de jove pel món de la automoció i la tecnologia i la intenció de formar part d'ells en un futur. El primer pas va ser començar estudiant el Cicle Superior en Tècnic d'Automoció, on vaig aprendre de forma més pràctica el funcionament dels vehicles, però vaig comprendre que les possibilitats de creixement professional d'aquesta titulació no em satisfarien.

Així doncs, un cop finalitzat el cicle superior, vaig decidir estudiar el Grau en Enginyeria Elèctrica sabent que la tracció elèctrica començava a guanyar rellevància, amb la intenció de col·laborar en el desenvolupament de sistemes que milloressin tant la tecnologia com l'automoció.

El factor més important per a la realització d'aquest Treball Final de Grau (TFG) és la meva ambició per contribuir a la generalització dels vehicles autònoms, que integra perfectament els dos mons del que m'apassiona i implica una millora en la vida de la gent. Així mateix, el fet de poder col·laborar en aquesta labor, em permetria sentir-me realitzat i deixar, d'alguna forma, una petjada en la història.

1.3. Requeriments previs

Per poder realitzar aquest treball, es recomanen alguns requisits previs:

- **Coneixements bàsics de programació:** es recomana tenir una comprensió bàsica de llenguatges de programació com Python o C++ per poder treballar amb el kit.
- **Coneixements de robòtica:** és recomanable tenir alguna experiència prèvia amb robots o sistemes autònoms per poder comprendre millor el funcionament.
- **Coneixements de maquinari:** es recomana tenir coneixements bàsics de muntatge i soldadura per poder muntar el vehicle autònom amb èxit.
- **Coneixements d'electrònica:** es recomana tenir coneixements bàsics d'electrònica per poder connectar i configurar els diferents sensors i components.

Una bona pràctica prèvia a realitzar aquest projecte seria cursar alguna assignatura relacionada amb el tema d'estudi, així com la realització d'algun curs online com a complement.

Existeixen gran quantitat de tutorials, guies i material d'aprenentatge en línia per poder ajudar a aquells que es volen iniciar en conducció autònoma i la IA.

En el meu cas, a fi de tenir una millor perspectiva per a dur a terme aquest treball, vaig realitzar els següents cursos:

- **The Complete Self-Driving Car Course - Applied Deep Learning [1]:** curs en línia disponible a Udemy que ensenya com aplicar tècniques d'aprenentatge profund per a la creació de cotxes autònoms. El curs inclou temes com el *machine learning*, la visió per computador, el processament d'imatges i la detecció d'objectes
  Exemples d'eines utilitzades en aquest curs són TensorFlow, Keras, OpenCV, i diferents conjunts de dades per a la creació de models d'aprenentatge automàtic mitjançant el llenguatge Python. També es realitzen exercicis pràctics i projectes per ajudar l'assoliment de conceptes.

- **Getting started with AI on Jetson Nano [2]**: formació oferta per NVIDIA que té com a objectiu introduir als estudiants en l'ús de la plataforma Jetson Nano per a l'execució de tasques d'aprenentatge automàtic. Aquest curs es divideix en mòduls que cobreixen temes com l'arquitectura de la plataforma, l'aprenentatge automàtic i la creació de models.

  Alguns exemples de cursos que es poden realitzar per a adquirir coneixements previs en la matèria són els següents:

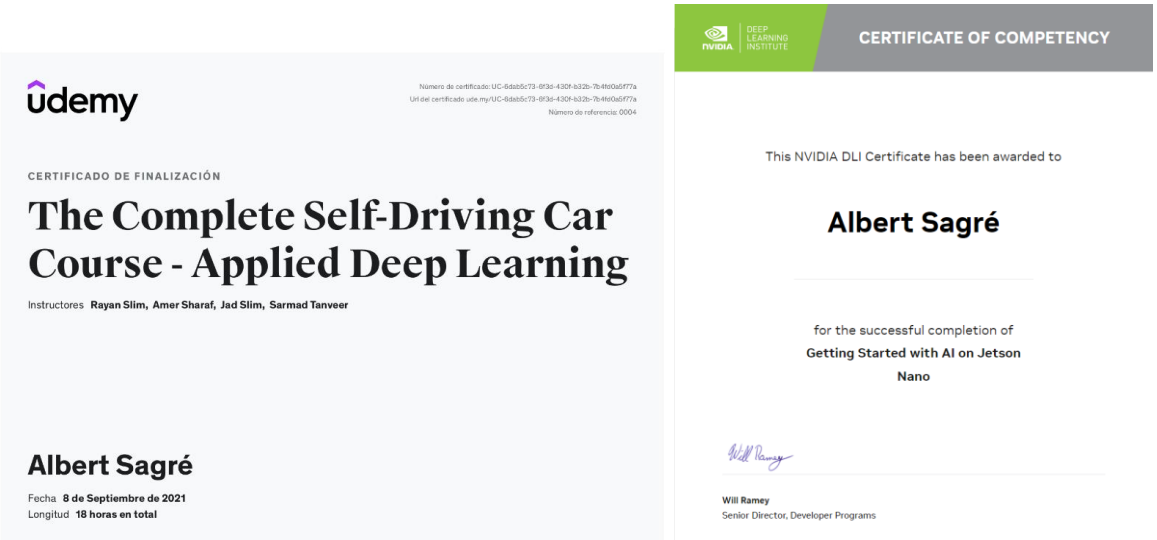


Figura 1.1 Certificacions obtingudes dels cursos realitzats (Fonts: [3], [4])

2. Introducció

Aquest projecte té com a objectiu proporcionar una aproximació a la conducció autònoma mitjançant tècniques de IA, amb la intenció de millorar la comprensió del seu funcionament i de les eines necessàries per al seu desenvolupament.

En l'actualitat, Tesla, una de les empreses més innovadores en el desenvolupament de la conducció autònoma, ha centrat els seus esforços en l'ús exclusiu de càmeres i intel·ligència artificial per dotar els seus vehicles d'aquestes capacitats. Aquest projecte planteja un enfocament similar, però a petita escala, sense sensors LiDAR o RADAR com en altres marques reconegudes.

Tot i que aquest projecte es centra exclusivament en l'ús i estudi de la conducció autònoma, actualment hi ha molts sistemes que fan ús de la intel·ligència artificial per intentar millorar el seu funcionament dia a dia. Per aquest motiu s'utilitza en molts altres sectors, trobant formes més resolutives, eficients i econòmiques de resoldre els problemes actuals.

2.1. Objectius del treball

La conducció autònoma està transformant la forma en què s'utilitza el transport. Fent ús de sensors i tècniques d'intel·ligència artificial permet als vehicles moure's de forma autònoma sense intervenció externa.

En aquest treball final de grau es pretén explorar el paper de la càmera i la intel·ligència artificial en el desenvolupament de la conducció autònoma, únicament fent ús d'aquests dos recursos. L'objectiu final és aconseguir la conducció autònoma del vehicle mitjançant l'ús de xarxes neuronals que permetin el seu aprenentatge.

2.2. Abast del treball

Aquest projecte pretén assolir la conducció autònoma mitjançant una càmera i una xarxa neuronal, seguint els passos marcats a continuació:

- Inicialment aconseguir connectar el vehicle mitjançant el comandament de control *bluetooth* i verificar que funciona correctament. Pas molt important, verificar sempre que tot funciona correctament abans de prosseguir amb l'estudi.

- Crear un banc de dades per tal d'entrenar la xarxa neuronal, que posteriorment permetrà al vehicle circular de forma autònoma. Realitzar una posterior validació del model per comprovar que funciona correctament i el seu error és petit.

- Testejar el model creat i comprovar que el vehicle és capaç de seguir el circuit sense interacció externa, evitant obstacles i regulant la seva velocitat de forma autònoma.

- Implementar una interfície que permeti observar les dades en temps real, controlar el rang de velocitat dinàmic escollit i realitzar una parada d'emergència.

2.3. Metodologia

La realització d'aquest treball ha emprat l'anomenada metodologia àgil [5] i [6], és un enfocament per a la gestió del projecte que es basa en la flexibilitat i la iteració. Es basa en la idea que el projecte es divideix en petites etapes o cicles, anomenats "*sprints*". En aquestes etapes es desenvolupen i es lliuren petites parts del projecte, fet que permet anar adaptant el projecte a les necessitats manera més ràpida i eficaç.

La metodologia àgil s'aplica sovint en el desenvolupament de software, ja que permet una major flexibilitat i capacitat d'adaptació. En aquest cas, la metodologia *agile* ha sigut útil per a la gestió del treball i la planificació dels diferents cicles, permetent ajustar i modificar els objectius en funció del parlat amb els tutors i l'evolució del projecte. Això ha afavorit a mantenir el treball enfocat en les tasques més importants i a evitar els retards i els canvis de darrera hora.

D'aquesta forma, el projecte ha quedat definit per les fases o cicles següents:

1. Elecció del kit a emprar pel projecte: - Muntatge del kit a escollir
2. Implementació del software: - Posada en funcionament
3. Investigació
    - Vehicle autònom
    - Xarxes Neuronals
    - Intel·ligència artificial
    - Altres
    - Software
4. Desenvolupament i implementació del codi:
    - 1a Iteració
    - 2a Iteració
    - 3a Iteració
    - 4a Iteració
5. Proves dels models:
    - Executar proves al circuit inicial a casa
    - Executar proves al circuit del laboratori 5.4 a la universitat
6. Elaboració de la documentació
    - Redacció de la memòria
    - Repàs de la documentació

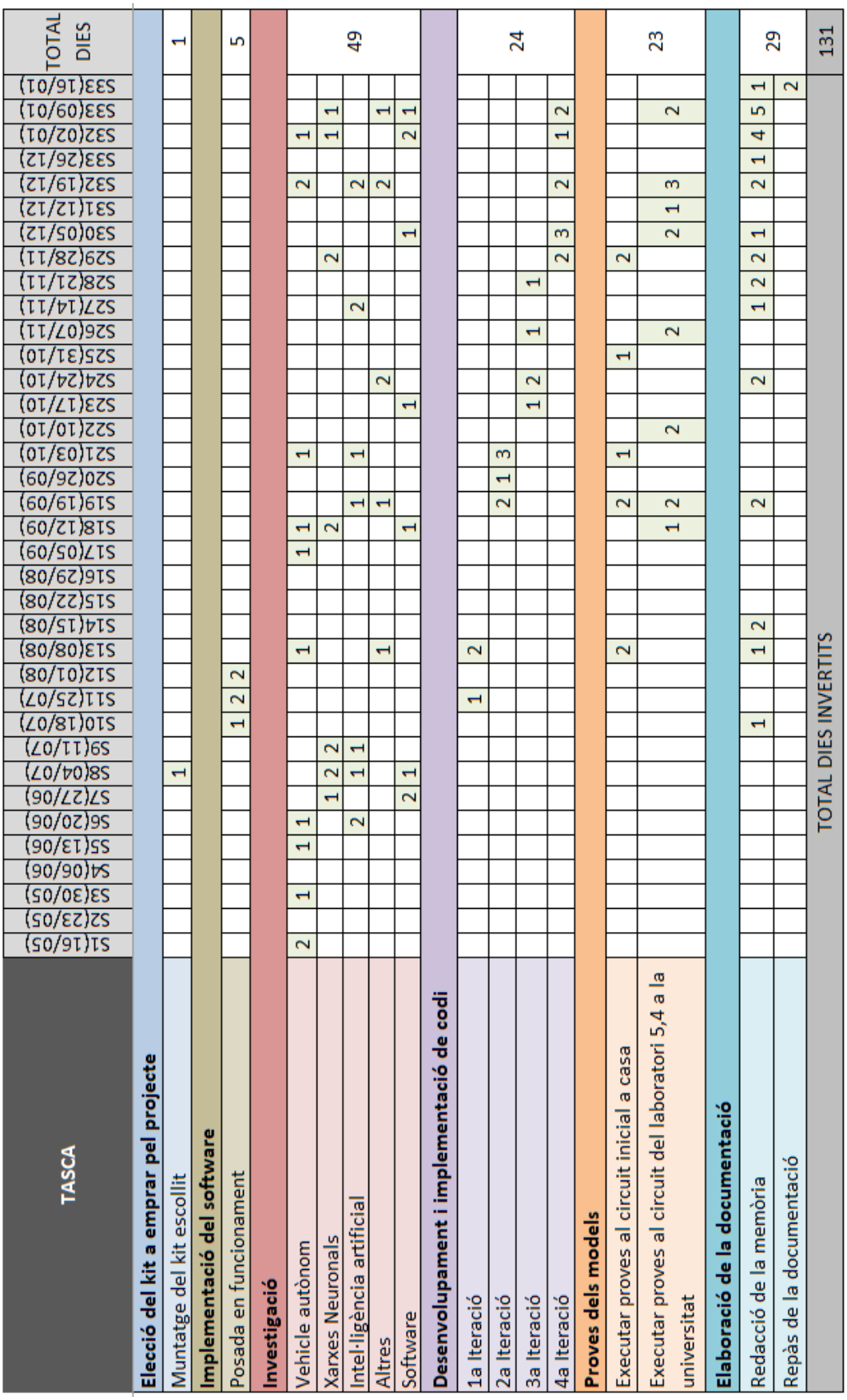


Figura 2.1 Diagrama de Gantt de la planificació del projecte (Font: pròpia)

3. Estat de l'art

Es denomina intel·ligència artificial o IA segons les seves sigles, al camp de la ciència informàtica amb capacitat per imitar la intel·ligència o el comportament humà, desenvolupant tasques com la resolució de problemes o el reconeixement de patrons.

Hi ha diversos tipus de intel·ligència artificial (IA) que de forma generalitzada es classifiquen de dues formes:

- En funció de les seves funcionalitats:

    1. **Intel·ligència artificial no adaptativa (*reactive*):** no té capacitat per modificar la seva estratègia o el seu comportament en funció de les experiències passades o les dades emmagatzemades. Únicament percep l'entorn que l'envolta i reacciona amb una tasca específica, sempre de la mateixa forma i sense produir-se modificacions d'una execució a la següent.

        Es pot utilitzar per a la conducció autònoma de nivells més baixos, com el nivell 0 o el nivell 1, en què el conductor ha de prendre les decisions i controlar el vehicle en tot moment.

    2. **Intel·ligència artificial adaptativa lleugera (*limited memory*):** és capaç de modificar lleugerament la seva estratègia en base a experiències passades o les dades recopilades. El fet de poder emmagatzemar dades de forma limitada (com la velocitat i direcció de vehicles colindants), permet realitzar una predicció acurada per decidir quin és el millor moment per canviar de carril o realitzar un avançament.

        S'associa al nivell 2 o nivell 3 de conducció autònoma, en què el vehicle és capaç de controlar la velocitat, la direcció i els moviments, però el conductor ha de mantenir la concentració i estar preparat per prendre el control en qualsevol moment. Així mateix, amb assistència d'altres sistemes i millores aquest tipus de IA pot arribar a assolir el nivell 4 d'autonomia.

    3. **Intel·ligència artificial completa (*Theory of mind*):** pot modificar totalment el seu comportament en funció de les experiències passades. Es considera que aquest nivell de IA encara és únicament un concepte tot i que s'estan realitzant avenços molt propers. En conducció autònoma implicaria un nivell d'autonomia molt elevat, com el nivell 4 i el 5, on el conductor humà ja no és necessari.

    4. **Intel·ligència artificial auto-conscient (*Self-Aware)*:** tipus de IA capaç de tenir consciencia de si mateixa, de les seves accions i les seves decisions. És considerada una de les fases finals de IA, i es troba actualment en fase de recerca. Aquesta IA implica un alt grau de

complexitat en termes de processament de dades i capacitat de raonament, requerint de la unió de diversos tipus de IA per a la seva concepció.

- En funció de les seves capacitats:

    1. **Intel·ligència artificial estreta (*narrow AI*):** tipus de IA programada per a la realització de tasques específiques, sense capacitat per generalitzar les seves habilitats a fi de realitzar altres tasques similars.

    2. **Intel·ligència artificial general (*general AI*):** IA capaç de dur a terme qualsevol tasca que pugui realitzar un ésser humà fent ús de la seva intel·ligència, sense requerir d'una programació específica per a cada tasca. Actualment es troba en desenvolupament i es considerat com un dels objectius finals de la IA.

    3. **Intel·ligència artificial superhumana (*super AI*):** es considera un nivell superior, l'evolució d'una IA general, amb una intel·ligència i capacitat de raonament per sobre de les capacitats humanes. Aquesta IA, conceptualment, podria ser utilitzada en un futur per a la resolució de problemes extremadament complexes, la previsió del futur o la generació de coneixements.

3.1. Vehicle autònom i conducció autònoma

La conducció autònoma és una tecnologia emergent que promet revolucionar la forma en què es fa servir el transport. Aquesta tecnologia, permet als vehicles moure's de forma autònoma sense intervenció externa. Això es fa possible gràcies a un conjunt de sensors i tecnologies d'intel·ligència artificial que permeten al vehicle percebre el seu entorn i prendre decisions de conducció en funció de les dades recollides. El document J3016 [14], és la referència utilitzada a la indústria automotriu per a la conducció automatitzada. Detalla els nivells en què els vehicles són capaços de circular de forma autònoma, numerats del 0 al 5, i n'explica les capacitats associades de forma breu.

Malgrat els avenços significatius en el desenvolupament de la conducció autònoma, hi ha molts reptes a superar abans que aquesta tecnologia esdevingui una realitat generalitzada. Això inclou qüestions de seguretat, acceptació social de la tecnologia i la regulació i normatives necessàries.

3.1.1. Nivells de conducció autònoma

Els nivells de conducció autònoma existents es classifiquen de 0 a 5 de la forma següent:

- **Nivell 0:** No hi ha cap tipus de conducció autònoma. El conductor és el responsable de totes les funcions de conducció.

- **Nivell 1:** Assistència a la conducció. en aquest nivell, el cotxe té alguns sistemes d'assistència que ajuden al conductor en tasques com el manteniment de la velocitat o la distància de seguretat. No obstant, el conductor ha de mantenir la concentració en tot moment i estar preparat per prendre el control en qualsevol moment.

- **Nivell 2:** Conducció parcialment autònoma: en aquest nivell, el cotxe és capaç de conduir de forma autònoma en algunes circumstàncies, com per exemple en carreteres de dos o més carrils amb marcadors de carril clars. Tanmateix, com en el cas del nivell 1, el conductor ha d'estar preparat per prendre el control en tot moment.

- **Nivell 3:** Conducció condicionada o autònoma limitada. És un nivell de conducció autònoma en què el cotxe és capaç de prendre decisions de conducció i controlar la velocitat i l'acceleració en determinades circumstàncies. En aquest nivell, el cotxe està equipat amb sensors i sistemes de control que li permeten detectar i evitar obstacles, seguir les línies de trànsit i mantenir la seva posició en la carretera. En general, el nivell 3 de conducció autònoma és considerat com un pas important cap a la conducció totalment autònoma, ja que permet que el cotxe prengui decisions de conducció sense la intervenció constant del conductor. Destacar que, com en els casos anteriors, es requereix que el conductor es mantingui atent prengui el control si és necessari.

- **Nivell 4:** Conducció altament autònoma. El vehicle és capaç de traçar la ruta, controlar el trànsit adequant la velocitat de marxa, girant i adaptant-se a les condicions de l'entorn de forma autònoma. Posseeix capacitat per respondre a multitud de situacions pel que el conductor només haurà d'intervenir quan el sistema es trobi en situacions extremes o excepcionals. En aquest nivell d'autonomia el conductor pot fer altres activitats mentre el cotxe està en conducció autònoma, però ha de ser capaç de prendre el control si el sistema es troba en situacions excepcionals on no pot seguir.

- **Nivell 5:** Conducció totalment autònoma. Es considera el nivell més elevat de conducció autònoma. No requereix de conductor en cap circumstància ja que el cotxe és capaç de conduir de forma autònoma en qualsevol escenari.

  En aquest nivell, únicament serà necessari indicar el destí i el sistema gestionarà tot el necessari, pel que no es requereix que hi hagi cap conductor. Gràcies a les característiques d'aquest nivell desapareix la necessitat de fer servir pedals i elements de control pel que es pot prescindir d'aquests si els fabricants ho desitgen.

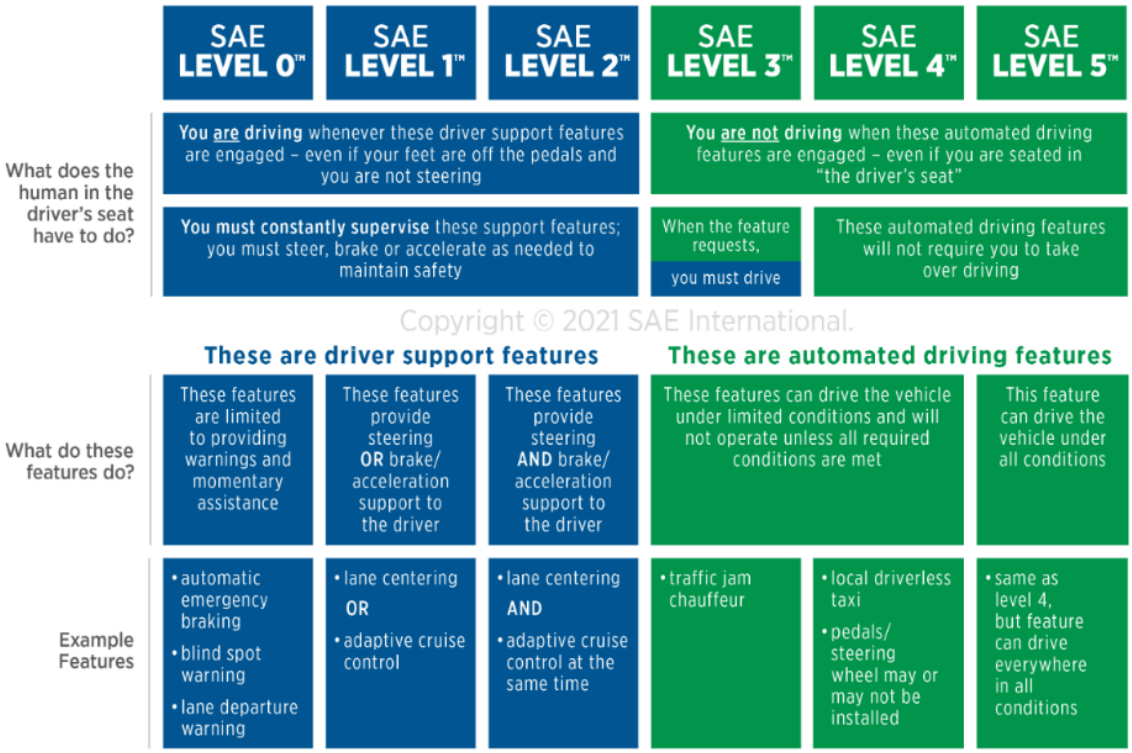


Figura 3.1 Nivells de conducció autònoma segons SAE International (Font: [15])

3.1.2. Dispositius de conducció autònoma

La conducció autònoma es basa en l'ús de dispositius de sensors i tecnologia de computació per a la captació d'informació de l'entorn i prendre decisions sobre les accions a realitzar. Alguns dels dispositius que es fan servir de forma habitual per a la conducció autònoma són:

- **Sensors ultrasònics:** utilitzats per detectar elements que es troben a poca distància del vehicle. Emeten ones sonores imperceptibles pels humans i calculen el temps que triguen en tornar al punt d'emissió.

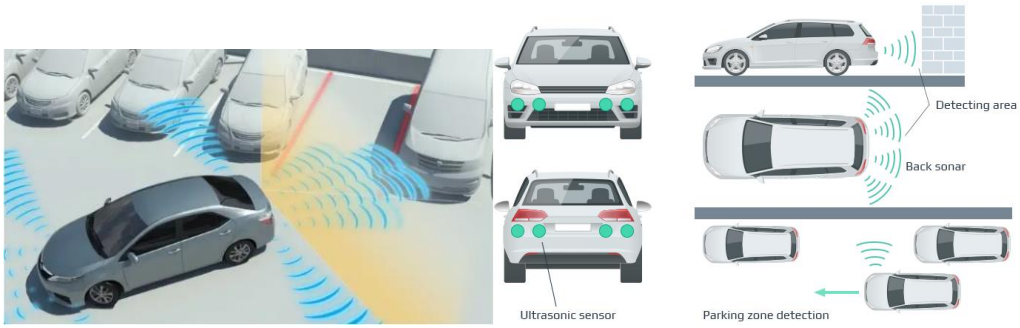


Figura 3.2 Funcionament dels sensors ultrasònics d'un vehicle (Fonts: [16], [17])

- **Posicionament i navegació per satèl·lit:** sistema de navegació global que proporciona informació sobre la ubicació i la trajectòria del vehicle. La seva precisió és un factor vital per al funcionament en els nivells més elevats de conducció autònoma. Els sistemes de que consta aquesta categoria són el GPS, GLONASS i Galileo.

- **Sistema de navegació inercial:** mitjançant l'ús de sensors de moviment i sensors giroscòpics el sistema és capaç de fer una estimació de la posició, la trajectòria i la velocitat del vehicle. Permet augmentar la precisió dels sistemes de posicionament però no suficient com per recolzar els nivells de conducció autònoma més elevats.

- **Sensors infrarojos:** aquests sensors permeten detectar i realitzar el seguiment d'objectes en circumstàncies on hi ha poca il·luminació, per això s'empren com a sistema de visió nocturna.



Figura 3.3 Imatges captades al cockpit d'un vehicle equipat amb infrarojos. (Fonts: [18], [19])

- **Sistema de càmeres:** dispositius de captació d'imatge que permeten al vehicle obtenir informació visual del seu entorn. La tecnologia de gravació esta molt desenvolupada, fet que permet equipar el vehicle amb sistema de càmeres a un cost reduït. Amb el sistema de càmeres i un software capaç de realitzar una bona interpretació de les imatges, el vehicle es capaç de confeccionar una bona resposta, permetent que a dia d'avui hi hagi vehicles com els Tesla, circulant exclusivament amb l'ús d'aquests sistema. Destacar que en condicions on hi ha baixa visibilitat o climatologia adversa l'efectivitat del sistema es veu reduïda.



Figura 3.4 Vehicles equipats amb sistema de càmeres per a la conducció autònoma (Fonts: [20], [21])

- **Radar:** mitjançant ones electromagnètiques és capaç de detectar i realitzar seguiment d'objectes. Té una precisió elevada i permet determinar la direcció i velocitat en que viatja un cos (sigui una persona, una bici, un camió, etc). La seva efectivitat no es veu reduïda en condicions de baixa visibilitat o climatologia adversa com és el cas de les càmeres.

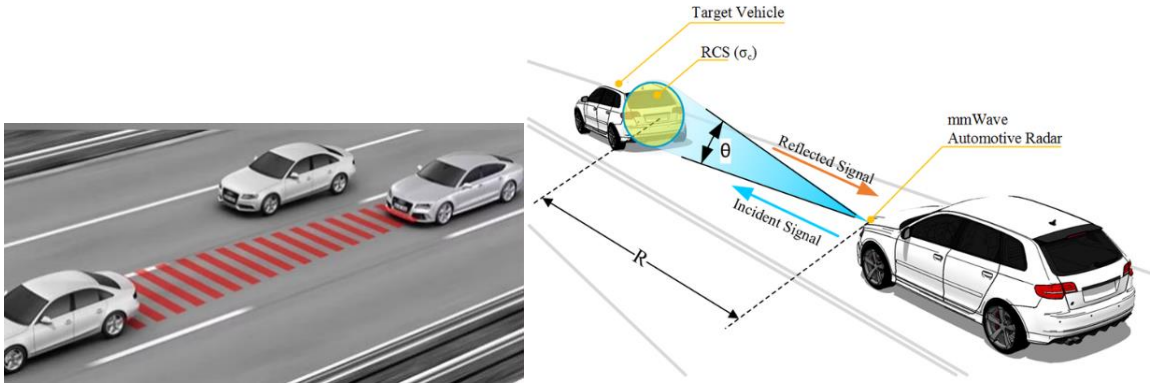


Figura 3.5 Il·lustracions del funcionament d'un radar (Fonts: [22], [23])

- **Lidar (*Laser imaging detection and ranging*):** aquest sistema s'utilitza per determinar la distància des d'un emissor fins a un objecte mitjançant pulsacions làser. És un gran complement al radar ja que és capaç d'arribar a una major distància i aconsegueix una precisió major. Tota la informació recollida al sistema lidar facilita la possibilitat de crear mapes molt detallats i imatges 3D del que veu el sistema, posicionant-se com un dels sistemes més importants per a l'assoliment de la conducció autònoma de nivell 4 i 5. Així mateix, el seu elevat cost fa que no sigui tan comú.

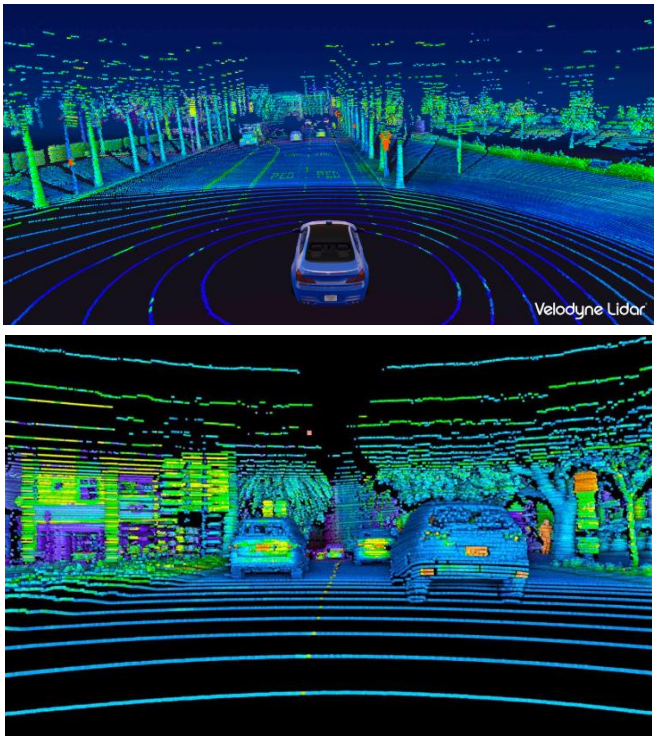


Figura 3.6 Mapa 3D elaborat pel sensor LiDAR de l'empresa Velodyne (Fonts: [24], [26])

3.1.3. Avantatges/desavantatges de la conducció autònoma

La conducció autònoma és un avenç que per la seva implicació social provoca que hi hagi gent a favor i gent en contra. A continuació es presenten alguns dels avantatges i desavantatges de la conducció autònoma més reconeguts i pels que existeixen aquestes dues percepcions:

Avantatges:

1. **Millora la seguretat:** Els cotxes autònoms poden reduir significativament el nombre d'accidents a les carreteres gràcies a la seva capacitat per detectar i evitar situacions perilloses de manera més ràpida i precisa que els humans.

2. **Eficiència del trànsit:** Els cotxes autònoms poden comunicar-se entre ells i amb els semàfors per millorar la circulació i reduir els embussos.

3. **Accessibilitat:** La conducció autònoma pot ser especialment útil per a persones que tenen dificultats per conduir, com persones amb discapacitats, persones grans, persones amb por a conduir o simplement persones que no volen conduir.

4. **Reducció de la contaminació:** Els cotxes autònoms poden ser més eficients que altres degut a la comunicació que tenen. Aquesta eficiència en el trànsit permet reduir la quantitat de temps que passen en circulació, contribuint a reduir la contaminació de l'aire.

Desavantatges:

1. **Cost:** La tecnologia de conducció autònoma encara és molt cara, fet que es veu reflectit en el cost dels vehicles que equipen aquestes prestacions.

2. **Qüestions de seguretat:** Encara hi ha preocupació sobre la seguretat de la conducció autònoma, ja que hi ha hagut alguns accidents relacionats amb aquesta tecnologia.

3. **Qüestions ètiques:** Hi ha qüestions ètiques complexes associades a la conducció autònoma, com les decisions que han de prendre els cotxes autònoms en situacions de conflicte. S'han realitzat estudis socials en que les persones havien de decidir en situacions de conflicte el "mal menor", però sempre hi ha divisió d'opinions o conflictes. Es fa responsable al programador de la decisió? Al conductor?

4. **Impacte en el treball:** La introducció de la conducció autònoma pot tenir un impacte en el treball dels conductors professionals i els transportistes, ja que pot deixar-los sense feina.

3.2. Xarxes Neuronals

Les xarxes neuronals són una part important de la intel·ligència artificial (IA). Són un tipus de model de *machine learning* que es basa en el funcionament del cervell humà per aplicar tècniques d'aprenentatge automàtic a diferents tasques.

Les xarxes neuronals consisteixen en una gran quantitat de nodes o neurones que s'organitzen en diferents capes. Cada node està connectat a altres nodes amb diferents pesos, que s'ajusten durant el procés d'aprenentatge de la xarxa neuronal. Això permet que la xarxa pugui aprendre per si mateixa i millorar els seus resultats a mesura que s'alimenta amb més dades.

Les xarxes neuronals s'han utilitzat amb èxit en una gran varietat de tasques, com la classificació d'imatges, la detecció de patrons, la predicció de series temporals i el processament del llenguatge natural. El seu èxit es deu a la capacitat que tenen per aprofitar l'estructura i les relacions dins de les dades per millorar l'exactitud de les prediccions.

Les xarxes neuronals són una part important de la IA, ja que permeten a les màquines aprendre de manera autònoma i millorar les seves prestacions amb l'experiència. A més tenen capacitat per processar grans quantitats de dades i prendre decisions basades en patrons complexos, cosa que les converteix en una eina valuosa per a moltes aplicacions pràctiques.

3.2.1. Xarxes neuronals aplicades a la conducció autònoma

Hi ha diversos tipus de xarxes neuronals que es poden utilitzar per a la conducció autònoma, cada un amb les seves pròpies característiques i aplicacions. A continuació, s'enumeren i es defineixen breument alguns dels tipus de xarxes neuronals més comuns:

1. **Xarxes neuronals *feedforward* (FNN):** aquestes són les xarxes neuronals més bàsiques i es fonamenten en l'estructura d'un sistema de entrada, processament i sortida. No tenen cap tipus de retroalimentació i són útils per a tasques de classificació o predicció.
2. **Xarxes neuronals recurrents (RNN):** xarxes neuronals que inclouen una retroalimentació interna i permeten que les dades passin d'un estat a l'altre. Són útils per a tasques que requereixen l'ús d'informació històrica, com per exemple la predicció de sèries temporals o el processament de llenguatge natural.
3. **Xarxes neuronals de múltiples capes (MLN):** aquestes xarxes neuronals tenen múltiples capes interconnectades entre si. Són utilitzades per a tasques complexes que requereixen un processament profund: identificació de patrons de dades o la classificació d'imatges es troben entre les seves aplicacions.
4. **Xarxes neuronals convolucionals (CNN):** xarxes neuronals especialitzades en el processament d'imatges, basades en la idea de "convolució". Aquest concepte és la combinació de diferents filtres per extreure característiques representatives de les imatges.

D'entre els tipus de xarxes neuronals exposades, tant les MLN com les CNN formen part del que s'anomena *deep learning* o aprenentatge profund ja que es basen en el desenvolupament de models de *machine learning* molt complexes i profunds. Fan us de diverses capes i multitud de nodes per aprendre de forma automàtica les característiques rellevants.

En aquest projecte, com ja s'ha comentat i es veurà a continuació, s'ha fet ús d'una xarxa neuronal convolucional, l'anomenada Resnet18. ResNet18 és coneguda per la seva capacitat per extreure les característiques rellevants de les imatges de manera eficient, fent-la ideal per a la detecció d'objectes en imatges amb poca llum o en situacions amb gran complexitat visual.

Amb l'ús de les eines i llibreries disponibles i la xarxa neuronal Resnet18, es considera que aquest projecte assoleix un nivell 2 de conducció autònoma. Amb algunes modificacions i l'addició d'altres dispositius o sensors seria capaç d'assolir el nivell 3 de forma satisfactòria.

4. Hardware

D'entre les alternatives existents al mercat per a dur a terme la tasca de processat per a treballar amb conducció autònoma i IA, les opcions a que es van valorar són les següents:

1. **NVIDIA Jetson Nano:** és un ordinador de placa única de baix cost, creat per Nvidia i dissenyat per a utilitzar-se en aplicacions d'IA. Té un factor de forma petit i un baix consum d'energia, motiu pel qual és en una bona opció per a utilitzar en dispositius mòbils com el vehicle d'aquest projecte.



Figura 4.1 Nvidia Jetson Nano Developer Kit (Font: [])

- **Microprocessador:** NVIDIA Jetson Nano amb quad core ARM Cortex-A57
- **GPU:** NVIDIA Maxwell amb 128 cores a 921 MHz
- **RAM:** 4GB 64-bit LPDDR4
- **Emmagatzematge:** Targeta micro SD
- **Càmeres:** x2 (USB+CSI)
- **Interfície de entrada/sortida:** HDMI, USB, Ethernet, GPIO, UART, SPI, I2S, I2C, PWM, GPIO
- **Connectivitat:** Wi-Fi, Bluetooth
- **Rendiment:** Fins a 472 GFLOPS (operacions de coma flotant per segon)
- **Consum energètic:** 5-10 W
- **Preu**: 139,90 €

2. **NVIDIA Jetson AGX Xavier :** és una plataforma d'IA d'alt rendiment, també creada per Nvidia i enfocada en l'ús en màquines autònomes, com ara robots i drones. Té un factor de forma similar a Jetson Nano i també es basa en l'arquitectura de la GPU de NVIDIA, però disposa d'una GPU i una CPU més potents, pel que pot assolir càrregues de treball d'IA més exigents.



Figura 4.2 Nvidia Jetson AGX Xavier Module 32GB (Fonts: [39], [40])

- **Microprocessador:** 8-Core ARM v8.2 64-Bit CPU, 8 MB L2 + 4 MB L3
- **GPU:** NVIDIA Volta amb 512 cores i 48 cores Tensor
- **RAM:** 32GB 256-Bit LPDDR4x| 136.5GB/s
- **Emmagatzematge:** 32 GB eMMC 5.1
- **CSI:** Fins a 16 càmeres
- **Interfície de entrada/sortida:** HDMI, USB, Ethernet, GPIO, UART, SPI, CAN, I2 C, I2 S, DMIC, GPIOs, PCIE, SLVS-EC
- **Connectivitat:** Wi-Fi, Bluetooth
- **Rendiment:** Fins a 21 TFLOPS (operacions de coma flotant per segon)
- **Consum energètic:** 10-30 W
- **Preu**: 2.138,60 €

3. **Google Coral Dev Board:** és una plataforma de desenvolupament *open source* utilitzada per a la creació de sistemes amb intel·ligència artificial (IA) i processament de dades en temps real. Es tracta d'un dispositiu de petit tamany i baix cost que es pot fer servir com a dispositiu d'acceleració per a processament de dades amb IA.



Figura 4.3 Google Coral Dev Board (Fonts: [41],[42])

- **Microprocessador:** ARM Cortex-A53 a 1.5 GHz
- **GPU:** Integrated GC7000 Lite Graphics
- **RAM:** 1-4 GB LPDDR4
- **Emmagatzematge:** 8 GB eMMC, Targeta microSD
- **Càmera:** x2 (USB+CSI)
- **Interfície de entrada/sortida:** HDMI, USB, Ethernet, GPIO
- **Connectivitat:** Wi-Fi, Bluetooth
- **Rendiment:** Fins a 4 TFLOPS (operacions de coma flotant per segon)
- **Consum energètic:** 3.3W
- **Preu**: 158,03 €

4. **Raspberry Pi:** ordinador d'una sola placa de baix cost que és popularment utilitzat en projectes educatius i d'aficionats. Té un factor de forma petit i un baix consum d'energia, com les altres opcions comparades, motiu pel qual es podria considerar una bona opció per a dur a terme aquest treball. Malauradament el rendiment d'aquest dispositiu és el menor de totes les opcions valorades.



Figura 4.4 Raspberry Pi 4 Model B (Font: [43])

- **Microprocessador:** Broadcom BCM2711 Cortex-A72 1,5GHz Quad-Core 64bits
- **GPU:** Broadcom VideoCore VI
- **RAM:** 8 GB LPDDR4
- **Emmagatzematge:** Targeta microSD
- **Interfície de entrada/sortida:** HDMI, USB, Ethernet, GPIO
- **Connectivitat:** Wi-Fi, Bluetooth
- **Rendiment**: Fins a 1 GFLOPS (operacions de coma flotant per segon)
- **Consum energètic:** 5W
- **Preu**: 95,95 €

5. **NVIDIA Jetson TX2I:** dispositiu de gamma mitjana dissenyada per el seu ús amb dispositius autònoms i aplicacions avançades d'IA. Com els altres dispositius, també té un factor de forma petit-mitjà, amb una GPU i CPU notablement més potents que Jetson Nano però un rendiment llunyà a l'assolit per Jetson AGX Xavier.

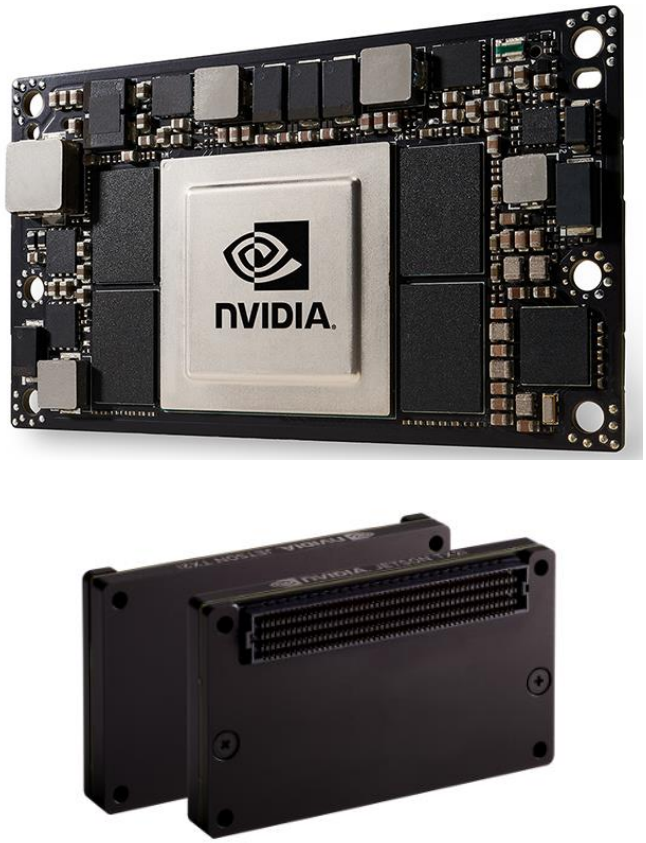


Figura 4.5 Nvidia Jetson TX2I (Font: [44])

- **Microprocessador:** NVIDIA Dual Denver2/2MB L2 + ARM Cortex-A57/2MB L2
- **GPU:** NVIDIA Pascal, 256 NVIDIA CUDA cores
- **RAM:** 8 GB 128-bit LPDDR4
- **Emmagatzematge:** 32 GB eMMC, SDIO, SATA
- **CSI:** Fins a 6 càmeres (2 vies)
- **Interfície de entrada/sortida:** HDMI, DP, USB, Ethernet, GPIO, CAN, UART, SPI, I2C, I2S, PCIE
- **Rendiment**: Fins a 1,26 TFLOPS (operacions de coma flotant per segon)
- **Consum energètic:** 10-20W
- **Preu**: 743,69 €

Coneguts els dispositius més rellevants per a treballar amb conducció autònoma i IA s'ha determinat que el dispositiu més adequat pel projecte és Jetson Nano.

Els motius que han portat a escollir aquest dispositiu són els següents:

1. **Preu:** és un dels dispositius més econòmics de la llista comparada
2. **Potència**: tot i que no té un rendiment tan alt com el NVIDIA Jetson Xavier AGX o el NVIDIA Jetson TX2I, és capaç de processar dades a una velocitat elevada. Això el fa una opció viable per a moltes aplicacions en IA i conducció autònoma.
3. **Dimensions:** és un dispositiu de tamany reduït, cosa que el fa ideal per a sistemes integrats o per a muntatge en espais reduïts, com és el cas del xassís d'un vehicle RC.
4. **Consum energètic:** té un consum energètic baix molt útil en situacions i aplicacions on els dispositius són petits, podent treballar amb bateries de menor format.
5. **Suport:** disposa de gran suport de la comunitat, amb documentació i exemples disponibles en línia i models de xarxes neuronals que es poden utilitzar directament amb el dispositiu.

4.1. Projectes RC autònoms (context)

Per a tenir una millor comprensió del context en que es troba aquest treball, a continuació es mostren diversos projectes i kits que tenen l'objectiu d'assolir la conducció autònoma mitjançant tècniques d'IA amb diferents interpretacions i sistemes per a dur a terme aquesta tasca:

4.1.1. JetBot

JetBot és un robot educatiu desenvolupat per NVIDIA basat en una placa Jetson Nano, amb una GPU que li permet processar càlculs de visió per computador i aprenentatge automàtic. Disposa d'una càmera que permet al dispositiu processar imatges i vídeo en temps real i un gran compatibilitat de sensors, que permeten al robot detectar i respondre a canvis en la seva orientació i moviment.

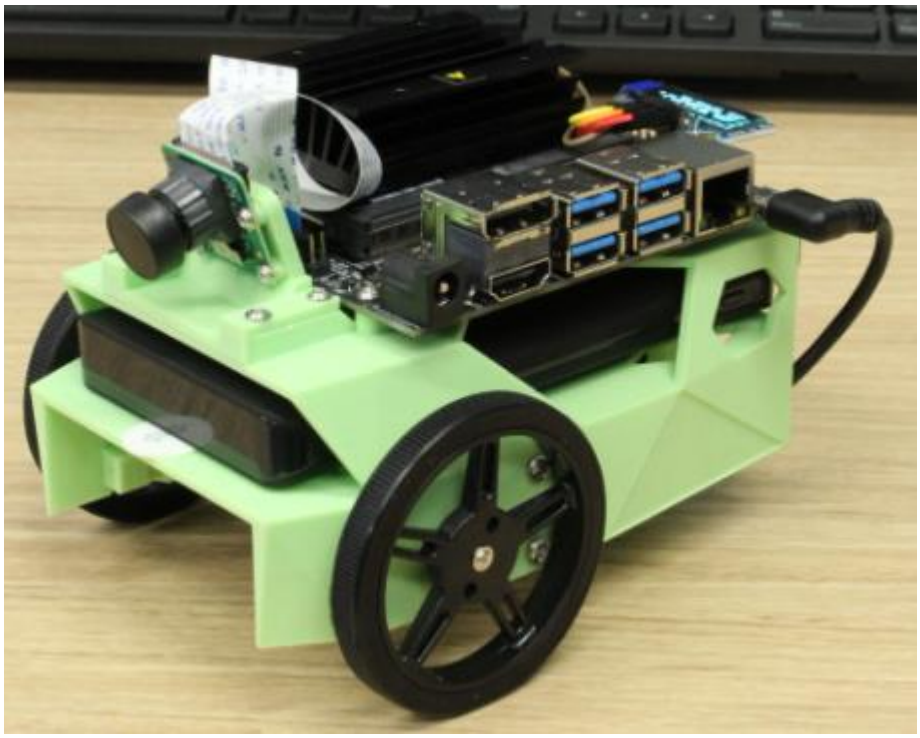


Figura 4.6 JetBot AI Kit venut per l'empresa Waveshare. (Font: [45])

Gaudeix d'un gran repertori de projectes realitzats per la comunitat que poden servir de guia en projectes de característiques similars. Entre els exemples es pot trobar: seguiment de carreteres, seguiment d'objectes i esquivar obstacles.

4.1.2. Triton AI (*Autonomous Racing Team*)

Triton AI és una organització d'estudiants que es dedica a la investigació i el desenvolupament en el camp de la intel·ligència artificial amb la finalitat de participar en competicions de robòtica.

Aquesta organització ha realitzat diversos projectes amb diferents sistemes per a dur a terme la conducció autònoma que es poden consultar al seu web [56].

Alguns dels exemples amb característiques més rellevants per aquest treball són:

- **SLAM Racer:** projecte que té com a objectiu desenvolupar un vehicle autònom capaç de navegar en entorns desconeguts utilitzant tècniques de SLAM (*Simultaneous Localization and Mapping*). El projecte fa ús de càmeres, LiDAR's i controladors, per generar un mapa 3D en temps real de l'entorn i localitzar la posició del vehicle dins d'aquest mapa. Posteriorment, un sistema de planificació de rutes permet al robot seguir un recorregut específic mentre evita obstacles.



Figura 4.7 SLAM Racer by Triton AI. (Font: [46])

- **Jetracer:** projecte que busca desenvolupar un vehicle autònom capaç de competir en un circuit de curses de robòtica utilitzant una Jetson Nano. El projecte combina tècniques de visió per computador, aprenentatge automàtic i control de sistemes per permetre al vehicle detectar i seguir la pista i ajustar la direcció en temps real. Aquest projecte és el més similar a nivell conceptual del que es vol dur a terme en aquest projecte.



Figura 4.8 JetRacer by Triton AI. (Font: [46])

- **Remote ROS Racer:** projecte operat amb ROS (*Robot Operating System*) per gestionar les diferents funcionalitats i tasques del vehicle. Com en altres casos, s'utilitzen tècniques de visió per computador i *machine learning* per permetre al vehicle seguir un recorregut, realitzant les prediccions pertinents i ajustant la velocitat i direcció en temps real.



Figura 4.9 ROS Racer by Triton AI. (Font: [46])

Tanmateix, Triton AI posseeix projectes molt més ambiciosos que els comentats, com és la construcció d'un Kart elèctric per competir en l'EVGrand Prix University Challenge o fins i tot la IAC (Indy Autonomous Challenge), que consisteix en crear un vehicle autònom capaç de completar una volta al circuit d'Indianapolis Motor Speedway. Els equips estan formats per estudiants universitaris de diferents disciplines i són responsables del disseny, construcció i programació dels vehicles per a que siguin capaços de donar una volta completa de forma autònoma. Tot això tenint en compte les restriccions i la normativa establerta per la orgnatització.

4.1.3. Donkey Car

Projecte *open source* que proporciona un marc de desenvolupament per a vehicles autònoms a nivell aficionat. Ofereix una plataforma amb hardware, software i documentació per construir i entrenar vehicles autònoms basats en Raspberry Pi amb una càmera. L'objectiu és proporcionar a la comunitat científica i educativa una plataforma per investigar i experimentar amb tecnologies de conducció autònoma a un cost reduit, i proporcionar a aficionats una forma de construir i jugar amb vehicles autònoms.

Disposa de diversos tutorials a la seva pròpia web i una gran comunitat que afavoreixen l'aparició de millores dia a dia, optimitzant la seva implementació.



Figura 4.10 Donkey Car (Font:[47])

4.1.4. Vehicle desenvolupat a l'assignatura ISCA (Implementació de Sistemes de Control Automàtic)

Finalment, l'origen més immediat d'aquest projecte es troba en el desenvolupament de vehicles autònoms realitzat a l'assignatura Implementació de Sistemes de Control Automàtic (ISCA). Aquests vehicles utilitzen un dispositiu Raspberry Pi connectat externament a un ordinador, encarregat de processar les dades capturades per la càmera i, a continuació, realitza els càlculs necessaris per determinar la direcció que ha de prendre el vehicle. Una vegada finalitzat aquest procés, l'ordinador envia les ordres apropiades a la Raspberry Pi, encarregada de la gestió dels motors i controladors per implementar les accions requerides.



Figura 4.11 Vehicle utilitzat a l'assignatura d'ISCA. D12 Kei Truck (Font: [48])

Taula 4.1 Comparativa d'avantatges i inconvenients de vehicles similars a JetRacer Pro.

	Avantatges	Inconvenients
Jetbot	- Fàcil configuració i utilització - Alta precisió en la navegació - Compatibilitat amb diversos sensors	- Cost elevat - Requereix un gran coneixement tècnic per a la personalització
SLAM Racer	- Mapeig amb SLAM en temps real - Capacitat per navegar en entorns desconeguts - Alta precisió en la navegació - Compatible amb ROS	- Cost elevat - Pot ser difícil de configurar
Remote ROS Racer	- Control remot i monitorització mitjançant ROS	- Cost elevat - Pot ser difícil de configurar
Donkey Car	- Baix cost - Fàcil de configu8rar i utilitzar - Ampli suport comunitari	- Menor precisió en la navegació - Poques opcions de personalització
Vehicle d'ISCA	- Baix cost - Fàcil de personalitzar - Ampli suport comunitari	- Menor precisió en la navegació - Requereix de connexió externa per al processat d'imatge
JetRacer Pro	- Alt rendiment - Alta precisió en la navegació - Alta velocitat - Compatibilitat amb diversos sensors - Compatible amb ROS	- Cost elevat - Requereix un gran coneixement tècnic per a la personalització

5. Software

5.1. Programari base i SO

Per a dur a terme aquest projecte, es parteix d'una imatge ISO preparada amb el programari i eines necessàries. Nvidia facilita aquests programari ja preparat sota la denominació JetPack, essent la versió utilitzada per aquest treball la 4.5.1.

El programari que inclou aquesta imatge ve preparat amb Ubuntu 18.04 amb llibreries necessàries per a poder desenvolupar el projecte treballant en Python. A la secció següent es mostren les possibilitats disponibles i finalment es farà un resum de les eines emprades per a tal objectiu.

El sistema operatiu Ubuntu 18.04 és una distribució de Linux àmpliament utilitzada en aplicacions relacionades amb la conducció autònoma i la intel·ligència artificial. Una de les principals característiques és la seva estabilitat i seguretat.

Inclou gran quantitat de programari per a l'ús en aplicacions de conducció autònoma i IA. Això comporta implementació per a llibreries molt utilitzades en aquest tipus d'aplicacions. Gràcies a això, es facilita significativament el desenvolupament de projectes relacionats amb aquestes tecnologies.

5.2. Python: llibreries per a conducció autònoma

Python és un llenguatge de programació utilitzat en moltes àrees diferents, com la ciència de dades, la intel·ligència artificial, i el desenvolupament web. Així mateix, disposa d'un gran nombre de llibreries i *frameworks* disponibles per a moltes tasques diferents com poden ser el processament numèric, la visualització de dades o les xarxes neuronals.

Les característiques principals que fan que Python sigui una gran eina de treball en l'àmbit de la conducció autònoma i la IA són les següents:

- **Flexibilitat:** és un llenguatge de programació de propòsit general, permetent als desenvolupadors implementar moltes tasques diferents en un sol llenguatge. Això facilita la creació de sistemes de conducció autònoma complets que inclouen algorismes de control, processament de dades dels sensor i integració de diferents mòduls.

- **Amplia disponibilitat de llibreries:** disposa d'un gran nombre de llibreries per a processament de dades dels sensor, xarxes neuronals i el processament d'imatges.

- **Comunitat activa:** posseeix una gran comunitat que contribueix a millorar les llibreries i els *frameworks* existents. Això implica que els desenvolupadors i investigadors de conducció autònoma tinguin accés a moltes eines i recursos de codi obert.

- **Fàcil de llegir i escriure:** el llenguatge Python és fàcil de llegir i escriure, i això ho fa accessible per a desenvolupadors de diferents nivells d'habilitat i experiència, a diferència de C++.

5.2.1. Open CV

OpenCV (*Open Source Computer Vision*) és una llibreria de visió per computador de codi obert que es fa servir àmpliament en la investigació i el desenvolupament de sistemes d'intel·ligència artificial (IA) i conducció autònoma. Disposa d'algorismes de processament d'imatge i video en temps real que permeten la detecció de patrons, el reconeixement de formes i la detecció de moviments. Està optimitzat per a l'ús de la GPU i inclou diverses implementacions d'algorismes per aconseguir gran velocitat de processament.

És compatible amb altres llibreries de càlcul científic com Numpy i Matplotlib, facilitant la integració amb altres aplicacions d'IA. La seva sintaxi és clara i intuïtiva i la fa fàcil d'utilitzar, afavorint el desenvolupament ràpid d'aplicacions de visió per computador.

5.2.2. PyTorch

PyTorch és una llibreria de computació científica de *open source* basada en Python que és utilitzada en la implementació i entrenament de xarxes neuronals profundes. Aquesta llibreria proporciona una API (*Aplication Programming Interace)* senzilla per definir i entrenar xarxes neuronals i la manipulació de les dades emmagatzemades.

La seva una arquitectura permet crear models de xarxes neuronals de manera molt fàcil i eficient. Gràcies a la integració amb altres llibreries de computació científica, com NumPy, es permet la manipulació de dades en forma de tensors, fet que facilita treballar amb aquestes dades.

5.2.3. Keras

Llibreria de xarxes neuronals de codi obert que permet crear una API intuitiva per a la implementació i entrenament de xarxes neuronals profundes. La seva arquitectura basada en mòduls la fa fàcil d'utilitzar i permet crear xarxes neuronals personalitzades sense haver de preocupar-se en per detalls tècnics complexes. Permet l'ús de la GPU per accelerar el processament de les xarxes neuronals.

En conducció autònoma, s'utilitza per entrenar models capaços de detectar objectes, reconeixerem senyals de trànsit o realitzar prediccions de la trajectòria del vehicle. També es pot fer servir per a la implementació de xarxes neuronals més complexes, les anomenamdes *deep neural networks*(DNN), en anglès.

5.2.4. TensorFlow

Eina *open source* que àmpliament utilitzada en intel·ligència artificial (IA) i la conducció autònoma. Permet treballar amb xarxes neuronals, que s'utilitzen per processar i interpretar les dades de sensors, càmeres, radars i lidars, que es fan servir per a la percepció de l'entorn del vehicle. Aquestes xarxes neuronals són entrenades amb grans conjunts de dades etiquetades, amb imatges, vídeos i dades de sensors, a fi de detectar objectes, reconèixer senyals de trànsit i prendre decisions en la conducció.

Ofereix un conjunt d'eines per treballar amb les xarxes neuronals i diferents mètodes d'optimització. També inclou eines per a crear perfils i depurar els models d'aprenentatge automàtic, com distribuir el càlcul entre diferents dispositius, ja sigui CPU, GPU o TPU per millorar la velocitat d'entrenament dels models.

5.2.5. TensorRT

Plataforma desenvolupada per NVIDIA que s'utilitza per accelerar el rendiment de dispositius que disposen de GPU. Es fa servir de forma habitual en l'aplicació de la intel·ligència artificial enfocada a la conducció autònoma per accelerar la velocitat de les xarxes neuronals quan processen i interpreten les dades de sensors.

La seva funció és optimitzar les xarxes neuronals per a l'execució en dispositius amb GPU, eliminant capes i elements del model que no són crítics per al rendiment i fusionant altres per reduir la càrrega computacional.

Una vegada optimitzat, el model s'exporta a un arxiu que es pot utilitzar posteriorment en l'execució de la conducció autònoma. També disposa d'eines per a la integració amb TensorFlow, PyTorch i Caffe2 i gran quantitat d'exemples en línia.

En l'execució de la conducció autònoma és molt útil ja que en accelerar la velocitat de resposta de les xarxes neuronals, la resposta del vehicle també és més ràpida. Aquestes xarxes han de processar grans quantitats de dades en temps real per prendre decisions de conducció i qualsevol millora és un factor a valorar molt positivament. Això es fa especialment important en sistemes de conducció autònoma de nivells més alts, on la precisió i la seguretat són crítiques.

5.2.6. Numpy

Llibreria de Python que té funcions matemàtiques avançades per a analitzar dades i realitzar operacions matemàtiques. Una de les característiques més importants és la seva capacitat per treballar ràpidament i eficient amb matrius multidimensionals. Això ho pot realitzar gràcies a que utilitza una estructura de dades que permet emmagatzemar i processar gran volum de dades.

Entre la amplia varietat de funcions matemàtiques disponibles trobem: operacions amb matrius, estadístiques bàsiques, operacions trigonomètriques i l'ús de funcions vectoritzades, per exemple.

Per aquest motiu és molt útil per a tasques que impliquin el processament de dades en volum, com el que es requereix en l'entrenament de models d'aprenentatge automàtic.

En conducció autònoma permet processar i interpretar les dades de sensors de forma molt eficient. Per exemple, es pot fer servir per a la detecció d'objectes o el reconeixement de senyals de trànsit a partir d'imatges. Una altra funcionalitat pot ser el recull de dades estadístic del vehicle, com la velocitat o la posició, fet que pot influir a l'hora de prendre decisions.

5.3. Xarxes neuronals

Les xarxes neuronals valorades per aquest projecte són:

ALEXNET

1. Arquitectura: consta de cinc capes convolucionals i tres capes totalment connectades o denses, amb un total de més de 60 milions de paràmetres.

2. Precisió en l'aprenentatge: és coneguda per la seva gran precisió en l'aprenentatge a ImageNet, amb un rendiment del 75 en la classificació d'imatges.

3. Temps d'entrenament: requereix d'un temps d'entrenament comparativament menor a les xarxes neuronals comparades en aquest projecte.

4. Validació de dades: la seva validació de dades és baixa, amb menor capacitat per generalitzar a partir de conjunts de dades etiquetats.

5. Ús en conducció autònoma: processa i interpreta les dades de càmeres per detectar objectes i reconèixer senyals de trànsit. També es pot fer servir per a l'entrenament de models de *machine learning* més complexes.

SQUEEZENET

1. Arquitectura: consta de 18 capes amb una arquitectura "*fire module*" que consisteix en repetir una sèrie de capes amb diferents mides de filtre, amb l'objectiu de reduir la complexitat del model. D'aquí ve el seu nom "*squeeze"* que significa estrènyer, a cada capa estrenyent més les dades amb els filtres. Malgrat tenir 18 capes, està pensada per ser petita i ràpida.

2. Precisió en l'aprenentatge: té una precisió en l'aprenentatge comparable lleugerament inferior a les xarxes comparades en aquest treball, assolint valors propers al 60% a ImageNet.

3. Temps d'entrenament: gracies a la seva menor complexitat, el seu temps d'entrenament és relativament menor comparat amb els models d'aquest projecte.

4. Validació de dades: ImageNet presenta resultats satisfactoris en quant a la validació de dades d'aquesta xarxa neuronal.

5. Ús en conducció autònoma: té una reduïda complexitat i rendiment general acceptable. Pot treballar amb imatges a fi de detectar objectes o reconèixer senyals de trànsit.

RESNET18 i RESNET34

1. Arquitectura: ResNet és una xarxa neuronal amb una arquitectura basada en "*residual learning*", que consisteix en afegir capes de "residus" a la xarxa per millorar la seva capacitat de representació. Aquesta arquitectura ha demostrat ser molt eficaç per a tasques de classificació d'imatges. Posseeixen 18 i 34 capes respectivament.

2. Precisió en l'aprenentatge: són xarxes neuronals molt potents amb una alta precisió en l'aprenentatge, assolint rendiments del 80% i 85% respectivament en tasques d'aprenentatge automàtic. Si bé Resnet 34 és més pesada, amb un bon equip, representar una millora respecte Resnet 18.

3. Temps d'entrenament: requereixen d'un temps d'entrenament elevat, essent superior aquest temps per al cas de ResNet 34. En el temps d'entrenament ResNet 18 surt afavorida.

4. Validació de dades: ambdues xarxes tenen una alta validació de dades, amb gran capacitat per generalitzar quan disposen de dades etiquetats. Requereixen una gran quantitat de dades d'entrenament per a desenvolupar el seu potencial, i és important assegurar que les dades utilitzades per a l'entrenament són de qualitat i representatives dels diferents escenaris que es volen afrontar.

5. Ús en la conducció autònoma: com les xarxes vistes en aquest projecte, disposen de capacitat per processar i interpretar dades provinents de sensors com càmeres per a realitzar prediccions en el moviment del vehicle.

DENSENET 121

1. Arquitectura: té una arquitectura amb capes de transició de densitat, concatenant les sortides de totes les capes anteriors com a entrades de la capa actual. Fa ús de diferents nivells de capes convolucionals, capes normalitzadores o *avgpool* i una capa totalment connectada. Aquesta arquitectura de 121 capes permet augmentar la profunditat de la xarxa sense patir el problema de la desaparició del gradient, fet que facilita les tasques de visió per computador amb molta informació a processar.

2. Precisió en l'aprenentatge: té una precisió en l'aprenentatge similar a la obtinguda per ResNet 18 i ResNet 34, assolint un rendiment del 80% aproximadament.

3. Temps d'entrenament: l'elevat nombre de capes que té implica un temps d'entrenament molt elevat comparat amb les arquitectures disponibles en aquest treball. És la xarxa neuronal amb un temps d'entrenament major i una major necessitat de recursos per a ser entrenada.

4. Validació de dades: degut a la seva alta complexitat, posseeix una gran validació de dades, permetent generalitzar a partir de conjunts de dades etiquetats amb facilitat.

5. Ús en la conducció autònoma: gràcies a la seva alta precisió en l'aprenentatge i la seva capacitat per a processar grans quantitats de dades d'imatge és una bona opció per a la conducció autònoma si es disposa dels recursos que requereix per al seu entrenament i execució.

5.4. Ús de llibreries i elecció de la xarxa neuronal

En l'apartat anterior s'han donat a conèixer diverses llibreries utilitzades de forma habitual en el tractament de dades enfocades a l'aplicació de IA i conducció autònoma. Ja que el projecte s'ha basat en la implementació i millora dels arxius de que disposa Jetpack 4.5.1, a continuació es troba el recull de llibreries emprades en aquests documents:

- OpenCV: s'utilitza al quadern *interactive_regression* en la fase de obtenció de dades i l'execució en temps real.
- PyTorch: s'utilitza al quadern *interactive_regression* en la fase de creació del model de xarxa neuronal i l'execució en temps real.
- Numpy: s'utilitza únicament en el quadern *road_following* original, per a convertir les dades a un format *array,* que permet més compatibilitat amb alters llibreries de Python.
- Widgets diversos: s'utilitzen en els quaderns *teleoperation, interactive_regression* i *road_following* per a dur a terme tasques variades de visualització i control.
- TensorRT: s'utilitza al quadern *road_following* i realitza la conversió del model creat en Pytorch a TensorRT per a poder aprofitar les capacitats d'acceleració de dispositius Nvidia.

En segon lloc, l'elecció de xarxes neuronals es limita a les opcions comentades ja que venien pre-carregades en el sistema i això facilitava la seva implementació. De totes les xarxes neuronals disponibles, s'ha escollit Resnet 18 degut a diversos factors:

- Es considerada una xarxa neuronal de referència en reconeixement d'objectes
- Posseeix una profunditat major que AlexNet o SqueezeNet, permetent una capacitat d'aprenentatge i generalització a partir de dades etiquetades major.
- A nivell de càrrega computacional és més eficient que ResNet 34 o Densenet 121.

Es posa de manifest que ResNet 18 es troba en equilibri entre les xarxes neuronals vistes, ja que no es tan pesada com ResNet 34 o Densenet 121, però manté un rendiment superior a AlexNet o SqueezeNet.

6. Implementació

D'entre els diferents kits i dispositius valorats per a la realització d'aquest projecte, finalment s'ha seleccionat JetRacer Pro AI Kit de l'empresa Waveshare.

Alguns dels motius més rellevants per a prendre aquesta decisió han sigut:

- **Hardware d'alta qualitat:** utilitza una plataforma Jetson Nano, que és un ordinador amb processador NVIDIA i GPU, amb un baix consum energètic i gran capacitat de processament de dades, molt útil per a aplicacions de *machine learning.*

- **Gran suport al desenvolupament:** disposa de documentació i recursos en línia per ajudar els usuaris a començar a treballar amb la plataforma, incloent tutorials i exemples de codi.

- **Aplicacions de conducció autònoma:** el seu disseny té un clara orientació a aplicacions de conducció autònoma en vehicles com el d'aquest projecte, incloent la capacitat de detectar i evitar obstacles, seguir les línies i d'altres funcions.

- **Gran compatibilitat:** té gran compatibilitat de sensors i càmeres i es pot adaptar fàcilment a diferents configuracions que poden incorporar altres sistemes com LiDAR, sensors infrarojos o radar, per exemple. El fet de tenir aquesta compatibilitat permet que se'n puguin derivar altres treballs i projectes posteriors a aquest, donant una gran versatilitat a aquest kit i un potencial per a treballar que d'altres no poden oferir.

6.1. Jetracer Pro AI kit by Waveshare

El JetRacer Pro AI Kit de Waveshare és un kit avançat preparat per a conducció autònoma que proporciona les eines i els dispositius necessaris pel desenvolupament i investigació d'aplicacions en intel·ligència artificial aplicada a vehicles autònoms. Aquest kit consta d'un xassís equipat amb tot l'indispensable per a un vehicle RC, un ordinador Jetson Nano i una càmera. També inclou documentació del muntatge i configuració del sistema.

Aquest kit de conducció autònoma representa una oportunitat per a investigadors, desenvolupadors, estudiants i educadors per poder experimentar amb tecnologies de IA aplicades a la conducció autònoma. Té unes dimensions petites i permet, el desenvolupament i validació de models creats en un ambient controlat i segur.



Figura 6.1 JetRacer Pro AI Kit by Waveshare (Fonts: [56],[57])

6.2. Posada en funcionament

A continuació, es descriu de forma detallada i il·lustrada els passos a seguir per treballar amb Jetson Nano, aplicant tècniques d'intel·ligència artificial per a assolir la conducció autònoma.

La primera tasca a realitzar després de la recepció del kit serà procedir al muntatge i verificació del funcionament de tots els components. El procés de muntatge està documentat a l'annex A1, on es troben totes les peces i les instruccions per dur a terme aquesta labor. Com es pot observar, no es caracteritza per tenir una complexitat excessiva, però és recomanable haver realitzat muntatges d'altres dispositius prèviament per tenir una major facilitat.

6.2.1. Carregar Imatge a la targeta SD

El següent pas per continuar amb la posada en funcionament del vehicle és carregar la imatge de Jetpack 4.5.1 a una targeta microSD. Per realitzar aquesta tasca serà necessari descarregar l'arxiu "jetracer_pro_ws.zip" disponible en la font [58]. Aquest arxiu consta del sistema operatiu Ubuntu 18.04 LTS, prèviament carregat amb les llibreries necessàries per a realitzar un projecte de conducció autònoma. Una vegada descarregat, cal descomprimir l'arxiu al nostre ordinador i preparar la targeta microSD per a rebre la instal·lació. És important destacar que l'arxiu és de gran mida, pel que es requereix l'ús d'una targeta amb una capacitat mínima de 64 GB.

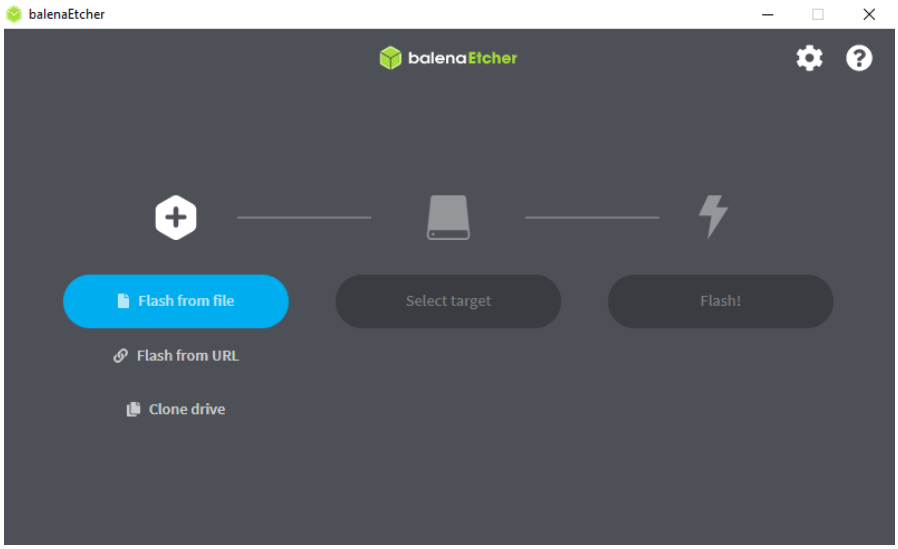


Figura 6.2 Programari Balena Etcher (Font: pròpia)

A continuació, s'introdueix la targeta al lector de l'ordinador i mitjançant el programari Etcher de Balena [59] es realitzarà la instal·lació. Aquest programa disposa d'una interfície clara, senzilla i intuïtiva que és fàcil de seguir i entendre. En aquest procés, es selecciona la imatge que es vol carregar, després es selecciona la ubicació on es vol instal·lar i finalment es prem el botó Flash. El procés d'instal·lació es

mostra mitjançant una barra de progrés i posteriorment es realitza una verificació de l'arxiu, comprovant que totes les dades originals s'han transcrit correctament a la targeta microSD.

Un cop carregat el programari connectarem una pantalla, un teclat i un ratolí per poder veure i controlar el sistema un cop encès.



Figura 6.3 Escriptori d'inici del sistema de la Jetson Nano (Font: pròpia)

Per continuar amb el desenvolupament del projecte, es considera necessari establir una connexió entre el vehicle i la xarxa wifi. Aquesta connexió permet accedir al vehicle de forma remota sense la necessitat de connectar pantalla, ratolí o teclat cada vegada que es vulgui realitzar un canvi. Això facilita la realització del projecte, ja que es pot fer ús d'un ordinador portàtil. Malgrat disposa de compatibilitat amb sistemes Windows i Linux, es recomana l'ús d'un ordinador amb Linux com a sistema operatiu per a aquesta tasca. Això es deu a que en aquest procés no va ser possible la connexió mitjançant un ordinador amb Windows mentre que amb Linux no ha hagut cap problema.

Per establir la connexió, es pot utilitzar el gestor de connexions de xarxa per buscar la xarxa desitjada i introduir la contrasenya corresponent.

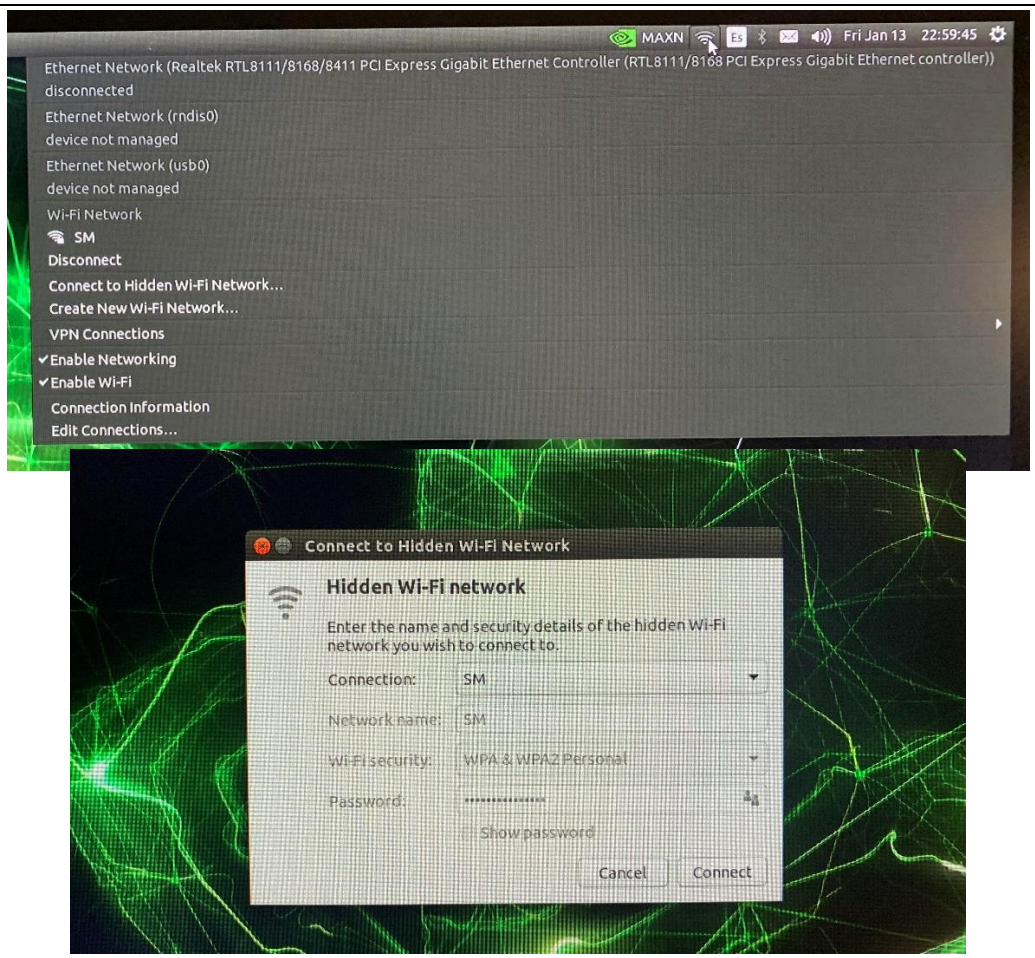


Figura 6.4 Procés de connexió de la Jetson Nano a la xarxa wifi (Font: pròpia)

Una forma senzilla de verificar que el dispositiu s'ha connectat correctament a la xarxa és comprovar si la pantalla OLED de la placa mostra l'adreça IP del vehicle.. Aquesta adreça IP serà utilitzada per a la connexió remota del vehicle.

Aquesta pantalla OLED també mostra informació rellevant sobre la Jetson Nano i l'electrònica del vehicle. En aquest cas, a la figura 6.5 es pot observar la paraula MAXN que fa referència al mode en què està operant la placa, considerat el mode de màxim rendiment. Existeix la possibilitat de canviar el mode a 5W però això implica un rendiment inferior.

També es mostra informació sobre el voltatge i corrent que hi ha a la bateria. En aquest cas, com la Jetson Nano està connectada mitjançant el microUSB, no està drenant energia de la bateria, per això el corrent que es pot observar a la imatge és 0 A. En últim lloc, es mostra un valor que indica l'estat de càrrega en què es troba la bateria en percentatge.

Finalment, la línia inferior mostra informació referent al consum que s'està produint en W, l'ús de la CPU, la GPU, la memòria RAM i el disc dur intern en percentatge.

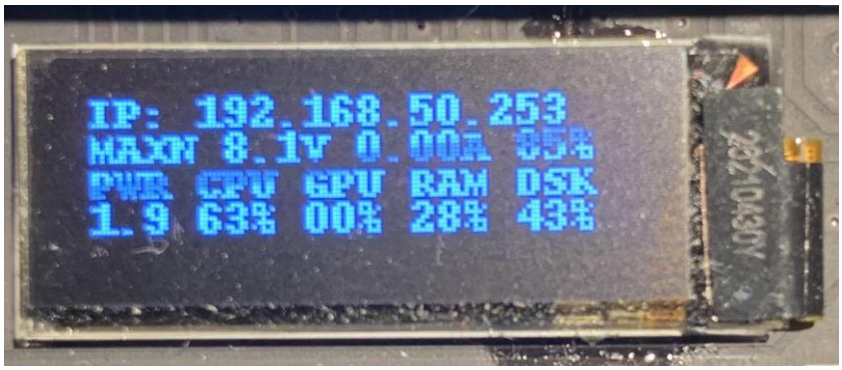


Figura 6.5 Pantalla oled del vehicle (Font: pròpia)

6.2.1. Connexió remota al vehicle mitjançant JupyterLab

Per realitzar aquest pas, sense tenir el vehicle connectat a cap dispositiu d'entrada o sortida i amb el sistema encès, ens disposarem a accedir a través de la plataforma JupyterLab. Per ingressar a aquesta plataforma ens dirigirem al navegador (preferiblement basat en Chromium degut a compatibilitats amb el comandament de control) i accedirem a l'adreça: "<jetracer_ip_address>:8888".

En aquest cas, l'adreça per accedir-hi en el meu cas és: "192.168.50.253:8888"). Un cop s'accedeix a l'adreça, es sol·licitarà una contrasenya, que de forma predeterminada és "jetson".

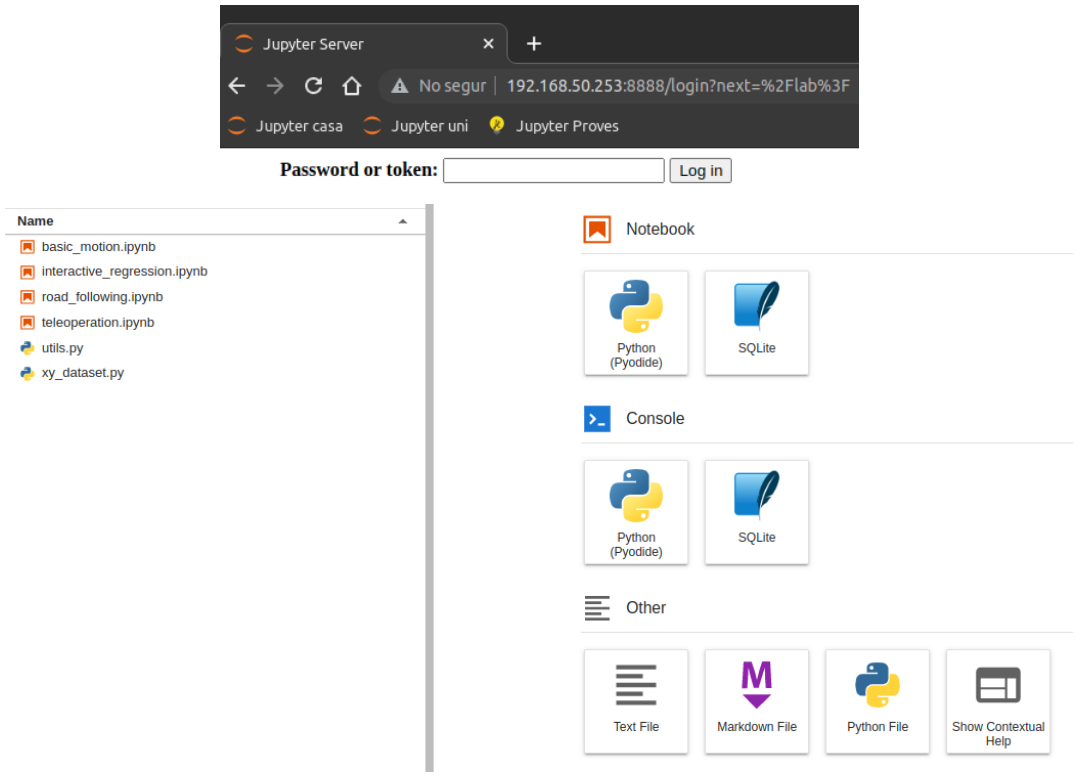


Figura 6.6 Imatge d'accés a JupyterLab amb el menu disponible (Font: [pròpia])

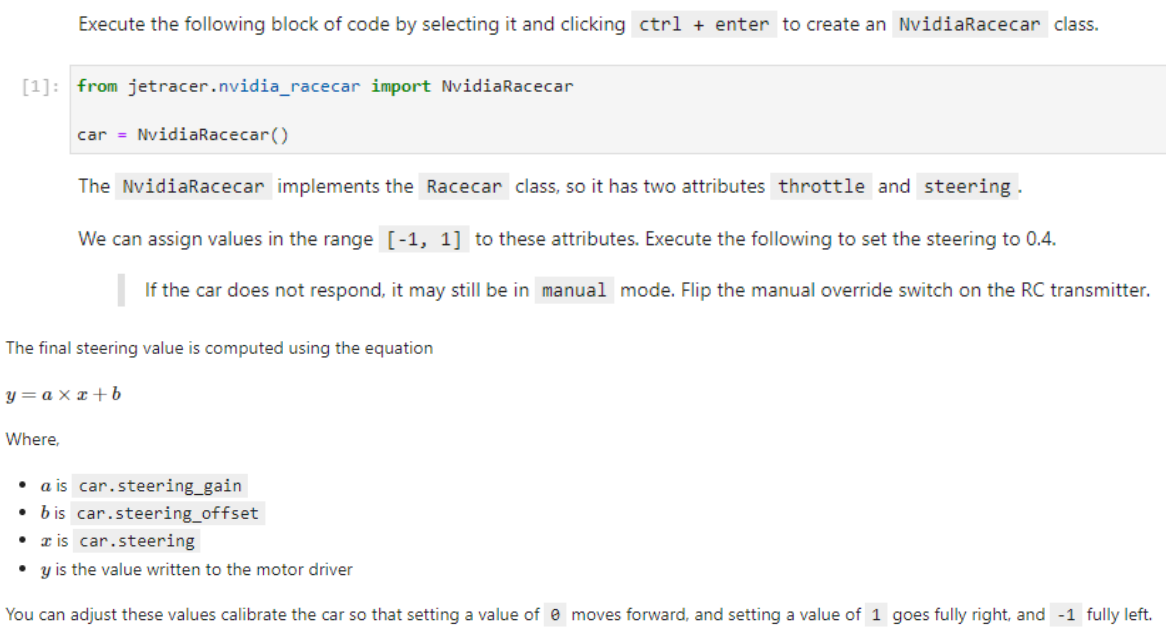
Dins la plataforma JupyterLab podrem observar un panell de control que mostra a l'esquerra un seguit d'arxius i carpetes i al centre diferents opcions de documents i terminal. Els arxius que es poden apreciar a l'esquerra són els arxius amb que treballarem i que es poden trobar als annexes d'aquest document.

Aquests *Notebooks,* són els arxius amb extensió .ipynb i són els següents:

- ***Basic_motion*:** document introductori per a veure el funcionament de la plataforma introducció a l'execució de blocs de codi.
- ***Teleoperations:*** document es vincularà el vehicle amb el comandament de control per a poder operar-lo de forma remota.
- ***Interactive_regression*:** document amb major importància del projecte, on es durà a terme l'adquisició de dades i l'entrenament de la xarxa neuronal escollida.
- ***Road_following:*** document on es realitzarà la tasca d'executar el codi per a que la xarxa neuronal realitzi conducció autònoma.

basic_motion

Aquest primer document és una introducció per a poder controlar el moviment bàsic del vehicle. L'arxiu utilitza un conjunt de comandaments per a la gestió dels motors i la direcció, com permetent controlar la velocitat i la direcció. També es poden configurar paràmetres com la velocitat màxima i la sensibilitat dels comandaments per a tenir en compte al pas posterior.


Execute the following block of code by selecting it and clicking `ctrl + enter` to create an `NvidiaRacecar` class.

```
[1]: from jetracer.nvidia_racecar import NvidiaRacecar

     car = NvidiaRacecar()
```

The `NvidiaRacecar` implements the `Racecar` class, so it has two attributes `throttle` and `steering`.

We can assign values in the range `[-1, 1]` to these attributes. Execute the following to set the steering to 0.4.

> If the car does not respond, it may still be in `manual` mode. Flip the manual override switch on the RC transmitter.

The final steering value is computed using the equation

$$y = a \times x + b$$

Where,

- a is `car.steering_gain`
- b is `car.steering_offset`
- x is `car.steering`
- y is the value written to the motor driver

You can adjust these values calibrate the car so that setting a value of `0` moves forward, and setting a value of `1` goes fully right, and `-1` fully left.

Figura 6.7 Exemple de la importació de llibreries i les explicacions esmentades (Font: [58])

6.2.2. Connexió i funcionament mitjançant comandament bluetooth

teleoperation

Permet utilitzar una interfície de comunicació i connexió del vehicle amb l'ordinador o el comandament de control. A través d'aquesta interfície, es poden enviar comandes de moviment i configuració al vehicle.

Accedint a la web [62] i seguint les instruccions, el nostre comandament serà reconegut. Un cop identificat el tipus de comandament de control caldrà verificar que aquest es connecta al vehicle seguint els passos que es mostren a continuació.

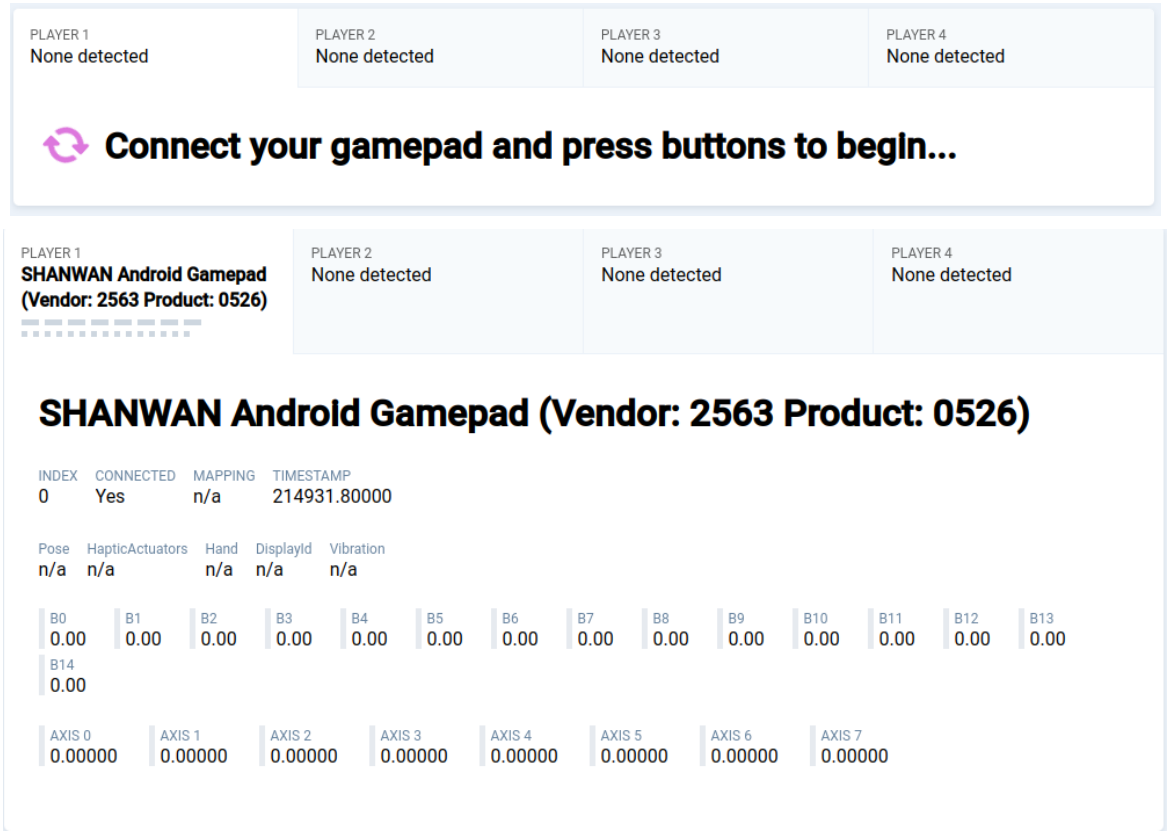


Figura 6.8 Interfície per al reconeixement del comandament *bluetooth* (Font: [63])

Tot seguit, executant el primer bloc de codi es mostrarà una interfície de control vinculada als botons del comandament que permetrà conèixer l'ordre d'assignació dels botons.

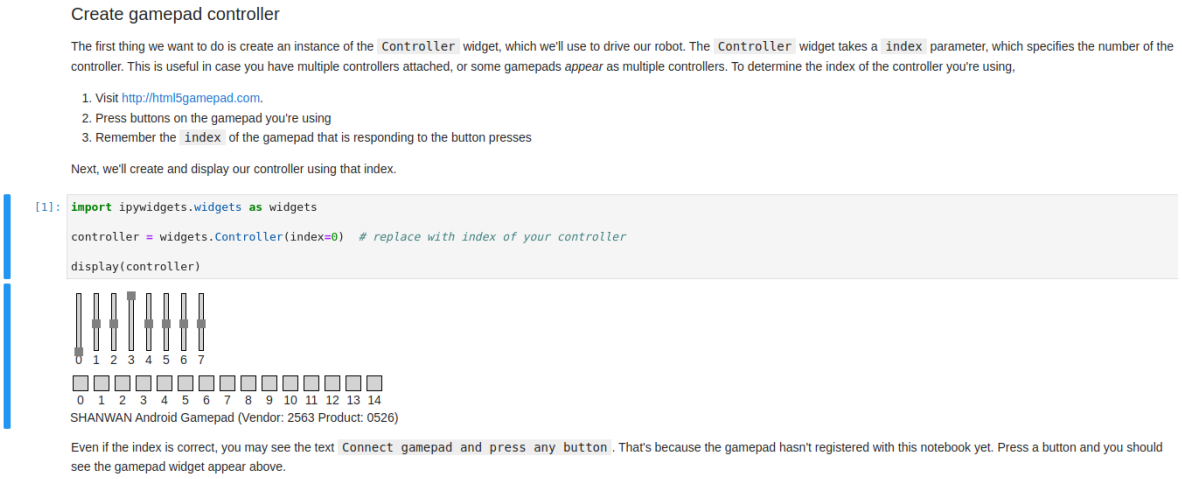


Figura 6.9 Interfície de connexió del comandament *bluetooth* (Font: [58])

Com es pot observar a la **Figura 6.9** per a entendre el funcionament de la vinculació dels botons s'han mogut els *joysticks* esquerra (cap a la dreta) i dret (cap amunt). És important aquest factor ja que posteriorment s'associaran aquests moviments a la direcció i la velocitat del vehicle. Per aquest motiu els eixos que cal posar al codi de la **Figura 6.10** són el 0 i el 3.

```
left_link = traitlets.dlink((controller.axes[0], 'value'), (car, 'steering'), transform=lambda x: x)
right_link = traitlets.dlink((controller.axes[3], 'value'), (car, 'throttle'), transform=lambda x: -x)
```

Figura 6.10 Vinculació dels botons del comandament amb el vehicle (Font: [58])

Un cop executat tot el codi i vinculat el comandament bluetooth, podrem verificar que l'electrònica funciona correctament comprovant que el vehicle és capaç de respondre a les ordres enviades per a realitzar diferents moviments.

En cas que la direcció o el control de velocitat es trobin invertits, modificar el valor de de lambda per a l'eix corresponent de positiu a negatiu o viceversa.

6.2.3. Procés d'obtenció de dades i entrenament de la xarxa neuronal

Interactive_regression

En aquest document es proporciona una interfície gràfica per a l'entrada de dades i eines que permeten seleccionar variables i visualitzar els resultats. Aquesta interfície permet explorar i analitzar les variables d'un conjunt de dades, i utilitzar aquestes relacions per a la predicció de valors futurs. Aquests conjunts de dades rebran el nom de "dades d'entrenament" i "dades de validació".

En aquest document caldrà llegir amb atenció cada bloc de codi per seleccionar quines eines es volen utilitzar i com es volen utilitzar. Així mateix, cada bloc queda ben definit pel seu títol amb la funció que realitza.

Task

Aquest codi crea un conjunt de dades amb l'objectiu d'entrenar models de xarxes neuronals. Utilitza la llibreria Pytorch per crear una seqüència de transformacions que s'apliquen a les imatges abans d'utilitzar-les com a entrades per al model.

```
import torchvision.transforms as transforms
from xy_dataset import XYDataset

TASK = 'road_following'

CATEGORIES = ['apex']

DATASETS = ['A', 'B']

TRANSFORMS = transforms.Compose([
    transforms.ColorJitter(0.2, 0.2, 0.2, 0.2),
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

datasets = {}
for name in DATASETS:
    datasets[name] = XYDataset(TASK + '_' + name, CATEGORIES, TRANSFORMS, random_hflip=True)
```

Figura 6.11 Codi encarregat de crear i transformar les dades provinents de la càmera (Font: [58])

El processament de dades d'aquesta figura realitza canvis en els colors, redimensionat d'imatge i la conversió d'aquesta imatge en valors tensor pel seu posterior tractament i es normalitza la imatge. Totes aquestes transformacions li aporten a la xarxa neuronal una robustesa que d'una altra forma no obtindria.

Data Collection

En aquest bloc es crea una interfície que serveix per recopilar i etiquetar dades d'una càmera. Crea desplegables que permeten seleccionar el conjunt de dades i la categoria, un quadre de recompte de dades i la possibilitat de clicar la imatge.

Quan es fa clic a la imatge de la càmera, aquesta es desa, es marca la ubicació de la categoria seleccionada i es mostra una instantània amb un cercle verd on s'ha marcat.



Figura 6.12 Interfície de recol·lecció de dades (Font: [58])

Model

En aquest bloc es crea un model de xarxa neuronal per a realitzar seguiment de carretera. Aquest model es basa en una de les arquitectures de xarxa neuronal disponibles a torchvision. Les opcions de que es disposa són AlexNet, SqueezeNet, ResNet 18 o 34 o DenseNet 121.

Per últim es crea un *widget* amb la possibilitat de carregar o guardar el model de xarxa neuronal.

Live Execution

En aquest bloc es selecciona la xarxa neuronal que s'utilitzarà per a l'entrenament del model. Per a dur a terme aquest entrenament, s'han recopilat dades significatives relacionades amb el seguiment del recorregut. Aquesta tasca es duu a terme col·locant el vehicle en diferents posicions del recorregut i indicant la direcció correcta en relació a la imatge transmesa per la càmera en temps real.

És important destacar que abans de ser emmagatzemades, les dades passen per diversos filtres que extreuen informació característica per al correcte aprenentatge de la xarxa neuronal. Aquest filtres

apliquen un conjunt d'operacions matemàtiques a cada imatge per separar la informació rellevant per a l'entrenament, en aquest cas específic els límits del recorregut.

Una vegada s'han recopilat les dades rellevants per al model es procedeix a l'entrenament, establint un nombre determinat d'EPOCHS (terme que fa referència a un cicle d'entrenament a través de totes les dades recopilades). Per a una primera aproximació, es recomana realitzar un entrenament d'entre 5 i 10 EPOCHS.

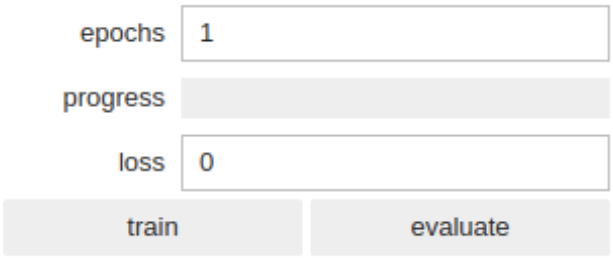


Figura 6.13 Interfície d'entrenament del model (Font: [58])

Un cop finalitzat l'entrenament s'activarà la predicció del comportament, fet que es pot aprofitar situant el vehicle en diferents posicions i observant on estima que hauria d'anar per a circular correctament. A la figura 6.14 es pot apreciar com la predicció a la corba manté el vehicle centrat en el circuit.

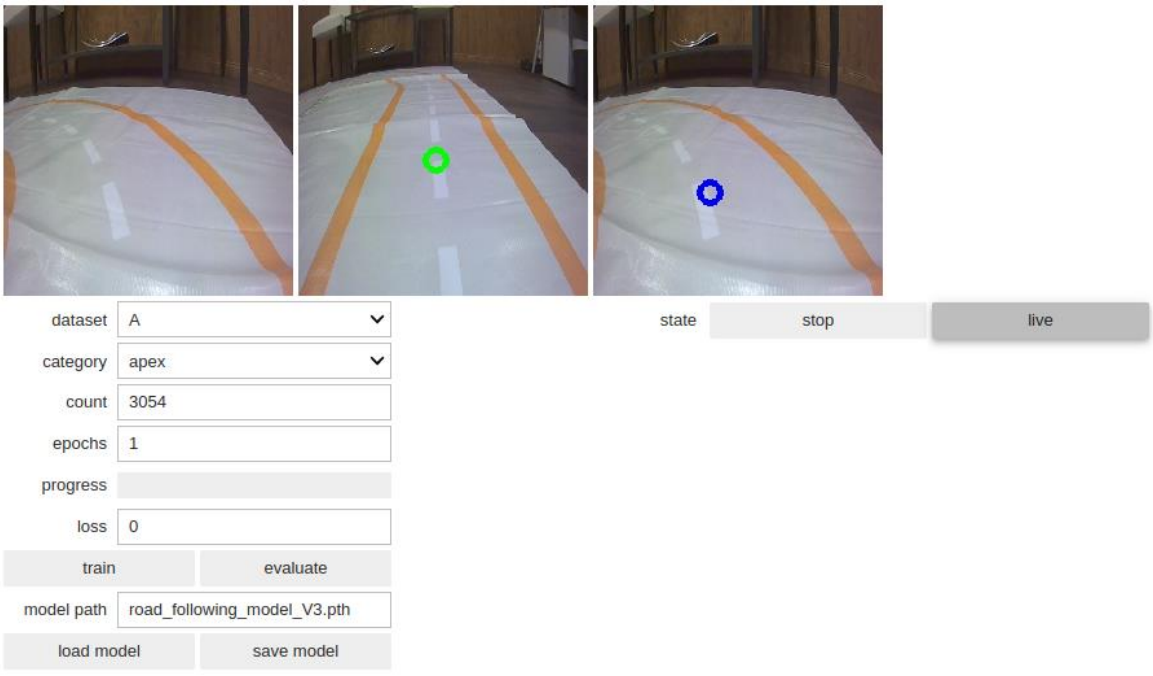


Figura 6.14 Interfície gràfica completa del quadern *interactive_regression* (Font: [58])

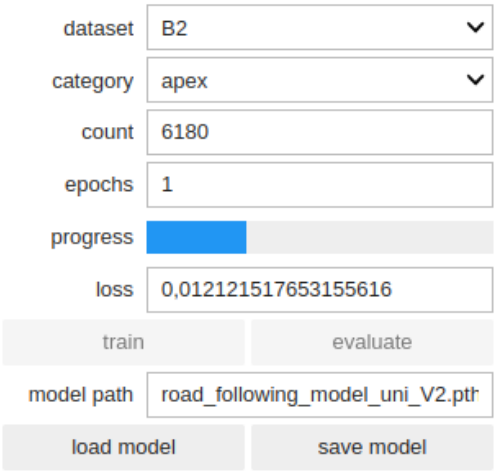


Figura 6.15 Aspecte de la interfície gràfica durant l'entrenament (Font: [58])

6.2.4. Model capaç de realitzar conducció autònoma

Road_following

Per últim, aquest document utilitza un conjunt de funcions per executar la detecció del traçat i la navegació del vehicle pel recorregut. Aquestes funcions fan us de tècniques de visió per computador per detectar les característiques de la carretera i guiar el vehicle. Un exemple de les capacitats que pot realitzar són mantenir la posició del vehicle a la carretera i evitar obstacles. A través d'aquestes funcions i capacitats el cotxe aconsegueix circular de forma autònoma pel circuit.

En primer lloc es realitza un depurat de la informació obtinguda de l'entrenament de la xarxa neuronal per a aconseguir un millor resultat. Posteriorment es tractaran les tasques que realitza de forma més concisa en l'apartat 7 d'aquest document.

Per últim destacar que en aquest document cal fixar una velocitat prèvia que representarà la velocitat constant de moviment del vehicle. És important considerar que és un vehicle molt potent i que la seva velocitat pot ser molt elevada. Per aquest motiu i per seguretat, es recomana fixar aquest valor entre 0.12 i 0.15 com a màxim ja que una velocitat superior pot ser perillosa tant per a l'usuari com pel risc de trencar el dispositiu.

Un inconvenient que presenta aquest arxiu és que cal fixar la velocitat constant i el cotxe no es podrà aturar fins que el bloc de codi de control es torni a executar amb una valor de velocitat 0.

```
from jetcam.csi_camera import CSICamera

camera = CSICamera(width=224, height=224)
#camera = CSICamera(width=224, height=224, capture_fps=65)
```

```
Finally, execute the cell below to make the racecar move forward, steering the racecar based on the x value of the apex.

Here are some tips,

* If the car wobbles left and right,  lower the steering gain
* If the car misses turns,  raise the steering gain
* If the car tends right, make the steering bias more negative (in small increments like -0.05)
* If the car tends left, make the steering bias more postive (in small increments +0.05)
```

```
from utils import preprocess
import numpy as np

STEERING_GAIN = 0.6
STEERING_BIAS = 0.0

car.throttle = 0.0

while True:
    image = camera.read()
    image = preprocess(image).half()
    output = model_trt(image).detach().cpu().numpy().flatten()
    x = float(output[0])
    car.steering = x * STEERING_GAIN + STEERING_BIAS
```

Figura 6.16 Codi del *road_following* (Font: [58])

7. Funcionament i resultats

Destacar que per a la realització d'aquest projecte, s'ha fet ús dels arxius originals de que disposa l'entorn instal·lat, preparat per treballar amb el kit de JetRacer. Aquest treball s'ha dut a terme treballant sempre a través de la plataforma Jupyter Lab i implementant les millores o modificacions als arxius de que es disposava.

Totes les iteracions realitzades en aquest projecte han seguit el mateix procés d'evolució, realitzant les proves en el circuit original a casa i posteriorment posats a prova en l'entorn del laboratori 5.4 de la EEBE.

7.1. 1r Model/iteració

Com bé s'ha explicat a l'apartat anterior, el primer model emprat en aquest treball és amb els arxius originals. Les primeres proves mancaven de dades rellevants suficients per a dur a terme la conducció autònoma però, progressivament, vaig anar entenent el funcionament i vaig ser capaç de crear un banc de dades suficient per a poder entrenar el model correctament. Així mateix, aquest model no posseïa la possibilitat de controlar la velocitat o realitzar una parada d'emergència.

7.2. 2n Model/iteració

En aquest segon model, es va identificar les limitacions de la primera iteració i es va buscar una forma de controlar la velocitat del vehicle. A través de la investigació de mètodes per controlar les variables del vehicle en temps real[59][60], en aquest segon model s'ha implementat el control manual de velocitat. Aquest control s'ha implementat mitjançant l'ús d'una funció específica basada en [61], ja que en l'estructura original del codi, no es va poder trobar una forma d'incloure aquestes característiques en el bucle *while*. Gràcies a aquesta implementació les proves ja es poden realitzar amb major seguretat i control.

```
from utils import preprocess
import numpy as np

STEERING_GAIN = 0.6
STEERING_BIAS = 0.0

car.throttle = 0.0

while True:
    image = camera.read()
    image = preprocess(image).half()
    output = model_trt(image).detach().cpu().numpy().flatten()
    x = float(output[0])
    car.steering = x * STEERING_GAIN + STEERING_BIAS
```

Figura 7.1 Algorisme d'execució de *road_following* amb bucle *while* (Font: [58])

```
from utils import preprocess
from jetcam.utils import bgr8_to_jpeg

def update(change):
    global blocked_slider, robot
    new_image = change['new']

    image = preprocess(new_image).half()
    output = model_trt(image).detach().cpu().numpy().flatten()
    x = float(output[0])
    y = float(output[0])

    network_output_slider.value = x
    steering = x * steering_gain_slider.value + steering_bias_slider.value
    if(steering<-1.0):
        steering_value_slider.value = -1.0
    elif(steering>1.0):
        steering_value_slider.value = 1.0
    else:
        steering_value_slider.value = steering
    car.steering = x

update({'new': camera.value})
camera.observe(update, names='value')
camera.running = True
```

Figura 7.2 Algorisme d'execució de *road_following* amb funció (Font: pròpia)

En aquest model es destaca la importància d'un conjunt de dades equitatiu per al procés d'entrenament de la xarxa neuronal. El desequilibri en la distribució de les dades, amb un nombre excessiu de girs en una direcció o en l'altra, pot generar un biaix en l'entrenament, provocant que el vehicle circuli millor en un sentit que en l'altre.

Així mateix, es va observar que el model no era capaç de circular amb fluïdesa. Els primers intents per fer funcionar aquest model van revelar que el vehicle responia al traçat de manera més lenta que en el primer model. Després de diverses proves es va descobrir que el codi implementat amb funcions per a la modificació de variables en temps real comportava una pèrdua de rendiment en el processat dels fotogrames per segon.

Aquest comportament es va poder apreciar amb girs realitzats fora de temps i amb una tendència del vehicle a desviar-se del traçat en les rectes. Per a solucionar aquest comportament es van realitzar diverses proves, disminuint els fps d'un valor inicial de 30 fins a 15-20 en un model que buscava un compromís entre la velocitat i capacitat de resposta. Gràcies a la disminució de dades que cal processar, el vehicle pot respondre d'una forma més ràpida i acurada.

7.2.1. Implementació d'interfície de control amb regulació manual de velocitat

```
import traitlets
from IPython.display import display
from ipywidgets import Layout, Button, Box
import ipywidgets.widgets as widgets

network_output_slider = widgets.FloatSlider(description='Network Output', min=-1.0, max=1.0, value=0, step=0.01, orientation='horizontal', disabled=False, layout={'width': '400px'})
steering_gain_slider  = widgets.FloatSlider(description='Steering Gain', min=-1.0, max=1.0, value=-0.7, step=0.01, orientation='horizontal', layout={'width': '300px'})
steering_bias_slider  = widgets.FloatSlider(description='Steering Bias', min=-0.5, max=0.5, value=0.0, step=0.01, orientation='horizontal', layout={'width': '300px'})
steering_value_slider = widgets.FloatSlider(description='Steering', min=-1.0, max=1.0, value=0, step=0.01, orientation='horizontal', disabled=False, layout={'width': '400px'})
throttle_slider = widgets.FloatSlider(description='Throttle', min=0.0, max=0.2, value=0.0, step=0.01, orientation='vertical')
#  throttle_slider value should be [-1,1] set to [0,0.2] for prevention

steering_gain_link   = traitlets.link((steering_gain_slider, 'value'), (car, 'steering_gain'))
steering_offset_link = traitlets.link((steering_bias_slider, 'value'), (car, 'steering_offset'))
steering_value_link  = traitlets.link((steering_value_slider, 'value'), (car, 'steering'))
throttle_slider_link = traitlets.link((throttle_slider, 'value'), (car, 'throttle'))

display(
    widgets.HBox(
        [widgets.VBox([network_output_slider,
                       widgets.Label(value="X"),
                       steering_gain_slider,
                       widgets.Label(value="+"),
                       steering_bias_slider,
                       widgets.Label(value="||"),
                       steering_value_slider], layout=Layout(align_items='center')
                      ),
         throttle_slider]
            )
        )
```

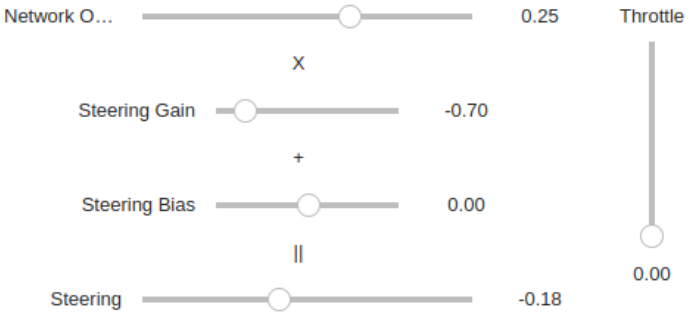
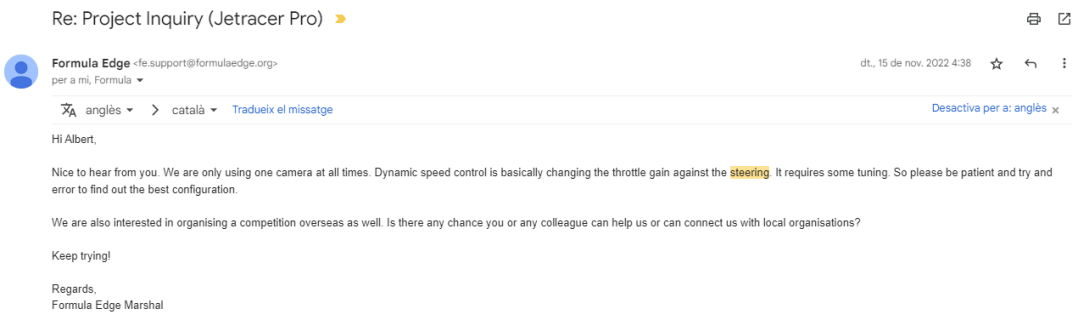


Figura 7.3 Codi i visualització de la interfície de control (Font: pròpia)

7.3. 3r Model/iteració

En aquest tercer model es va realitzar una investigació per implementar el control dinàmic de velocitat. Aquesta cerca va incloure la consulta a l'equip de Formula Edge per obtenir informació sobre les seves pràctiques d'implementació.


Re: Project Inquiry (Jetracer Pro)

Formula Edge <fe.support@formulaedge.org> dt., 15 de nov. 2022 4:38
per a mi, Formula

anglès › català Tradueix el missatge Desactiva per a: anglès

Hi Albert,

Nice to hear from you. We are only using one camera at all times. Dynamic speed control is basically changing the throttle gain against the steering. It requires some tuning. So please be patient and try and error to find out the best configuration.

We are also interested in organising a competition overseas as well. Is there any chance you or any colleague can help us or can connect us with local organisations?

Keep trying!

Regards,
Formula Edge Marshal

Figura 7.4 Resposta sobre la consulta de control dinàmic de velocitat (Font: pròpia)

A través d'aquesta informació, es van realitzar diverses proves per establir el rang de velocitats en funció de l'angle de gir. Després de diversos intents es va determinar la necessitat d'implementar una parada d'emergència, ja que amb la implementació del rang de velocitat dinàmic es feia més difícil que el vehicle s'aturés. Aquesta implementació es durà a terme en la següent iteració.

Un altre aspecte modificat en aquest model és la qualitat de les imatges utilitzades per a la conducció autònoma. Es va observar que les imatges no eren nítides i això podria provocar una pèrdua de dades. Després de consultar la documentació de Nvidia sobre Jetson Nano, es va descobrir una forma de solucionar aquest problema mitjançant l'execució de comandaments al terminal de Linux. Aquesta solució implicava descarregar un perfil distribuït per Nvidia que, una vegada instal·lat a la Jetson Nano, va ser capaç de corregir aquest efecte i millorar la qualitat de les imatges. En aquest cas es tracta d'una càmera IMX219, pel que el codi aplicar és el següent:

IMX219:

```
wget https://www.arducam.com/downloads/Jetson/Camera_overrides.tar.gz
tar zxvf Camera_overrides.tar.gz
sudo cp camera_overrides.isp /var/nvidia/nvcam/settings/
sudo chmod 664 /var/nvidia/nvcam/settings/camera_overrides.isp
sudo chown root:root /var/nvidia/nvcam/settings/camera_overrides.isp
```

Figura 7.5 Codi necessari per a la correcció de la distorsió d'imatge existent (Font: [64])



Figura 7.6 Aspecte de les imatges abans i després de la correcció aplicada (Font: pròpia)

En la realització de les proves d'aquest model, es va observar la problemàtica associada a les superfícies reflectants i l'impacte directe de la llum sobre la càmera. Aquest factor va causar que el vehicle no fos capaç de seguir el recorregut correctament en algunes ocasions, influint en la trajectòria fins al punt de provocar una possibilitat de col·lisió.

Per tal de solucionar aquest problema es van buscar possibles solucions, com per exemple els filtres de llum, esmentats en la secció de millores. Aquesta solució no es va poder implementar degut a factors externs, però amb temps i dedicació es podria trobar una forma factible d'aplicar-la.

7.3.1. Implementació de codi per al control dinàmic de velocitat

Es tracta de la primera iteració per a controlar de forma dinàmica la velocitat del vehicle. Com es pot observar pren el valor de gir per a modificar la velocitat entre els valors establerts.

```
if (steering<=0.3 and steering>=-0.3 and throttle_slider.value<=0.16 and throttle_slider.value>0.12):
    throttle_slider.value += 0.01
elif((steering>0.3 or steering<-0.3) and throttle_slider.value>0.14):
    throttle_slider.value += - 0.01
else:
    throttle_slider.value = throttle_slider.value
```

Figura 7.7 Codi inicial control dinàmic velocitat (Font: pròpia)

7.4. 4t Model/iteració

Finalment, en aquest projecte s'ha desenvolupat el model més complet, que disposa d'una interfície gràfica millorada amb parada d'emergència. En quan al control dinàmic de la velocitat s'han creat diferents modes associats a diferents rangs dinàmics per a major control.

7.4.1. Interfície de control amb visualització en temps real

Es realitzen ajustaments a la interfície gràfica perquè tingui un aspecte més compacte i s'implementa la visualització de la predicció del model en temps real.

```
display(
    widgets.HBox(
        [widgets.VBox([network_output_slider,
                       widgets.Label(value="X"),
                       steering_gain_slider,
                       widgets.Label(value="+"),
                       steering_bias_slider,
                       widgets.Label(value="||"),
                       steering_value_slider,
                       button,out,], layout=Layout(align_items='center')
                      ),
         throttle_slider,
         speed_slider,
         live_execution_widget]
                )
        )
```

Figura 7.8 Codi de la interfície gràfica final (Font: pròpia)

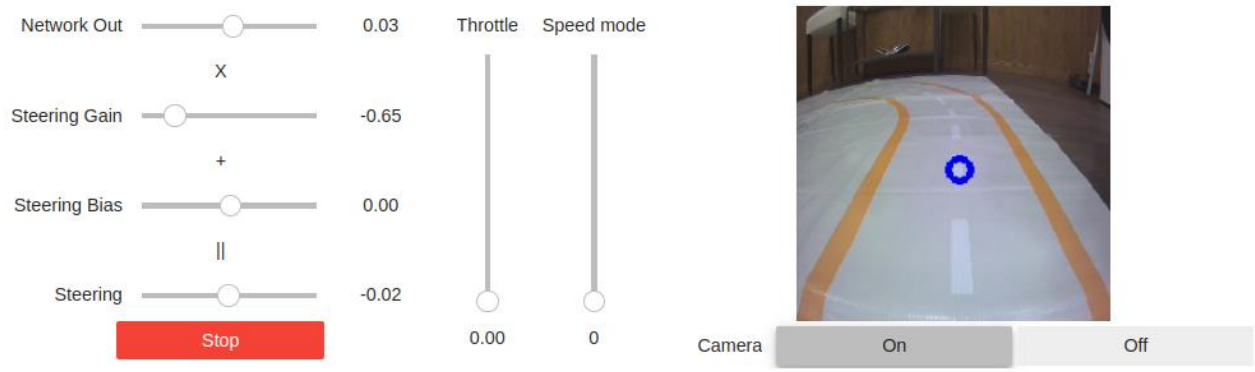


Figura 7.9 Interfície gràfica completa (Font: pròpia)

7.4.2. Implementació parada d'emergència

A més, s'ha incorporat una funció de parada d'emergència, que és una implementació necessària, malgrat que amb la implementació dels modes de velocitat ja s'aplica un condicional que atura completament el vehicle en cas de situar el *speed mode* a 0.

```
button = widgets.Button(description='Stop', button_style='danger')
out = widgets.Output()

def on_button_clicked(_):
    with out:
        speed_slider.value = 0
button.on_click(on_button_clicked)
```

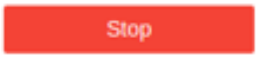


Figura 7.10 Implementació i visualització de la parada d'emergència (Font: pròpia)

7.4.3. Implementació control dinàmic de velocitat amb modes de velocitat

Per a la implementació del control dinàmic amb modes de velocitat s'ha optat per dotar el sistema d'un total de 8 modes de velocitat amb una progressió ascendent.

Com es pot observar a la següent figura, aquests modes permeten un ajust més acurat dels valors de velocitat, ja que un valor de 0.01 treballant amb unitats tan petites, pot suposar un gran augment de velocitat. Aquest fet es posa de manifest en observar que en cap moment la velocitat superarà el llindar de 0.2 per seguretat, ja que es tracta d'un dispositiu amb molta potència.

```
steering_max = 0.13
steering_min = -0.13

if speed_slider.value == 0:
    throttle_slider.value = 0

elif speed_slider.value == 1:
    if (steering<=steering_max and steering>=steering_min and throttle_slider.value<0.14 and throttle_slider.value>0.11):
        throttle_slider.value += 0.001
    elif((steering>steering_max or steering<steering_min) and throttle_slider.value>0.12):
        throttle_slider.value += - 0.001

elif speed_slider.value == 2:
    if (steering<=steering_max and steering>=steering_min and throttle_slider.value<0.15 and throttle_slider.value>0.11):
        throttle_slider.value += 0.001
    elif((steering>steering_max or steering<steering_min) and throttle_slider.value>0.13):
        throttle_slider.value += - 0.001

elif speed_slider.value == 3:
    if (steering<=steering_max and steering>=steering_min and throttle_slider.value<0.15 and throttle_slider.value>0.11):
        throttle_slider.value += 0.001
    elif((steering>steering_max or steering<steering_min) and throttle_slider.value>0.14):
        throttle_slider.value += - 0.001

elif speed_slider.value == 4:
    if (steering<=steering_max and steering>=steering_min and throttle_slider.value<0.16 and throttle_slider.value>0.11):
        throttle_slider.value += 0.001
    elif((steering>steering_max or steering<steering_min) and throttle_slider.value>0.15):
        throttle_slider.value += - 0.001

elif speed_slider.value == 5:
    if (steering<=steering_max and steering>=steering_min and throttle_slider.value<0.17 and throttle_slider.value>0.11):
        throttle_slider.value += 0.001
    elif((steering>steering_max or steering<steering_min) and throttle_slider.value>0.16):
        throttle_slider.value += - 0.001

elif speed_slider.value == 6:
    if (steering<=steering_max and steering>=steering_min and throttle_slider.value<0.18 and throttle_slider.value>0.11):
        throttle_slider.value += 0.001
    elif((steering>steering_max or steering<steering_min) and throttle_slider.value>0.17):
        throttle_slider.value += - 0.001

elif speed_slider.value == 7:
    if (steering<=steering_max and steering>=steering_min and throttle_slider.value<0.19 and throttle_slider.value>0.11):
        throttle_slider.value += 0.001
    elif((steering>steering_max or steering<steering_min) and throttle_slider.value>0.18):
        throttle_slider.value += - 0.001

elif speed_slider.value == 8:
    if (steering<=steering_max and steering>=steering_min and throttle_slider.value<0.2 and throttle_slider.value>0.11):
        throttle_slider.value += 0.001
    elif((steering>steering_max or steering<steering_min) and throttle_slider.value>0.19):
        throttle_slider.value += - 0.001

else:
    throttle_slider.value = throttle_slider.value
```

Figura 7.11 Codi encarregat de gestionar el control dinàmic de velocitat (Font: pròpia)

Els valors emprats en cada un dels *speed modes* així com l'angle de gir per a que aquests canvis es duguin a terme s'han anat ajustant experimentalment, buscant la millor aproximació a una conducció precisa i fluida. Així mateix el valor llindar a partir del qual el vehicle comença a gestionar la seva velocitat de forma dinàmica està fixat en 0.11, ja que s'ha observat que amb nivells de càrrega elevats, el vehicle comença a moure's a partir de 0.12.

7.4.4. Evitar obstacles

Aquest model és capaç d'evitar obstacles, en aquest cas, senyals blaus que ha identificat durant el procés d'entrenament. Això requereix que el banc de dades d'entrenament contingui informació sobre com evitar obstacles en diferents situacions i posicionaments.

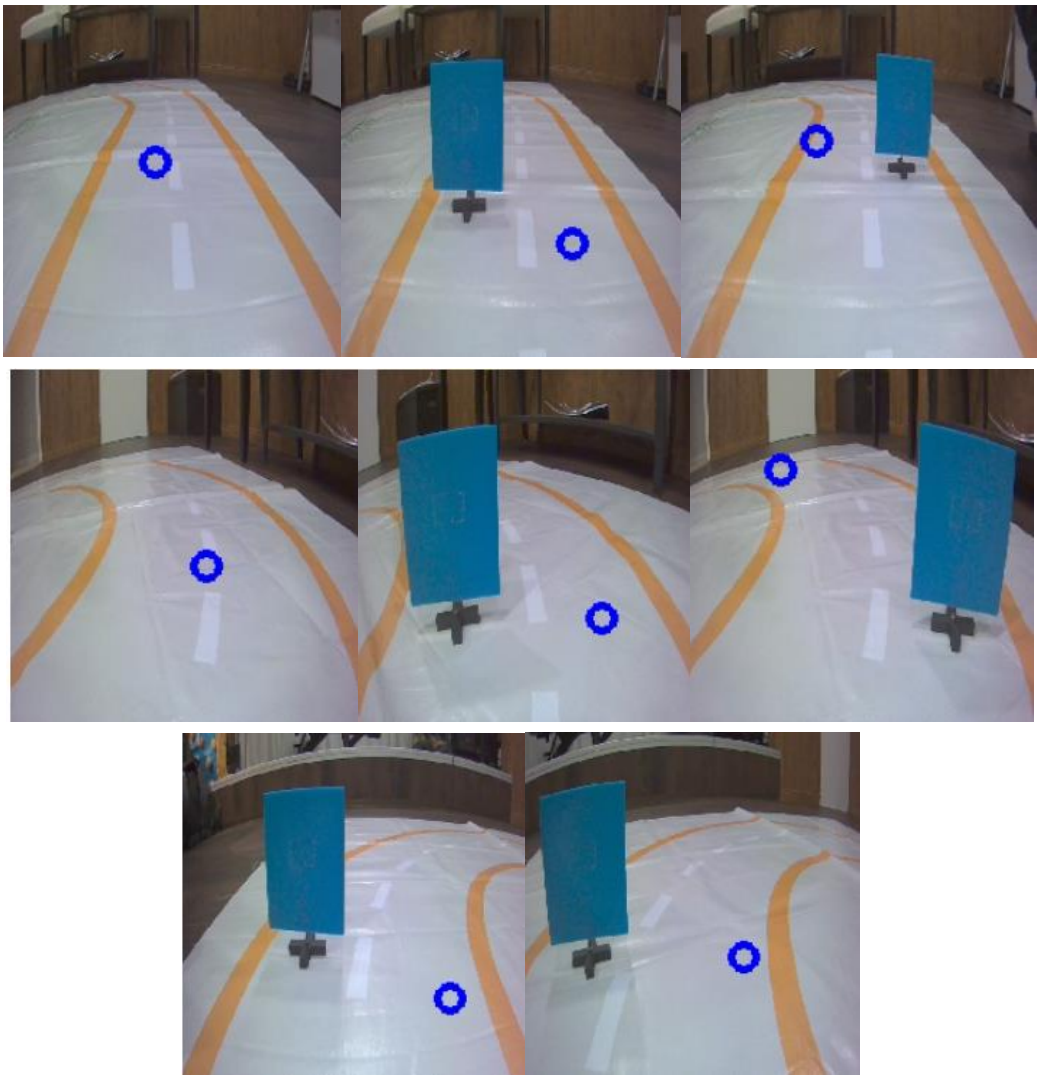


Figura 7.12 Imatges de la predicció evitant obstacles (Font: [64])

És important destacar que un excés d'entrenament amb obstacles d'una forma específica pot portar a un entrenament amb biaix, pel que es recomana proporcionar un context general.
Cal emmagatzemar dades amb diferents condicions de visibilitat dels obstacles i configuracions de posicionament permetre que la IA generalitzi en lloc d'aprendre un únic moviment específic. Així mateix, cal tenir en compte que l'increment del nombre de dades augmenta el temps d'entrenament, reduint el temps disponible per a testejar el model.

7.4.5. Resum progressió iteracions

Com es pot observar a la taula inferior, la quantitat de dades d'entrenament i els epochs (1 epoch equival a 1 cicle d'entrenament a través de totes les dades emmagatzemades) realitzats per al seu entrenament influeixen dràsticament en la inversió de temps necessària per a cada iteració. La taula mostra la relació dels factors claus per a l'entrenament d'una xarxa neuronal en la Jetson Nano.

Taula 7.1 Informació d'entrenament a la Jetson Nano.

Model	Quantitat de dades	Epochs	Temps d'entrenament per epoch / total
1a Iteració	830	10	1:48 min / 18:00 min
2a Iteració	3055	15	5:56 min / 1:29 h
3a Iteració	4966	15	9:38 min / 2:24 h
4a Iteració	6180	20	11:47 min / 2:56 h

Taula 7.2 Recull de les capacitats i implementacions de cada iteració.

Model	Millores respecte la iteració anterior	Capacitats
1a Iteració	-	- Seguiment del circuit
2a Iteració	- Visualització de variables de control en temps real. - Control manual de la velocitat	- Seguiment del circuit - Visualització de variables de control en temps real. - Control manual de la velocitat
3a Iteració	- Control dinàmic de la velocitat en un rang determinat en funció de l'angle de gir.	- Seguiment del circuit - Visualització de variables de control en temps real. - Control dinàmic de la velocitat
4a Iteració	- Millora en la interfície gràfica - Visualització de la predicció en temps real - Implementació d'un botó de parada d'emergència - Implementació dels *speed modes* i millor ajust dels paràmetres - Capacitat per evitar obstacles	- Seguiment del circuit - Visualització de variables de control i predicció en temps real - Control dinàmic de la velocitat amb *speed modes* - Capacitat per evitar obstacles - Parada d'emergència

8. Propostes de millora

Malgrat el vehicle ha complert els objectius establerts s'ha pogut observar un potencial de millora tant a nivell de components com a nivell de funcionalitats que es podria resumir ens els conceptes següents:

8.1. Components

8.1.1. Bateria amb major capacitat

Jetson Nano es considerat un dispositiu de baix consum. Malgrat aquest baix consum, s'ha pogut apreciar durant els transcurs d'aquest projecte que l'alimentació dels motors i controladors que es fan càrrec de la conducció, junt amb la Jetson Nano, representen un consum considerable.

Aquest consum posa de manifest la necessitat de recarregar la bateria amb gran assiduïtat entre proves. Una bateria major pot proporcionar una major autonomia, velocitat i potencia, afavorint la possibilitat que el vehicle pugui ser utilitzat durant més temps, posant a prova les seves capacitats.

8.1.2. Filtre de llum anti-enlluernament

Un gran problema durant el transcurs de la presa de dades i la verificació del funcionament del model ha sigut la il·luminació. Els enlluernaments provocats de forma directe o indirecte per la llum i les superfícies reflectants provoquen un mal funcionament del sistema de conducció autònoma.

L'ús de filtres anti-enlluernament en un vehicle autònom que es guia únicament per càmera, pot implicar una millora en la visió del sistema en escenaris amb alta lluminositat o amb superfícies reflectants. Això pot permetre que el vehicle sigui capaç de traçar el recorregut, detectar i evitar obstacles de forma més precisa i circular d'una forma més fluida independentment de la il·luminació.

Les mesures proposades per pal·liar aquest problema poden ser:

- Ús d'un filtre especialitzat ubicat enfront de l'objectiu de la càmera per evitar els enlluernaments en qualsevol escenari.
- Ús de pantalles o films de fotografia ubicades enfront de les fonts d'il·luminació, que filtren la llum provocant que aquesta no enlluerni indiferentment de la superfície on incideixi posteriorment.

8.1.3. Jetson AGX Xavier

En aquest projecte s'ha fes ús del dispositiu Jetson Nano per a dur a terme totes les tasques de *machine learning, deep learning* i el processat de dades que comporta. Si bé aquesta placa és potent, es considera un dispositiu d'entrada per a poder treballar amb IA i conducció autònoma. Per a aconseguir un millor rendiment i tenir unes majors prestacions es podria utilitzar la placa Jetson AGX Xavier. Aquesta placa, és utilitzada a nivell industrial però té un cost molt major, ja que posseeix un processador més potent, major quantitat de memòria i una major compatibilitat amb sensors i connectivitat. Tanmateix el rendiment operacional que ofereix supera l'ofert per la Jetson Nano de 472 GFLOPS a 21 TFLOPS, fet que dona un ventall de possibilitats molt major a l'hora de treballar amb IA i conducció autònoma.



Figura 8.1 Nvidia Jetson AGX Xavier (Font: [49])

8.2. Funcionalitats

8.1.4. Millora de la interfície gràfica

L'eina DASH AVS Explorer és una plataforma d'anàlisi de dades per a vehicles autònoms a valorar en futurs projectes que esdevinguin d'aquest. És un exemple de interfície gràfica que implementa la representació visual dels paràmetres mitjançant la visió per computador.

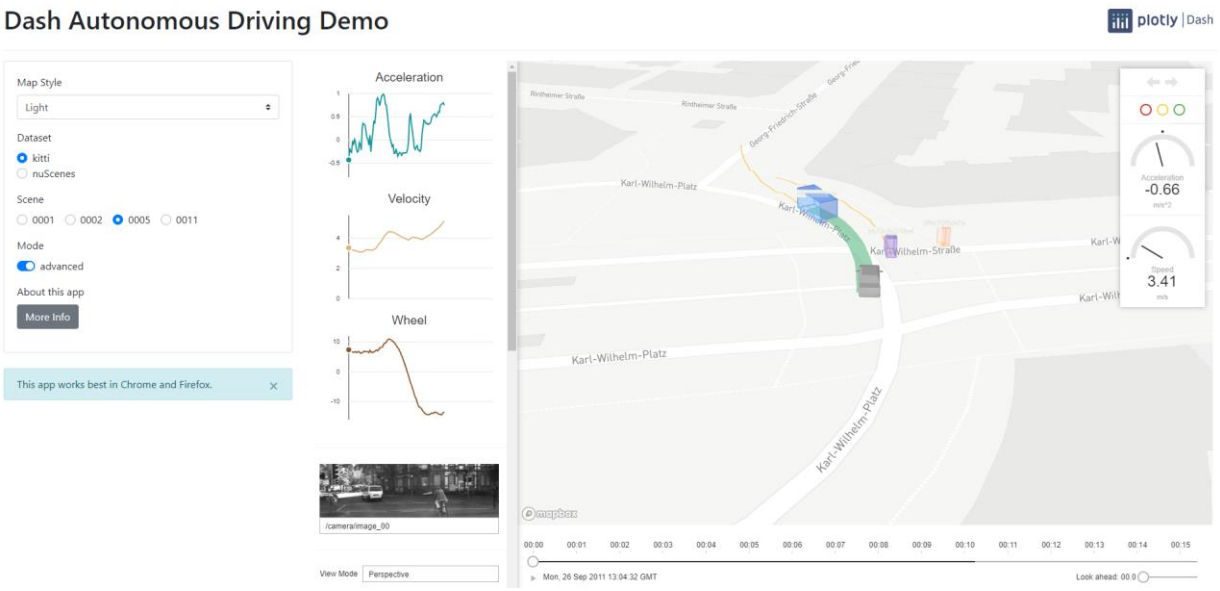


Figura 8.2 Interficie gràfica amb representació de paràmetres més visual (Font: [62])

8.2.1. Reconeixement de senyals i parada automàtica

El reconeixement de senyals i la parada automàtica són característiques importants per al desenvolupament de vehicles autònoms. Aquestes funcions permeten al vehicle circular amb major seguretat en entorns complexos, evitant accidents i controlant aspectes com la velocitat màxima de la via.

En primer lloc el reconeixement de senyals és clau, perquè permet al vehicle detectar i interpretar els senyals de trànsit com les senyals de stop i les línies de circulació. Gràcies a això, el vehicle podrà prendre decisions informades i circular amb major seguretat.

En segon lloc la parada automàtica en obstacles o situacions de risc o desconeixença, és inclús més important. Això no només millora la seguretat sinó que també pot evitar danys materials i reduir el risc de col·lisions. En el cas d'un vehicle de passatgers amb questes qualitats, pot evitar danys personals que d'una altre forma no s'haurien evitat.

8.2.2. Estat de la bateria: SoC(*State Of Charge*) i SoH (*State Of Health)*

El reconeixement del *state of charge* (SoC) i el *state of health* (SoH) d'una bateria és un factor rellevant en qualsevol dispositiu que disposi de bateria. Aquestes funcions permeten monitoritzar l'estat de la bateria i prendre decisions informades per assegurar una operació segura i eficient. La implementació d'aquests valors en la interfície gràfica podria permetre un millor control sobre el comportament del vehicle. Aquests valors poden suposar de gran ajuda en la competició de vehicles autònoms, canviant el comportament de més agressiu a més conservador si el SoC disminueix excessivament. Aquest comportament s'observa en les competicions de més alt nivell com F1 o Fòrmula E.

El SoH permet al vehicle monitoritzar l'estat de salut de la bateria. Això permet la detecció prematura de problemes a la bateria abans que esdevinguin problemes greus. Recordar que les fallades en bateries d'Ió Liti, que són les més utilitzades avui dia, poden comportar incendis molt perillosos i que no es poden apagar.

9. Anàlisi de l'impacte ambiental

L'impacte ambiental és l'efecte que les activitats humanes tenen sobre el medi ambient, tant positivament com negativament. Amb el transcurs del dia a dia, de forma directe o indirecte la societat produeix un impacte en el medi ambient.

Actualment ens trobem en una situació de crisi mediambiental que esta provocant entre d'altres efectes, l'augment de la temperatura global i l'empitjorament de la qualitat de l'aire que respirem. És primordial entendre que cada petita acció que es realitza com a individu pot tenir conseqüències devastadores a llarg termini. Per aquest motiu existeixen multitud d'organitzacions, campanyes i projectes per demanar responsabilitat social, per a poder trobar una solució a aquest problema.

En relació a aquest projecte, el JetRacer Pro AI Kit de Waveshare, com qualsevol altre dispositiu electrònic té un impacte en el medi ambient a causa dels materials utilitzats en la seva producció i eliminació. Aquests elements a tenir en compte són el xassís del cotxe, la càmera, el kit de desenvolupament Jetson Nano, el controlador del motor, la font d'alimentació i els motors.

El xassís del vehicle i altres components estan fets de plàstic, metall i altres materials que poden ser difícils de reciclar. Els components electrònics poden contenir materials tòxics com el plom, elmercuri o el cadmi, que poden ser nocius per al medi ambient si no s'eliminen correctament.

Tanmateix, és important tenir en compte que l'impacte ambiental es pot minimitzar mitjançant pràctiques de producció i eliminació responsables. L'ús d'aquest kit en un entorn acadèmic on fer un ús exhaustiu, proporciona oportunitats d'aprenentatge i desenvolupament que poden arribar a superar els possibles impactes negatius. Un altre factor a tenir en compte és el consum energètic, ja que el vehicle consumeix energia elèctrica, aquesta en la mesura del possible hauria de provenir de fonts sostenibles.

Conclusions

Aquest treball final de grau ha comprovat la viabilitat de dotar un vehicle RC amb la capacitat de circular amb un alt nivell d'autonomia sense intervenció externa. Per a dur a terme aquesta tasca, el vehicle fa ús d'una càmera i una placa Jetson Nano de Nvidia, que posseeix una CPU i una GPU amb potència suficient per realitzar els càlculs i processar les dades necessàries.

Per a la realització d'aquest projecte s'ha buscat la major similitud possible amb els entorns de treball dels vehicles autònoms d'avui dia, utilitzant les llibreries mencionades anteriorment. Com es descriu més amunt, actualment Tesla fa ús exclusivament d'un sistema de càmeres, processant aquestes imatges mitjançant visió per computador i emprant xarxes neuronals per a que el vehicle realitzi prediccions amb la finalitat de decidir per on ha de circular. El fet de fer servir una xarxa neuronal com ResNet18, i d'integrar-la amb altres eines com ara Python, JupyterLab, OpenCV, PyTorch i TensorRT, ha permès la creació d'un sistema que és capaç de circular de forma autònoma, evitant obstacles i regulant la seva velocitat sense intervenció externa.

Així mateix cal destacar la importància de les dades emprades per a l'entrenament de la xarxa neuronal. És preferible l'ús de dades representatives, prioritzant la qualitat enfront de la quantitat. Tanmateix, un major nombre dades implicarà un millor desenvolupament un cop realitzat l'entrenament. Cal considerar que un major nombre de dades també implica un major temps d'entrenament, fet que implica esperar el temps que requereixi l'entrenament abans de realitzar altres tasques, com per exemple validar si el model creat funciona correctament.

Un altre factor que ha suposat un gran repte a l'hora de verificar el funcionament dels models al circuit, el que anomenaríem "testeig", ha sigut la problemàtica de la potència que es capaç de brindar la Jetson Nano. Si be es tracta d'un dispositiu potent i capaç, a mesura que l'algorisme emprat es torna més pesat per les funcionalitats extra s'aprecia una disminució elevada en el rendiment del vehicle. Aquesta dificultat ha portat a la disminució de l'entrada de dades que la càmera subministra, obligant a modificar els "fps" subministrats de 30 inicialment, a uns 15-20.

Aquest treball final de grau posa de manifest la importància de la recerca en aquest camp i el potencial que té l'aprenentatge automàtic per a la creació de solucions innovadores en diferents àmbits. Queda vigent el potencial de millora de que disposa aquest projecte, amb un gran el ventall de possibilitats per a la formació i investigació en treballs futurs.

La implementació d'altres tècniques de IA és plenament viable i pot implicar descobrir nous camins per a millores que encara avui desconeixem. Com bé s'exposa a l'apartat 3 d'aquest projecte, l'evolució de la IA passa per la unió i implementació en conjunt de diversos tipus d'IA.

En conclusió, els objectius establerts s'han complert tal com s'esperava, i gràcies a haver tingut la possibilitat de realitzar aquest projecte, he pogut treballar i investigar en el que possiblement acabi sent el meu àmbit professional.

Pressupost i/o Anàlisi Econòmic

A continuació es mostren els costos derivats de la realització d'aquest projecte, dividits en mà d'obra i material utilitzat:

Components:

El total dels components que s'han requerit en aquest projecte és única i exclusivament el kit comprat. Aquest kit consta de tot l'indispensable per a dur a terme el treball sense necessitar material o components extra. Per aquest motiu l'únic cost computable com a material o components és el següent:

- JetRacer Pro Ai Kit by Waveshare: 455 €

Mà d'obra:

El desglossament referent a la mà d'obra necessària per a dur a terme aquest projecte s'ha aproximat amb el cost de contractar un enginyer junior el total d'hores invertides. En aquest cas s'ha considerat que el cost que suposa la contractació és de 15€/h.

Segons la guia docent de la UPC, la realització d'un treball final de grau atribueix o reconeix l'assoliment de competències equivalent a 24 crèdits o ECTS. Tenint en compte que cada crèdit implica una dedicació de 25h, s'estima que la dedicació requerida per a dur a terme aquesta labor ha sigut de 600h. Tanmateix, les hores dedicades a aquest projecte durant el transcurs d'aquest últim quadrimestre ascendeixen a un total de 670h aproximadament.

- La facturació de 670h de contractació d'enginyer junior a 15€/h té un cost total de: 10.050€

Total:

La realització d'aquest projecte finalment ha tingut un cost de: **10.505€**

Bibliografia

[1] R. Slim, A. Sharaf, J. Slim, and S. Tanveer, Udemy. "*The Complete Self-Driving Car Course - Applied Deep Learning*" A URL: https://www.udemy.com/course/applied-deep-learningtm-the-complete-self-driving-car-course/ (darrer accés 8 Septembre 2021)

[2] NVIDIA Deep Learning Institute. Courses, "*Getting Started with AI on Jetson Nano*" A URL: https://courses.nvidia.com/courses/course-v1:DLI+S-RX-02+V2/ (darrer accés 17 Octubre 2022).

[3] R. Slim, A. Sharaf, J. Slim, and S. Tanveer. Certificate- Udemy. "*The Complete Self-Driving Car Course - Applied Deep Learning*" A URL: https://www.udemy.com/certificate/UC-6dab5c73-6f3d-430f-b32b-7b4fd0a5f77a/ (darrer accés 8 Septembre 2021).

[4] Will Ramey. Certificates–Nvidia. "*Getting Started with AI on Jetson Nano*" A URL: https://courses.nvidia.com/certificates/f5ee50f5fbff487aa1056157e1731ae2/ (darrer accés 17 Octubre 2022)

[5] S. Pursell, (2021) , "*Metodología Agile: qué es y cómo aplicarla a tu proyecto*" A URL: https://blog.hubspot.es/marketing/metodologia-agile (darrer accés 20 Juliol 2022)

[6] K. Trapani, Cprime (2018), "*What is Agile and What is Scrum?*" A URL: https://www.cprime.com/resources/what-is-agile-what-is-scrum/ (darrer accés 20 Juliol 2022)

[7] Bernard Marr (2021), "*Understanding the 4 Types of Artificial intelligence*" A URL: https://bernardmarr.com/understanding-the-4-types-of-artificial-intelligence/ (darrer accés 21 Setembre 2022).

[8] JavaTpoint, "*Artificial Intelligence*" A URL: https://www.javatpoint.com/types-of-artificial-intelligence (darrer accés 21 Setembre 2022).

[9] SAS, "*Inteligencia Artificial: Qué es y Por Qué Importa*" A URL: https://www.sas.com/es_cl/insights/analytics/what-is-artificial-intelligence.html (darrer accés 21 Setembre 2022)

[10] A. Biswal, *Simplilearn* (2020), "*7 Types of Artificial Intelligence That You Should Know in 2023*" A URL: https://www.simplilearn.com/tutorials/artificial-intelligence-tutorial/types-of-artificial-intelligence (darrer accés 21 Setembre 2022).

[11] S. Betz, Built In (2022), "*4 Types of Artificial Intelligence*" A URL: https://builtin.com/artificial-intelligence/types-of-artificial-intelligence (darrer accés 21 Setembre 2022).

[12] R. Reynoso, G2 (2022), "*7 Major Types of AI That Can Bolster Your Decision Making*"
A URL: https://www.g2.com/articles/types-of-artificial-intelligence
(darrer accés 21 Setembre 2022).

[13] J. Johnson, bmcblogs (2020), "*4 Types of Artificial Intelligence*"
A URL: https://www.bmc.com/blogs/artificial-intelligence-types/
(darrer accés 21 Setembre 2022).

[14] SAE International (2021), "*Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles"*
A URL: https://www.sae.org/standards/content/j3016_202104/ (darrer accés 9 Setembre 2022)

[15] SAE International (2021), "*SAE Levels of Driving Automation™ Refined for Clarity and International Audience"* A URL: https://www.sae.org/blog/sae-j3016-update
(darrer accés 10 9 Setembre 2022)

[16] Babak Shahian Jahromi (2019), "*Ultrasonic Sensors in Self-Driving Cars*"
A URL: https://medium.com/@BabakShah/ultrasonic-sensors-in-self-driving-cars-d28b63be676f
(darrer accés 9 Setembre 2022)

[17] Intellias Mobility (2018), "*How Sensor Fusion for Autonomous Cars Helps Avoid Deaths on the Road*" A URL: https://intellias.com/sensor-fusion-autonomous-cars-helps-avoid-deaths-road/
(darrer accés 14 Setembre 2022)

[18]Wikipedia, *"Automotive night vision"* A URL:
https://en.wikipedia.org/wiki/Automotive_night_vision#/media/File:Audi_A8_2013_(11209949525).jpg. (darrer accés 14 Setembre 2022)

[19] *Caricos "Audi A7 US-Version (2012) Night Vision Display"*
A URL: https://www.caricos.com/cars/a/audi/2012_audi_a7_us-version/images/25.html.
(darrer accés 14 Setembre 2022)

[20] E. Roth, The Verge (2022) , "*Waymo can now provide fully driverless rides in San Francisco*"
A URL: https://www.theverge.com/2022/11/19/23467784/waymo-provide-fully-driverless-rides-san-francisco-california (darrer accés 14 Setembre 2022)

[21] A. Davies, WIRED (2020), "*Waymo's Self-Driving Jaguars Arrive With New, Homegrown Tech*" A URL:https://www.wired.com/story/waymos-self-driving-jaguars-arrive-new-homegrown-tech/ (darrer accés 14 Setembre 2022)

[22] Dr. S. Elisabeth, C. Malaquin, *(2018),* "*How Will Radar Sensor Technology Shape Cars of the Future?"* A URL:https://www.electronicdesign.com/markets/automotive/article/21806443/how-will-radar-sensor-technology-shape-cars-of-the-future (darrer accés 20 Desembre 2022)

[23] A. Al-Hourani, S. Kandeepan, B. Moran, (2016) , "*Stochastic Geometry Methods for Modeling Automotive Radar Interference*" A URL: https://www.researchgate.net/publication/305119388_Stochastic_Geometry_Methods_for_Modeling_Automotive_Radar_Interference (darrer accés 20 Desembre 2022).

[24] MOHIT SHARMA, Counterpoint Research (2022) , "*LiDAR Now High on Automotive Industry Radar*" A URL: https://www.counterpointresearch.com/lidar-now-high-automotive-industry-radar/ (darrer accés 3 Octubre 2022).

[25] M. Sachdev, RGBSI(2018), "*What is LiDAR? The eyes of self-driving vehicles*" A URL: https://blog.rgbsi.com/lidar-eyes-of-self-driving-vehicles (darrer accés 3 Octubre 2022).

[26] M. Lampinen, Automotive World, (2020),"*LiDARs for self-driving vehicles: a technological arms race*" A URL: https://www.automotiveworld.com/articles/lidars-for-self-driving-vehicles-a-technological-arms-race/ (darrer accés 3 Octubre 2022).

[25] Km77, "*Conducción autónoma*" *A URL:* https://www.km77.com/reportajes/varios/conduccion-autonoma-niveles (darrer accés 20 Desembre 2022).

[26] P. Ibáñez, Xataka, (2017), "*De 0 a 5: cuáles son los diferentes niveles de conducción autónoma, a fondo"* A URL: https://www.xataka.com/automovil/de-0-a-5-cuales-son-los-diferentes-niveles-de-conduccion-autonoma (darrer accés 20 Desembre 2022).

[27] Cochesquevansolos.com, (2021), "*Los niveles de autonomía del coche autónomo*" A URL: https://cochesquevansolos.com/informacion/niveles-autonomia-coche-autonomo/ (darrer 20 Desembre 2022).

[28] Synopsys Automotive, "The 6 Levels of Vehicle Autonomy Explained"
A URL: https://www.synopsys.com/automotive/autonomous-driving-levels.html
(darrer accés 24 Octubre 2022).

[29] K. Podbucki and T. Marciniak, Annals of Computer Science and Information Systems (2022). "*Aspects of autonomous drive control using NVIDIA Jetson Nano microcomputer*,"
A URL: http://dx.doi.org/10.15439/2022f89 (darrer accés 8 Desembre 2022).

[30] S. Bianco, R. Cadene, L. Celona, and P. Napoletano, "*Benchmark Analysis of Representative Deep Neural Network Architectures*," A URL: https://ieeexplore.ieee.org/document/8506339
(darrer accés 22 Setembre 2022).

[31] Anukrati Mehta, Digital Vidya (2019), "*A Comprehensive Guide to Types of Neural Networks"*
A URL: https://www.digitalvidya.com/blog/types-of-neural-networks/
(darrer accés 22 Setembre 2022).

[32] A. Pai, Analytics Vidhya, (2020), "*ANN vs CNN vs RNN*"
A URL: https://www.analyticsvidhya.com/blog/2020/02/cnn-vs-rnn-vs-mlp-analyzing-3-types-of-neural-networks-in-deep-learning/ (darrer accés 22 Setembre 2022).

[33] A. Tch, Towards Data Science (2017), "*The mostly complete chart of Neural Networks, explained"*
A URL: https://towardsdatascience.com/the-mostly-complete-chart-of-neural-networks-explained-3fb6f2367464 (darrer accés 23 Setembre 2022).

[34] Seldon (2022) "*Neural Network Models Explained*" A URL: https://www.seldon.io/neural-network-models-explained (darrer accés 30 Novembre 2022).

[35] P. Vadapalli, UpGrad (2020), *"7 Types of Neural Networks in Artificial Intelligence Explained"*
A URL: https://www.upgrad.com/blog/types-of-neural-networks/
(darrer accés 30 Novembre 2022).

[36] P. Narasimman (2022), *"9 Types of Neural Networks: Applications, Pros, and Cons"*
A URL: https://www.knowledgehut.com/blog/data-science/types-of-neural-networks
(darrer accés 30 Novembre 2022).

[37] G. L. Team, Great Learning Blog (2022), *"Types of Neural Networks and Definition of Neural Network*" A URL: https://www.mygreatlearning.com/blog/types-of-neural-networks/
(darrer accés 30 Novembre 2022).

[38] NVIDIA, "*NVIDIA Jetson Nano*" A URL: https://www.nvidia.com/es-es/autonomous-machines/embedded-systems/jetson-nano/product-development/
(darrer accés 23 Desembre 2022).

[39] Amazon.com "*NVIDIA Kit de desarrollador Jetson AGX Xavier (32GB)"*ha]. A URL: https://www.amazon.com/-/es/NVIDIA-desarrollador-Jetson-Xavier-32GB/dp/B083ZL3X5B.
(darrer accés 23 Desembre 2022).

[40] NVIDIA "*Implanta máquinas autónomas con IA a escala*"
A URL: https://www.nvidia.com/es-es/autonomous-machines/embedded-systems/jetson-agx-xavier/ (darrer accés 23 Desembre 2022).

[41] Coral, "*Dev Board*" A URL: https://coral.ai/products/dev-board/
(darrer accés 23 Desembre 2022).

[42] Coral, "Dev Board datasheet," A URL: https://coral.ai/docs/dev-board/datasheet/#overview
(darrer accés 23 Desembre 2022).

[43] *Raspberry Pi,*, "*Raspberry Pi 4 Model B specifications – Raspberry Pi*"
https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/
(darrer accés 23 Desembre 2022).

[44] NVIDIA, "*NVIDIA Jetson TX2: IA de alto rendimiento en primera línea*"
A URL: https://www.nvidia.com/es-es/autonomous-machines/embedded-systems/jetson-tx2/
(darrer accés 23 Desembre 2022).

[45] Hardware Setup, "*JetBot"* A URL: https://jetbot.org/master/hardware_setup.html
(darrer accés 23 Desembre 2022).

[46] TRITON AI "*Team projects – TRITON AI*" A URL: https://tritonai.org/teams/
(darrer accés 23 Desembre 2022).

[47] Donkey® Car "*Donkey Car*" A URL: https://www.donkeycar.com/
(darrer accés 23 Desembre 2022).

[48] WPL RC Official Store *"D12 Kei Truck - RTR - Silver/White"*
A URL: https://www.wpl-rc.com/products/d-12 (darrer accés 14 Gener 2023).

[49] Full Scale (2019), "*An Overview of OpenCV"*
A URL: https://fullscale.io/blog/opencv-overview/ (darrer accés 10 Gener 2023).

[50] K. Kargın, MLearning.ai (2021), "*Computer Vision Fundamentals and OpenCV Overview - MLearning.ai - Medium*" A URL: https://medium.com/mlearning-ai/computer-vision-fundamentals-and-opencv-overview-9a30fe94f0ce (darrer accés 10 Gener 2023).

[51] LearnOpenCV – Learn OpenCV, PyTorch, Keras, Tensorflow with examples and tutorials.
"*LearnOpenCV – Learn OpenCV, PyTorch, Keras, Tensorflow with examples and tutorials*"
A URL: https://learnopencv.com/ (darrer accés 19 Octubre 2022).

[52] K. Yasar and S. Lewis, TechTarget (2022) "*PyTorch*,".
A URL: https://www.techtarget.com/searchenterpriseai/definition/PyTorch
(darrer accés 19 Octubre 2022).

[53] KeepCoding Tech School *(2022),* "*KeepCoding Tech School*"
A URL: https://keepcoding.io/blog/para-que-sirve-tensorflow/
(darrer accés 19 Octubre 2022).

[54] NVIDIA Developer (2016), "*NVIDIA TensorRT*,".
A URL: https://developer.nvidia.com/tensorrt (darrer accés 21 Novembre 2022).

[55] NumPy "*What is NumPy? — NumPy v1.24 Manual."*
A URL: https://numpy.org/doc/stable/user/whatisnumpy.html
(darrer accés 21 Novembre 2022).

[56] Waveshare Wiki "*JetRacer Pro AI Kit*"
A URL: https://www.waveshare.com/wiki/JetRacer_Pro_AI_Kit
(darrer accés 2 Gener 2022).

[57] "JetRacer Pro AI Kit, High Speed AI Racing Robot Powered by Jetson Nano, Pro Version." A URL:
https://www.waveshare.com/jetracer-pro-ai-kit.htm?sku=18433
(darrer accés 2 Gener 2022).

[58] Google Docs, "*jetracer_pro_ws.zip*" A URL:
https://drive.google.com/file/d/1bgCAUJ9m16g5FGuYgKy3WmGI-pGjPXN0/view
(darrer accés 20 Juliol 2022).

[59] Widget List — Jupyter Widgets 7.6.5 documentation.
A URL: https://ipywidgets.readthedocs.io/en/7.6.5/examples/Widget%20List.html
(darrer accés 13 Setembre 2022).

[60] Python Functions. A URL: https://www.w3schools.com/python/python_functions.asp
(darrer accés 13 Setembre 2022).

[61] HAROONHUBLIKAR12, *"GitHub - haroonhublikar12/Self-driving-Car-Using-Machine-Learning-and-Neural-Networks"* A URL: https://github.com/haroonhublikar12/Self-driving-Car-Using-Machine-Learning-and-Neural-Networks (darrer accés 13 Setembre 2022).

[62] balenaEtcher *"balenaEtcher - Flash OS images to SD cards & USB drives*"
A URL: https://www.balena.io/etcher/ (darrer accés 20 Juliol 2022).

[63] Gamepad Tester "*Gamepad Tester and Debugger"* A URL: https://html5gamepad.com
(darrer accés 25 Juliol 2022).

[64] Arducam Wiki *"Fix Red Tint with ISP Tuning"*
A URL: https://docs.arducam.com/Nvidia-Jetson-Camera/Application-note/Fix-Red-Tint-with-ISP-Tuning/ (darrer accés 5 Octubre 2022).

[65] Autonomous Vehicle Visualization. A URL:https::/dash.gallery/dash-avs-explorer/
(darrer accés 14 Gener 2023).

Annexes

A1. Circuits de proves



Figura A.1 Imatge del circuit de proves inicial (Font: pròpia)



Figura A.2 Imatges del circuit de proves final, laboratori 5.4 de la EEBE (Font: pròpia)

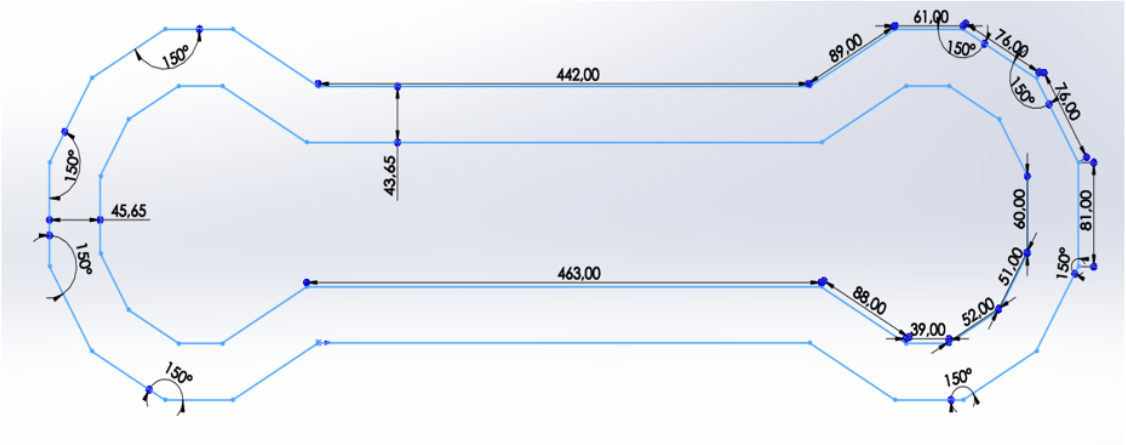


Figura A.3 Dimensions del circuit de proves final (Font: pròpia)

A2. Fitxa tècnica i muntatge del kit



Figura A.4 Components i instruccions de muntatge de JetRacer Pro AI Kit by Waveshare (Font: [56])

A3. Arxiu original *basic_motion*

Execute the following block of code by selecting it and clicking `ctrl + enter` to create an `NvidiaRacecar` class.

[1]:
```
from jetracer.nvidia_racecar import NvidiaRacecar

car = NvidiaRacecar()
```

The `NvidiaRacecar` implements the `Racecar` class, so it has two attributes `throttle` and `steering`.

We can assign values in the range `[-1, 1]` to these attributes. Execute the following to set the steering to 0.4.

> If the car does not respond, it may still be in `manual` mode. Flip the manual override switch on the RC transmitter.

[2]:
```
car.steering = 0
```

The `NvidiaRacecar` class has two values `steering_gain` and `steering_bias` that can be used to calibrate the steering.

We can view the default values by executing the cells below.

[3]:
```
car.steering_gain = 0.05
print(car.steering_gain)
```
```
0.05
```

[4]:
```
print(car.steering_offset)
```
```
0.0
```

The final steering value is computed using the equation

$$y = a \times x + b$$

Where,

- a is `car.steering_gain`
- b is `car.steering_offset`
- x is `car.steering`
- y is the value written to the motor driver

You can adjust these values calibrate the car so that setting a value of `0` moves forward, and setting a value of `1` goes fully right, and `-1` fully left.

To set the throttle of the car to `0.2`, you can call the following.

> Give JetRacer lots of space to move, and be ready on the manual override, JetRacer is *fast*

[5]:
```
car.throttle = 0
```

The throttle also has a gain value that could be used to control the speed response. The throttle output is computed as

$$y = a \times x$$

Where,

- a is `car.throttle_gain`
- x is `car.throttle`
- y is the value written to the speed controller

Execute the following to print the default gain

[6]:
```
print(car.throttle_gain)
```
```
0.8
```

Set the following to limit the throttle to half

[7]:
```
car.throttle_gain = 0.2
```

Please note the throttle is directly mapped to the RC car. When the car is stopped and a negative throttle is set, it will reverse. If the car is moving forward and a negative throttle is set, it will brake.

That's it for this notebook!

A4. Arxiu original *teleoperation*

Teleoperation

In this example we'll control the Jetbot remotely with a gamepad controller connected to our web browser machine.

Create gamepad controller

The first thing we want to do is create an instance of the `Controller` widget, which we'll use to drive our robot. The `Controller` widget takes a `index` parameter, which specifies the number of the controller. This is useful in case you have multiple controllers attached, or some gamepads *appear* as multiple controllers. To determine the index of the controller you're using,

1. Visit http://html5gamepad.com.
2. Press buttons on the gamepad you're using
3. Remember the `index` of the gamepad that is responding to the button presses

Next, we'll create and display our controller using that index.

[1]:
```
import ipywidgets.widgets as widgets

controller = widgets.Controller(index=0)  # replace with index of your controller

display(controller)
```

```
Controller()
```

Even if the index is correct, you may see the text `Connect gamepad and press any button`. That's because the gamepad hasn't registered with this notebook yet. Press a button and you should see the gamepad widget appear above.

Connect gamepad controller to robot motors

Now, even though we've connected our gamepad, we haven't yet attached the controls to our robot! The first, and most simple control we want to attach is the motor control. We'll connect that to the left and right vertical axes using the `dlink` function. The `dlink` function, unlike the `link` function, allows us to attach a transform between the `source` and `target`. Because the controller axes are flipped from what we think is intuitive for the motor control, we'll use a small *lambda* function to negate the value.

> WARNING: This next cell will move the robot if you touch the gamepad controller axes!

[2]:
```
from jetracer.nvidia_racecar import NvidiaRacecar
import traitlets

car = NvidiaRacecar()

car.throttle_gain = 0.2
```

[3]:
```
car.steering_offset=0
car.steering = 0
```

[4]:
```
left_link = traitlets.dlink((controller.axes[0], 'value'), (car, 'steering'), transform=lambda x: x)
right_link = traitlets.dlink((controller.axes[3], 'value'), (car, 'throttle'), transform=lambda x: -x)
```

Awesome! Our robot should now respond to our gamepad controller movements. Now we want to view the live video feed from the camera!

A5. Arxiu original *interactive_regressión*

Camera

[1]:
```
from jetcam.csi_camera import CSICamera
# from jetcam.usb_camera import USBCamera

camera = CSICamera(width=224, height=224)
#camera = USBCamera(width=224, height=224)

camera.running = True
```

Task

[2]:
```
import torchvision.transforms as transforms
from xy_dataset import XYDataset

TASK = 'road_following'

CATEGORIES = ['apex']

DATASETS = ['A', 'B']

TRANSFORMS = transforms.Compose([
    transforms.ColorJitter(0.2, 0.2, 0.2, 0.2),
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

datasets = {}
for name in DATASETS:
    datasets[name] = XYDataset(TASK + '_' + name, CATEGORIES, TRANSFORMS, random_hflip=True)
```

Data Collection

[3]:
```
import cv2
import ipywidgets
import traitlets
from IPython.display import display
from jetcam.utils import bgr8_to_jpeg
from jupyter_clickable_image_widget import ClickableImageWidget


# initialize active dataset
dataset = datasets[DATASETS[0]]

# unobserve all callbacks from camera in case we are running this cell for second time
camera.unobserve_all()

# create image preview
camera_widget = ClickableImageWidget(width=camera.width, height=camera.height)
snapshot_widget = ipywidgets.Image(width=camera.width, height=camera.height)
traitlets.dlink((camera, 'value'), (camera_widget, 'value'), transform=bgr8_to_jpeg)

# create widgets
dataset_widget = ipywidgets.Dropdown(options=DATASETS, description='dataset')
category_widget = ipywidgets.Dropdown(options=dataset.categories, description='category')
count_widget = ipywidgets.IntText(description='count')

# manually update counts at initialization
count_widget.value = dataset.get_count(category_widget.value)

# sets the active dataset
def set_dataset(change):
    global dataset
    dataset = datasets[change['new']]
    count_widget.value = dataset.get_count(category_widget.value)
dataset_widget.observe(set_dataset, names='value')

# update counts when we select a new category
def update_counts(change):
    count_widget.value = dataset.get_count(change['new'])
category_widget.observe(update_counts, names='value')


def save_snapshot(_, content, msg):
    if content['event'] == 'click':
        data = content['eventData']
        x = data['offsetX']
        y = data['offsetY']

        # save to disk
        dataset.save_entry(category_widget.value, camera.value, x, y)

        # display saved snapshot
        snapshot = camera.value.copy()
        snapshot = cv2.circle(snapshot, (x, y), 8, (0, 255, 0), 3)
        snapshot_widget.value = bgr8_to_jpeg(snapshot)
        count_widget.value = dataset.get_count(category_widget.value)

camera_widget.on_msg(save_snapshot)

data_collection_widget = ipywidgets.VBox([
    ipywidgets.HBox([camera_widget, snapshot_widget]),
    dataset_widget,
    category_widget,
    count_widget
])

display(data_collection_widget)
```

Model

[4]:
```
import torch
import torchvision

device = torch.device('cuda')
output_dim = 2 * len(dataset.categories)  # x, y coordinate for each category

# ALEXNET
model = torchvision.models.alexnet(pretrained=True)
model.classifier[-1] = torch.nn.Linear(4096, output_dim)

# SQUEEZENET
# model = torchvision.models.squeezenet1_1(pretrained=True)
# model.classifier[1] = torch.nn.Conv2d(512, output_dim, kernel_size=1)
# model.num_classes = len(dataset.categories)

# RESNET 18
# model = torchvision.models.resnet18(pretrained=True)
# model.fc = torch.nn.Linear(512, output_dim)

# RESNET 34
# model = torchvision.models.resnet34(pretrained=True)
# model.fc = torch.nn.Linear(512, output_dim)

# DENSENET 121
# model = torchvision.models.densenet121(pretrained=True)
# model.classifier = torch.nn.Linear(model.num_features, output_dim)

model = model.to(device)

model_save_button = ipywidgets.Button(description='save model')
model_load_button = ipywidgets.Button(description='load model')
model_path_widget = ipywidgets.Text(description='model path', value='road_following_model.pth')

def load_model(c):
    model.load_state_dict(torch.load(model_path_widget.value))
model_load_button.on_click(load_model)

def save_model(c):
    torch.save(model.state_dict(), model_path_widget.value)
model_save_button.on_click(save_model)

model_widget = ipywidgets.VBox([
    model_path_widget,
    ipywidgets.HBox([model_load_button, model_save_button])
])


display(model_widget)
```

Live Execution

[5]:
```
import threading
import time
from utils import preprocess
import torch.nn.functional as F

state_widget = ipywidgets.ToggleButtons(options=['stop', 'live'], description='state', value='stop')
prediction_widget = ipywidgets.Image(format='jpeg', width=camera.width, height=camera.height)

def live(state_widget, model, camera, prediction_widget):
    global dataset
    while state_widget.value == 'live':
        image = camera.value
        preprocessed = preprocess(image)
        output = model(preprocessed).detach().cpu().numpy().flatten()
        category_index = dataset.categories.index(category_widget.value)
        x = output[2 * category_index]
        y = output[2 * category_index + 1]

        x = int(camera.width * (x / 2.0 + 0.5))
        y = int(camera.height * (y / 2.0 + 0.5))

        prediction = image.copy()
        prediction = cv2.circle(prediction, (x, y), 8, (255, 0, 0), 3)
        prediction_widget.value = bgr8_to_jpeg(prediction)

def start_live(change):
    if change['new'] == 'live':
        execute_thread = threading.Thread(target=live, args=(state_widget, model, camera, prediction_widget))
        execute_thread.start()

state_widget.observe(start_live, names='value')

live_execution_widget = ipywidgets.VBox([
    prediction_widget,
    state_widget
])

display(live_execution_widget)
```

[6]:
```
BATCH_SIZE = 8

optimizer = torch.optim.Adam(model.parameters())
# optimizer = torch.optim.SGD(model.parameters(), lr=1e-3, momentum=0.9)

epochs_widget = ipywidgets.IntText(description='epochs', value=1)
eval_button = ipywidgets.Button(description='evaluate')
train_button = ipywidgets.Button(description='train')
loss_widget = ipywidgets.FloatText(description='loss')
progress_widget = ipywidgets.FloatProgress(min=0.0, max=1.0, description='progress')

def train_eval(is_training):
    global BATCH_SIZE, LEARNING_RATE, MOMENTUM, model, dataset, optimizer, eval_button, train_button, accuracy_widget, loss_widget, progress_widget, state_widget

    try:
        train_loader = torch.utils.data.DataLoader(
            dataset,
            batch_size=BATCH_SIZE,
            shuffle=True
        )

        state_widget.value = 'stop'
        train_button.disabled = True
        eval_button.disabled = True
        time.sleep(1)

        if is_training:
            model = model.train()
        else:
            model = model.eval()

        while epochs_widget.value > 0:
            i = 0
            sum_loss = 0.0
            error_count = 0.0
            for images, category_idx, xy in iter(train_loader):
                # send data to device
                images = images.to(device)
                xy = xy.to(device)

                if is_training:
                    # zero gradients of parameters
                    optimizer.zero_grad()

                # execute model to get outputs
                outputs = model(images)

                # compute MSE loss over x, y coordinates for associated categories
                loss = 0.0
                for batch_idx, cat_idx in enumerate(list(category_idx.flatten())):
                    loss += torch.mean((outputs[batch_idx][2 * cat_idx:2 * cat_idx+2] - xy[batch_idx])**2)
                loss /= len(category_idx)

                if is_training:
                    # run backpropogation to accumulate gradients
                    loss.backward()

                    # step optimizer to adjust parameters
                    optimizer.step()

                # increment progress
                count = len(category_idx.flatten())
                i += count
                sum_loss += float(loss)
                progress_widget.value = i / len(dataset)
                loss_widget.value = sum_loss / i

            if is_training:
                epochs_widget.value = epochs_widget.value - 1
            else:
                break
    except e:
        pass
    model = model.eval()

    train_button.disabled = False
    eval_button.disabled = False
    state_widget.value = 'live'

train_button.on_click(lambda c: train_eval(is_training=True))
eval_button.on_click(lambda c: train_eval(is_training=False))

train_eval_widget = ipywidgets.VBox([
    epochs_widget,
    progress_widget,
    loss_widget,
    ipywidgets.HBox([train_button, eval_button])
])

display(train_eval_widget)
```

All together!

The following widget can be used to label a multi-class x, y dataset. It supports labeling only one instance of each class per image (ie: only one dog), but multiple classes (ie: dog, cat, horse) per image are possible.

Click the image on the top left to save an image of `category` to `dataset` at the clicked location.

Widget	Description
dataset	Selects the active dataset
category	Selects the active category
epochs	Sets the number of epochs to train for
train	Trains on the active dataset for the number of epochs specified
evaluate	Evaluates the accuracy on the active dataset over one epoch
model path	Sets the active model path
load	Loads a model from the active model path
save	Saves a model to the active model path
stop	Disables the live demo
live	Enables the live demo

[7]:
```
all_widget = ipywidgets.VBox([
    ipywidgets.HBox([data_collection_widget, live_execution_widget]),
    train_eval_widget,
    model_widget
])

display(all_widget)
```

```
VBox(children=(HBox(children=(VBox(children=(HBox(children=(ClickableImageWidget(value=b'\xff\xd8\xff\xe0\x00\…
```

[ ]:
```
camera.stop()
```

A6. Arxiu original *road_following*

First, create the model. This must match the model used in the interactive training notebook.

[1]:
```
import torch
import torchvision

CATEGORIES = ['apex']

device = torch.device('cuda')
model = torchvision.models.alexnet(pretrained=True)
model.classifier[-1] = torch.nn.Linear(4096, output_dim)
model = model.cuda().eval().half()
```

Next, load the saved model. Enter the model path you used to save.

[2]:
```
model.load_state_dict(torch.load('road_following_model_uni.pth'))
```

[2]:
```
<All keys matched successfully>
```

Convert and optimize the model using `torch2trt` for faster inference with TensorRT. Please see the torch2trt readme for more details.

> This optimization process can take a couple minutes to complete.

[3]:
```
from torch2trt import torch2trt

data = torch.zeros((1, 3, 224, 224)).cuda().half()

model_trt = torch2trt(model, [data], fp16_mode=True)
```

Save the optimized model using the cell below

[4]:
```
torch.save(model_trt.state_dict(), 'road_following_model_uni_trt.pth')
```

Load the optimized model by executing the cell below

[4]:
```
import torch
from torch2trt import TRTModule

model_trt = TRTModule()
model_trt.load_state_dict(torch.load('road_following_model_uni_trt.pth'))
```

[4]:
```
<All keys matched successfully>
```

Create the racecar class

[5]:
```
from jetracer.nvidia_racecar import NvidiaRacecar

car = NvidiaRacecar()
```

Create the camera class.

[6]:
```
from jetcam.csi_camera import CSICamera

camera = CSICamera(width=224, height=224)
#camera = CSICamera(width=224, height=224, capture_fps=65)
```

Finally, execute the cell below to make the racecar move forward, steering the racecar based on the x value of the apex.

Here are some tips,

- If the car wobbles left and right, lower the steering gain
- If the car misses turns, raise the steering gain
- If the car tends right, make the steering bias more negative (in small increments like -0.05)
- If the car tends left, make the steering bias more postive (in small increments +0.05)

[8]:
```
from utils import preprocess
import numpy as np

STEERING_GAIN = 0.6
STEERING_BIAS = 0.0

car.throttle = 0.0

while True:
    image = camera.read()
    image = preprocess(image).half()
    output = model_trt(image).detach().cpu().numpy().flatten()
    x = float(output[0])
    car.steering = x * STEERING_GAIN + STEERING_BIAS
```

A7. Arxiu road_following_V2

Road Following + manual throttling

[1]:
```
import torch
from torch2trt import TRTModule

model_trt = TRTModule()
model_trt.load_state_dict(torch.load('road_following_model_V2_trt.pth'))
```

•••

[2]:
```
from jetracer.nvidia_racecar import NvidiaRacecar

car = NvidiaRacecar()
```

[3]:
```
from jetcam.csi_camera import CSICamera

camera = CSICamera(width=224, height=224, capture_fps=30)
```

[4]:
```
import traitlets
from IPython.display import display
from ipywidgets import Layout, Button, Box
import ipywidgets.widgets as widgets

network_output_slider = widgets.FloatSlider(description='Network Output', min=-1.0, max=1.0, value=0, step=0.01, orientation='horizontal', disabled=False, layout={'width': '4
steering_gain_slider  = widgets.FloatSlider(description='Steering Gain', min=-1.0, max=1.0, value=-0.7, step=0.01, orientation='horizontal', layout={'width': '300px'})
steering_bias_slider  = widgets.FloatSlider(description='Steering Bias', min=-0.5, max=0.5, value=0.0, step=0.01, orientation='horizontal', layout={'width': '300px'})
steering_value_slider = widgets.FloatSlider(description='Steering', min=-1.0, max=1.0, value=0, step=0.01, orientation='horizontal', disabled=False, layout={'width': '400px'
throttle_slider = widgets.FloatSlider(description='Throttle', min=0.0, max=0.2, value=0.0, step=0.01, orientation='vertical')
#  throttle_slider value should be [-1,1] set to [0,0.2] for prevention

steering_gain_link   = traitlets.link((steering_gain_slider, 'value'), (car, 'steering_gain'))
steering_offset_link = traitlets.link((steering_bias_slider, 'value'), (car, 'steering_offset'))
steering_value_link  = traitlets.link((steering_value_slider, 'value'), (car, 'steering'))
throttle_slider_link = traitlets.link((throttle_slider, 'value'), (car, 'throttle'))

display(
    widgets.HBox(
        [widgets.VBox([network_output_slider,
                       widgets.Label(value="X"),
                       steering_gain_slider,
                       widgets.Label(value="+"),
                       steering_bias_slider,
                       widgets.Label(value="||"),
                       steering_value_slider], layout=Layout(align_items='center')
                      ),
         throttle_slider]
            )
        )
```

[5]:
```
from utils import preprocess
from jetcam.utils import bgr8_to_jpeg

def update(change):
    global blocked_slider, robot
    new_image = change['new']

    image = preprocess(new_image).half()
    output = model_trt(image).detach().cpu().numpy().flatten()
    x = float(output[0])
    y = float(output[0])

    network_output_slider.value = x
    steering = x * steering_gain_slider.value + steering_bias_slider.value
    if(steering<-1.0):
        steering_value_slider.value = -1.0
    elif(steering>1.0):
        steering_value_slider.value = 1.0
    else:
        steering_value_slider.value = steering
    car.steering = x

update({'new': camera.value})
camera.observe(update, names='value')
camera.running = True
```

A8. Arxiu road_following_V3

Road Following + dynamic throttling

[3]:
```
import torch
from torch2trt import TRTModule

model_trt = TRTModule()
model_trt.load_state_dict(torch.load('road_following_model_uni_V3_trt.pth'))
```

[3]:
```
<All keys matched successfully>
```

[4]:
```
from jetracer.nvidia_racecar import NvidiaRacecar

car = NvidiaRacecar()
```

[5]:
```
from jetcam.csi_camera import CSICamera

camera = CSICamera(width=224, height=224, capture_fps=15)
```

[6]:
```
import traitlets
from IPython.display import display
from ipywidgets import Layout, Button, Box
import ipywidgets.widgets as widgets

network_output_slider = widgets.FloatSlider(description='Network Output', min=-1.0, max=1.0, value=0, step=0.01, orientation='horizontal', disabled=False, layout={'width': '400px'})
steering_gain_slider  = widgets.FloatSlider(description='Steering Gain', min=-1.0, max=1.0, value=-0.65, step=0.01, orientation='horizontal', layout={'width': '300px'})
steering_bias_slider  = widgets.FloatSlider(description='Steering Bias', min=-0.5, max=0.5, value=0.0, step=0.01, orientation='horizontal', layout={'width': '300px'})
steering_value_slider = widgets.FloatSlider(description='Steering', min=-1.0, max=1.0, value=0, step=0.01, orientation='horizontal', disabled=False, layout={'width': '400px'})
throttle_slider = widgets.FloatSlider(description='Throttle', min=0.0, max=0.2, value=0.0, step=0.01, orientation='vertical')
#  throttle_slider value should be [-1,1] set to [0,0.2] for prevention

steering_gain_link   = traitlets.link((steering_gain_slider, 'value'), (car, 'steering_gain'))
steering_offset_link = traitlets.link((steering_bias_slider, 'value'), (car, 'steering_offset'))
#steering_value_link  = traitlets.link((steering_value_slider, 'value'), (car, 'steering'))
throttle_slider_link = traitlets.link((throttle_slider, 'value'), (car, 'throttle'))
```

[7]:
```
from utils import preprocess
from jetcam.utils import bgr8_to_jpeg

def update(change):
    global blocked_slider, robot
    new_image = change['new']

    image = preprocess(new_image).half()
    output = model_trt(image).detach().cpu().numpy().flatten()
    x = float(output[0])
    y = float(output[0])

    network_output_slider.value = x
    steering = x * steering_gain_slider.value + steering_bias_slider.value

    if(steering<-1.0):
        steering_value_slider.value = -1.0
    elif(steering>1.0):
        steering_value_slider.value = 1.0
    else:
        steering_value_slider.value = steering
    car.steering = x


    if (steering<=0.3 and steering>=-0.3 and throttle_slider.value<=0.16 and throttle_slider.value>0.12):
        throttle_slider.value += 0.01
    elif ((steering>0.3 or steering<-0.3) and throttle_slider.value>0.14):
        throttle_slider.value += -0.01
    else:
        throttle_slider.value = throttle_slider.value


update({'new': camera.value})
camera.observe(update, names='value')
camera.running = True
```

[9]:
```
display(
    widgets.HBox(
        [widgets.VBox([network_output_slider,
                       widgets.Label(value="X"),
                       steering_gain_slider,
                       widgets.Label(value="+"),
                       steering_bias_slider,
                       widgets.Label(value="||"),
                       steering_value_slider], layout=Layout(align_items='center')
                      ),
         throttle_slider]
            )
        )
```

A9. Arxiu road_following_V4

Road Following + dynamic throttling + prediction in real time

[1]:
```
import torch
from torch2trt import TRTModule

model_trt = TRTModule()
model_trt.load_state_dict(torch.load('road_following_model_uni_V4_trt.pth'))
```

...

[2]:
```
from jetracer.nvidia_racecar import NvidiaRacecar

car = NvidiaRacecar()
```

[3]:
```
from jetcam.csi_camera import CSICamera

camera = CSICamera(width=224, height=224, capture_fps=15)
```

[18]:
```
import cv2
import traitlets
import ipywidgets
from IPython.display import display
from ipywidgets import Layout, Button, Box
import ipywidgets.widgets as widgets

state_widget = ipywidgets.ToggleButtons(options=['On', 'Off'], description='Camera', value='On')
prediction_widget = ipywidgets.Image(format='jpeg', width=camera.width, height=camera.height)

live_execution_widget = ipywidgets.VBox([
    prediction_widget,
    state_widget
],layout=Layout(align_items='center'))

network_output_slider = widgets.FloatSlider(description='Network Out', min=-1.0, max=1.0, value=0, step=0.01, orientation='horizontal', disabled=False, layout={'width': '300px'})
steering_gain_slider  = widgets.FloatSlider(description='Steering Gain', min=-1.0, max=1.0, value=-0.65, step=0.01, orientation='horizontal', layout={'width': '300px'})
steering_bias_slider  = widgets.FloatSlider(description='Steering Bias', min=-0.5, max=0.5, value=0.0, step=0.01, orientation='horizontal', layout={'width': '300px'})
steering_value_slider = widgets.FloatSlider(description='Steering', min=-1.0, max=1.0, value=0, step=0.01, orientation='horizontal', disabled=False, layout={'width': '300px'})
throttle_slider = widgets.FloatSlider(description='Throttle', min=0.0, max=0.2, value=0.0, step=0.01, orientation='vertical',layout={'height': '250px'})
#  throttle_slider value should be [-1,1] set to [0,0.2] for prevention
speed_slider = widgets.IntSlider(description='Speed mode', min=0, max=8, value=0, step=1, orientation='vertical', layout={'height': '250px'})

steering_gain_link   = traitlets.link((steering_gain_slider, 'value'), (car, 'steering_gain'))
steering_offset_link = traitlets.link((steering_bias_slider, 'value'), (car, 'steering_offset'))
#steering_value_link  = traitlets.link((steering_value_slider, 'value'), (car, 'steering'))
throttle_slider_link = traitlets.link((throttle_slider, 'value'), (car, 'throttle'))

button = widgets.Button(description='Stop', button_style='danger')
out = widgets.Output()

def on_button_clicked(_):
    with out:
        speed_slider.value = 0
button.on_click(on_button_clicked)
```

[9]:
```
from utils import preprocess
from jetcam.utils import bgr8_to_jpeg

def update(change):
    global blocked_slider, robot
    new_image = change['new']

    image = preprocess(new_image).half()
    output = model_trt(image).detach().cpu().numpy().flatten()
    x = float(output[0])
    y = float(output[0])

    network_output_slider.value = x
    steering = x * steering_gain_slider.value + steering_bias_slider.value
    if(steering<-1.0):
        steering_value_slider.value = -1.0
    elif(steering>1.0):
        steering_value_slider.value = 1.0
    else:
        steering_value_slider.value = steering
    car.steering = x

    if(state_widget.value == 'On'):
        x = int(camera.width * (x / 2.0 + 0.5))
        y = int(camera.height * (y / 2.0 + 0.5))
        prediction = new_image.copy()
        prediction = cv2.circle(prediction, (x, y), 8, (255, 0, 0), 3)
        prediction_widget.value = bgr8_to_jpeg(prediction)

    # Values below  may be personalized on personal preferences. Steering and speed values can be changed to fit purpose.
    steering_max = 0.13
    steering_min = -0.13
```

```
    if speed_slider.value == 0:
        throttle_slider.value = 0

    elif speed_slider.value == 1:
        if (steering<=steering_max and steering>=steering_min and throttle_slider.value<0.14 and throttle_slider.value>0.11):
            throttle_slider.value += 0.001
        elif((steering>steering_max or steering<steering_min) and throttle_slider.value>0.12):
            throttle_slider.value += - 0.001

    elif speed_slider.value == 2:
        if (steering<=steering_max and steering>=steering_min and throttle_slider.value<0.15 and throttle_slider.value>0.11):
            throttle_slider.value += 0.001
        elif((steering>steering_max or steering<steering_min) and throttle_slider.value>0.13):
            throttle_slider.value += - 0.001

    elif speed_slider.value == 3:
        if (steering<=steering_max and steering>=steering_min and throttle_slider.value<0.15 and throttle_slider.value>0.11):
            throttle_slider.value += 0.001
        elif((steering>steering_max or steering<steering_min) and throttle_slider.value>0.14):
            throttle_slider.value += - 0.001

    elif speed_slider.value == 4:
        if (steering<=steering_max and steering>=steering_min and throttle_slider.value<0.16 and throttle_slider.value>0.11):
            throttle_slider.value += 0.001
        elif((steering>steering_max or steering<steering_min) and throttle_slider.value>0.15):
            throttle_slider.value += - 0.001

    elif speed_slider.value == 5:
        if (steering<=steering_max and steering>=steering_min and throttle_slider.value<0.17 and throttle_slider.value>0.11):
            throttle_slider.value += 0.001
        elif((steering>steering_max or steering<steering_min) and throttle_slider.value>0.16):
            throttle_slider.value += - 0.001

    elif speed_slider.value == 6:
        if (steering<=steering_max and steering>=steering_min and throttle_slider.value<0.18 and throttle_slider.value>0.11):
            throttle_slider.value += 0.001
        elif((steering>steering_max or steering<steering_min) and throttle_slider.value>0.17):
            throttle_slider.value += - 0.001

    elif speed_slider.value == 7:
        if (steering<=steering_max and steering>=steering_min and throttle_slider.value<0.19 and throttle_slider.value>0.11):
            throttle_slider.value += 0.001
        elif((steering>steering_max or steering<steering_min) and throttle_slider.value>0.18):
            throttle_slider.value += - 0.001

    elif speed_slider.value == 8:
        if (steering<=steering_max and steering>=steering_min and throttle_slider.value<0.2 and throttle_slider.value>0.11):
            throttle_slider.value += 0.001
        elif((steering>steering_max or steering<steering_min) and throttle_slider.value>0.19):
            throttle_slider.value += - 0.001

    else:
        throttle_slider.value = throttle_slider.value

update({'new': camera.value})
camera.observe(update, names='value')
camera.running = True
```

[19]:
```
display(
    widgets.HBox(
        [widgets.VBox([network_output_slider,
                       widgets.Label(value="X"),
                       steering_gain_slider,
                       widgets.Label(value="+"),
                       steering_bias_slider,
                       widgets.Label(value="||"),
                       steering_value_slider,
                       button,out,], layout=Layout(align_items='center')
                      ),
         throttle_slider,
         speed_slider,
         live_execution_widget
         ]
               )
        )
```